NSSECU3 Digital Forensics AY2526T2

Laboratory Activity Report (LAR) / Digital Forensics Report (DFR) / Mini Project (MP) / Write-Up (WUP) / Windows Forensics Project (WFP)

MINI PROJECT 1

Document Number:	MP-01
Approved By:	Jerome E. Gutierrez NSSECU3 Course Instructor
Date Approved:	18 February 2026

Submitted By:	Miguel Sebastian C. Carlos
Group Members:	Luis Gabriel S. Biacora Jose Angelo C. Calpoporo Miguel Sebastian C. Carlos Daniel Gavrie Y. Clemente
Date Version:	18 February 2026
Version:	1.0

©Copyright 2026, m1rr4wr
All Rights Reserved
Any hard copy or unprotected soft copy of this document shall be regarded as uncontrolled copy.

Document Type: MP = Mini Project

Data Classification: Confidential: Students of this course, Professors of the Department, and other guest speakers and students may use this document and view the contents.

Table of Contents

Task Distribution Summary 4
- Change Control 4
- Task List 4

Member's Task Lists 7
- Detailed Task List by Biacora 7
- Detailed Task List by Calpoporo 7
- Detailed Task List by Carlos 9
- Detailed Task List by Clemente 10

Purpose 11

Scope 11

Definitions and Acronyms 13

Roles and Responsibilities 16
- Project Lead / Main Contributor 16
- Evidence Acquisition and Imaging Analyst 16
- DF Workstation Analyst 17
- Malware Scanner Developer 18
- YARA Rule Research and AI Comparison Analyst 19
- Documentation and Reporting Analyst 20

Materials and Equipment 21
- Virtual Machines and Hardware Resources 21
  - Victim-Target Virtual Machine 21
  - Digital Forensics (DF) Workstation 22
- Software Tools 23
  - FTK Imager (Standalone Version) 23
  - Arsenal Image Mounter (AIM) 23
  - Python Programming Environment 24
  - YARA and yara-python Library 25
- Supporting Tools and Resources 26
  - VMware Workstation / VMware Player 26
  - VMware Shared Folders 27

Additional Supporting Resources 28
Summary of Materials 29
Procedure Steps 29
1. Project Initialization and Evidence Preparation 29
2. Forensic Acquisition of Malware Samples 30
3. Forensic Mounting and Evidence Access 31
4. Malware Scanner Design and Development 32
5. YARA Rule Generation and AI-Based Evaluation 39
6. Malware Scanning and Quarantine Execution 50
7. Results Compilation and Evaluation 51
8. Documentation and Reporting 51
Safety or Compliance Considerations (if applicable) 53
References 54
Appendices 54
Appendix A: Virtual Machine Environment Setup 54
Appendix B: Malware Sample Dataset Inventory 56
Appendix C: Forensic Disk Image Artifacts 56
Appendix D: Forensic Mounting Artifacts 57
Appendix E: Malware Scanner Architecture and Source Modules 57
Appendix F: YARA Rule Generation and AI Comparison 62
Appendix G: Quarantine and Reporting Artifacts 62
Appendix H: Tool Limitations and Observations 62
Appendix I: YARA Rule 63
Appendix J: Development Artifacts 85
Appendix K: Reports and Results 98
Appendix L: Complete Scan Results 106
Additional Sections 139
1. Observations and Analysis 139
2. Tool and Methodological Limitations 145
3. Forensic and Operational Considerations 147
4. Implications for Future Work 150
5. Conclusions 153

Task Distribution Summary

Description	This document summarizes the complete forensic workflow for MP1 Malware Scanner project. The project encompasses evidence acquisition from Victim VM, forensic disk imaging, mounting via Arsenal Image Mounter, development of a custom Python-based malware scanner, generation and comparison of YARA rules from four AI platforms (ChatGPT, Deepseek, Gemini, Copilot), full scan execution on 220 files, quarantine of detected malware,
Start of Activity	01 February 2026
End of Activity	18 February 2026
Main Contributor	Jose Angelo C. Calpoporo

Change Control

Version	Changes	Date	Changed by
0.1	Initial document creation and template setup	01 Feb 26	Clemente, Daniel Gavrie
0.2	Added Purpose and Scope sections	03 Feb 26	Calpoporo, Jose Angelo C.
0.3	Completed Definitions, Roles, Materials sections	05 Feb 26	Carlos, Miguel Sebastian C.
0.4	Drafted Procedure Steps 1-4 (Evidence, Acquisition, Mounting, Scanner Dev)	07 Feb 26	Carlos, Miguel Sebastian C.
0.5	Drafted Procedure Steps 5-6 (YARA AI Generation, Scanning)	09 Feb 26	Calpoporo, Jose Angelo C.
0.6	Drafted Procedure Steps 7-8 (Evaluation, Documentation)	10 Feb 26	Calpoporo, Jose Angelo C.
0.7	Added Phase Diagram and Flowchart (Draw.io)	11 Feb 26	Calpoporo, Jose Angelo C.
0.8	Populated AI Comparison Matrix with metrics from test scans	12 Feb 26	Calpoporo, Jose Angelo C.
0.9	Integrated all screenshots and labeled per Appendix G	12 Feb 26	Carlos, Miguel Sebastian C. Calpoporo, Jose Angelo C.
1.0	Final grammar review, compliance check, PDF export	16 Feb 26	Biacora, Luis Gabriel S. Clemente, Daniel Gavrie

Task List

#	Item	Purpose	Assignee	Status
1	VMware Workstation 17 Pro Installation	To provide virtualization platform for hosting Victim and DF Workstation VMs	Calpoporo	Done
2	Victim VM (Windows 10 22H2) Creation	To create the evidence source machine containing the 220 malware files	Carlos	Done
3	DF Workstation VM (Windows 11 23H2) Creation	To create the forensic analysis machine for mounting and scanning	Calpoporo	Done
4	Victim VM Tools Installation	Install FTK Imager 4.7.3 on Victim VM for disk imaging	Carlos	Done
5	DF Workstation Tools Installation	Install Python 3.13.2, VSCode, AIM, yara-python, pefile, colorama	Calpoporo	Done

6	220 Files Acquisition	Download "Files" folder from NSSECU3 OneDrive (File001-File220)	Carlos	Done
7	Victim VM Pre-Acquisition Snapshot	Create forensic snapshot "MP1_Pre_Acquisition_20260213"	Carlos	Done
8	Forensic Image Creation	Create E01 image of 220-file folder using FTK Imager "Contents of a Folder"	Carlos	Done
9	Image Integrity Verification	Record and verify MD5/SHA1 hashes from FTK Imager	Carlos	Done
10	Image Transfer to DF Workstation	Transfer victim_220_samples.E01 via VMware Shared Folders	Carlos	Done
11	Image Re-verification on DF Workstation	Verify transferred image hashes match original	Carlos	Done
12	Image Mounting with AIM	Mount E01 as physical SCSI disk (writable mode) on DF Workstation	Calpoporo	Done
13	Drive Letter Documentation	Document assigned drive letter (F:) and verify 220 files accessible	Calpoporo	Done
14	Python Scanner Development - Core Structure	Create main.py, scanner.py, __init__.py	Calpoporo	Done
15	Python Scanner Development - Magic Number Detector	Implement magic_detector.py using Gary Kessler signature table	Calpoporo	Done
16	Python Scanner Development - Hash Calculator	Implement hash_utils.py (MD5, SHA1, SHA256)	Calpoporo	Done
17	Python Scanner Development - YARA Handler	Implement yara_handler.py (compile, scan, error handling)	Calpoporo	Done
18	Python Scanner Development - Quarantine Manager	Implement quarantine.py (.quarantine, .meta, copy-only)	Calpoporo	Done
19	Python Scanner Development - Reporter	Implement reporter.py (JSON, CSV export)	Calpoporo	Done
20	Python Scanner Unit Testing	Test each module with known file types	Calpoporo	Done
21	YARA Rule Generation - ChatGPT	Generate YARA rules using ChatGPT with standardized prompt	Calpoporo	Done
22	YARA Rule Generation - Deepseek	Generate YARA rules using Deepseek with standardized prompt	Calpoporo	Done
23	YARA Rule Generation - Gemini	Generate YARA rules using Gemini with standardized prompt	Carlos	Done
24	YARA Rule Generation - Copilot	Generate YARA rules using Copilot with standardized prompt	Carlos	Done
25	YARA Rule Syntax Validation	Test compile all 4 .yar files with yara-python	Calpoporo	Done

26	AI Comparative Scanning - ChatGPT	Scan mounted drive with ChatGPT rules, record metrics	Calpoporo	Done
27	AI Comparative Scanning - Deepseek	Scan mounted drive with Deepseek rules, record metrics	Calpoporo	Done
28	AI Comparative Scanning - Gemini	Scan mounted drive with Gemini rules, record metrics	Carlos	Done
29	AI Comparative Scanning - Copilot	Scan mounted drive with Copilot rules, record metrics	Carlos	Done
30	AI Comparison Matrix Compilation	Populate comparison table with detection rates, false positives, readability scores	Calpoporo	
31	Best Rule Set Selection	Select winner based on highest true positives, lowest false positives, best practices	Calpoporo	DONE
32	Combined Rules File Creation	Merge all 4 rule sets into ai_combined.yar	Calpoporo	Done
33	Final Scan Execution	Run full scan on mounted drive with combined rules + quarantine	Calpoporo	Done
34	Quarantine Verification	Verify .quarantine and .meta files created correctly	Calpoporo	Done
35	Scan Log Capture	Save console output to scan_log.txt	Calpoporo	Done
36	Results Analysis	Calculate summary statistics, detection rates, performance metrics	Calpoporo	Done
37	Phase Diagram Creation	Create high-level system architecture diagram (8 phases)	Clemente	Done
38	Flowchart Creation	Create scanner decision logic flowchart	Clemente	Done
39	Screenshot Capture and Labeling	Capture all 18 required screenshots, label per Appendix G	Clemente	Done
40	AI Comparison Report Section	Write detailed analysis of AI performance and winner justification	Calpoporo	Done
41	Source Code Appendix Compilation	Concatenate all .py and .yar files into appendix_source_code.txt	Calpoporo	Done
42	Final Compliance Check	Verify against project rubric checklist	Biacora	Done
43	PDF Export	Export final document as LAR-02_MP1_Group3.pdf	Clemente	Done
44	Submission	Upload to NSSECU3 submission portal	Carlos	Done

Member's Task Lists

Detailed Task List by Biacora

#	Item	Purpose	Section added	Status
1	VMware Download	To have a software for virtualization	Materials and Equipment	Done
2	Virtual Machine Windows Creation	To create a Windows VM for the group to share and be the victim machine	Appendix A	Done
3	Documentation and Completion	Compile all group contributions into unified forensic report	Entire Document	Done
4	Final Grammar and Compliance Review	Ensure report meets academic and forensic documentation standards	Entire Document	Done
5	Report Formatting and Word Export	Convert final report into submission-ready Word format	Final Submission Section	Done

Detailed Task List by Calpoporo

#	Item	Purpose	Section added	Status
1	DF Workstation VM (Windows 11 23H2) Creation	Create dedicated forensic analysis machine	Appendix A	Done
2	DF Workstation Tools Installation	Install Python 3.13.2, VSCode, AIM, yara-python, pefile, colorama	Materials and Equipment	Done
3	Image Mounting with AIM	Mount E01 as physical SCSI disk on DF Workstation with writable mode	3.Forensic Mounting and Evidence Access Appendix D	Done
4	Drive Letter Documentation	Document F:\ assignment and verify 220 file accessibility	3.Forensic Mounting and Evidence Access	Done
5	Python Scanner - Core Architecture	Design modular architecture with 6 core modules + entry point	4.Malware Scanner Design and Development Appendix E.1	Done
6	Python Scanner - Magic Number Detector	Implement 30+ signatures from Gary Kessler table	4.Malware Scanner Design and Development Appendix E.1	Done
7	Python Scanner - Hash Calculator	Implement MD5, SHA1, SHA256 with chunked reading	4.Malware Scanner Design and Development Appendix E.1	Done
8	Python Scanner - YARA Handler	Implement yara-python integration with error handling	4.Malware Scanner Design and Development	Done

			Appendix E.1	
9	Python Scanner - Quarantine Manager	Implement forensic quarantine (copy only, .quarantine, .meta)	4.Malware Scanner Design and Development Appendix G	Done
10	Python Scanner - Scanner Engine	Implement recursive directory walking, parallel processing, result aggregation	4.Malware Scanner Design and Development	Done
11	Python Scanner - Reporter	Implement JSON and CSV export	4.Malware Scanner Design and Development	Done
12	Python Scanner - Entry Point	Implement CLI with argparse (target, rules, quarantine, threads, output)	4.Malware Scanner Design and Development	
13	Unit Testing	Test all modules with known file types (test.pdf, test.exe, test.docx)	4.Malware Scanner Design and Development	Done
14	YARA Rule Generation - ChatGPT	Generate 12+ YARA rules using standardized prompt	5. YARA Rule Generation and AI-Based Evaluation Appendix F	Done
15	YARA Rule Generation - Deepseek	Generate 12+ YARA rules using standardized prompt	5. YARA Rule Generation and AI-Based Evaluation Appendix F	Done
16	YARA Rule Syntax Validation	Test compile all 4 .yar files, document errors/corrections	5. YARA Rule Generation and AI-Based Evaluation	Done
17	AI Comparative Scanning - ChatGPT	Execute full scan with ChatGPT rules, record detection count	5. YARA Rule Generation and AI-Based Evaluation	Done
18	AI Comparative Scanning - Deepseek	Execute full scan with Deepseek rules, record detection count	5. YARA Rule Generation and AI-Based Evaluation	Done
19	Best Rule Set Selection	Analyze metrics and select winner	5. YARA Rule Generation and AI-Based Evaluation Appendix F	Done
20	Combined Rules File Creation	Merge all 4 rule sets into ai_combined.yar	6. Malware Scanning and Quarantine Execution	Done
21	Final Scan Execution	Run scanner with combined rules + quarantine-auto on F:\	6. Malware Scanning and Quarantine Execution	Done
22	Quarantine Verification	Verify .quarantine and .meta file creation	6. Malware Scanning and Quarantine Execution	Done
23	Results Analysis	Calculate summary statistics, detection rates, performance metrics	7. Results Compilation and Evaluation	Done

24	Project Lead Responsibilities	Overall timeline management, requirement compliance, final integration	Entire Document	Done
25	Purpose and Scope Authoring	Write comprehensive Purpose and Scope sections	Purpose Scope	Done

Detailed Task List by Carlos

#	Item	Purpose	Section added	Status
1	Victim VM (Windows 10 22H2) Creation	Create evidence source machine with 40GB disk, 4GB RAM	Appendix A.2	Done
2	Victim VM Tools Installation	Install FTK Imager 4.7.3 standalone	Materials and Equipment	Done
3	220 Files Acquisition	Download "Files" folder from NSSECU3 OneDrive, verify File001-File220	1. Project Initialization and Evidence Preparation Appendix B	Done
4	File Manifest Creation	Generate dir /b > file_manifest.txt	1. Project Initialization and Evidence Preparation	Done
5	Victim VM Pre-Acquisition Snapshot	Create "MP1_Pre_Acquisition_20260213" snapshot	1. Project Initialization and Evidence Preparation	Done
6	Forensic Image Creation	Create E01 image via FTK Imager "Contents of a Folder"	2. Forensic Acquisition of Malware Samples Appendix C	Done
7	Evidence Information Entry	Case Number: MP1-2026-001, Examiner: Carlos	2.Forensic Acquisition of Malware Samples	Done
8	Image Integrity Verification	Record MD5/SHA1 from verification dialog	2.Forensic Acquisition of Malware Samples	Done
9	Image Transfer to DF Workstation	Transfer victim_220_samples.E01 via VMware Shared Folders	2.Forensic Acquisition of Malware Samples	Done
10	Image Re-verification on DF Workstation	Verify hashes match original using FTK Imager "Verify Image"	2.Forensic Acquisition of Malware Samples	Done
11	YARA Rule Generation - Gemini	Generate 12+ YARA rules using standardized prompt	5. YARA Rule Generation and AI-Based Evaluation Appendix F	Done
12	YARA Rule Generation - Copilot	Generate 12+ YARA rules using standardized prompt	5. YARA Rule Generation and AI-Based Evaluation Appendix F	Done

13	AI Comparative Scanning - Gemini	Execute full scan with Gemini rules, record detection count	5. YARA Rule Generation and AI-Based Evaluation	Done
14	AI Comparative Scanning - Copilot	Execute full scan with Copilot rules, record detection count	5. YARA Rule Generation and AI-Based Evaluation	Done
15	Materials and Equipment Documentation	Document all hardware, software, versions, specifications	Materials and Equipment	Done
16	Definitions and Acronyms	Define technical terms used throughout document	Definitions and Acronyms	Done

Detailed Task List by Clemente

#	Item	Purpose	Section added	Status
1	Header and Footer Consistency	To have a software for virtualization	Entire Document	Done
2	Document Template Alignment	Verify document title, document number, date version, and classification are consistent throughout	Header and Footer	Done
3	Screenshot Labeling and Indexing	Ensure that all required screenshots are properly labeled and indexed.	Appendix C	Done
4	Grammar, Clarity, and Technical Consistency Review	Improve readability without altering technical meaning	Entire Document	Done

Purpose

The purpose of this mini project is to perform a structured and forensically sound malware analysis on a collection of 220 unknown files by applying fundamental digital forensics and static malware analysis techniques. This project aims to demonstrate proper evidence handling through disk imaging and forensic mounting, followed by the development of a custom Python-based malware scanner. The scanner utilizes file signature analysis through magic numbers, cryptographic hash generation (MD5 and SHA variants), and YARA rule matching to identify potentially malicious files. Additionally, the project evaluates and compares YARA rules generated by multiple artificial intelligence tools to determine their effectiveness and reliability in malware detection. Through quarantine handling and detailed reporting, this mini project reinforces core forensic principles such as evidence integrity, repeatability, and analytical documentation, while preparing students for more advanced malware analysis tasks in subsequent projects.

Scope

Boundaries

This procedure covers the **complete forensic workflow** for the MP1 Malware Scanner project, from initial evidence acquisition on the Victim Virtual Machine to final report generation on the Digital Forensics Workstation. The scope is strictly limited to:

Phase 1: Evidence Preparation

- Download and verification of exactly 220 files (File001-File220) from NSSECU3 OneDrive repository
- Organization of files into C:\Users\Victim\Documents\MP1_Files\
- Pre-acquisition snapshot of Victim VM with forensic naming convention
- File manifest generation using dir /b > file_manifest.txt

Phase 2: Forensic Acquisition

- Disk imaging using FTK Imager version 4.7.3
- **"Contents of a Folder"** acquisition method (NOT physical/logical drive imaging)
- E01 (EnCase evidence file format) with compression and integrity verification
- Evidence information documentation (Case Number, Evidence Number, Examiner, Notes)
- MD5 and SHA1 hash generation and verification at acquisition
- Image transfer to DF Workstation via VMware Shared Folders
- Post-transfer hash re-verification

Phase 3: Forensic Mounting

- Arsenal Image Mounter (AIM) version 3.7.281
- Mount as writable with differencing file (preserves original E01)
- Physical SCSI disk emulation
- Drive letter assignment (documentation of assigned letter)
- Verification of 220 file accessibility

Phase 4: Scanner Development

- Python 3.13.2 development environment

- Modular architecture with 6 core modules + entry point:
- magic_detector.py - Gary Kessler signature table (30+ signatures)
- hash_utils.py - MD5, SHA1, SHA256 with chunked reading
- yara_handler.py - YARA compilation, scanning, error handling
- quarantine.py - Forensic quarantine with .quarantine and .meta
- scanner.py - Recursive directory walking, parallel processing
- reporter.py - JSON and CSV export
- main.py - CLI entry point with argument parsing unit testing with known file types

Phase 5: YARA AI Generation and Comparison

- YARA rule generation from 4 AI platforms: ChatGPT, Deepseek, Gemini, Copilot
- Standardized prompt template with 8-12 rule requirement per AI
- Syntax validation using yara.compile()
- Comparative scanning on mounted drive with each rule set
- Metric collection: detection count, false positives, best practices adherence
- Winner selection and justification

Phase 6: Scanning and Quarantine

- Full recursive scan of all 220 files on mounted drive
- Combined YARA rule set execution
- **Automatic quarantine** of detected malware
- Quarantine naming: [OriginalFilename]_[First8MD5]_[YYYYMMDD_HHMMSS].quarantine
- Metadata file creation: .meta containing original path, timestamp, rules matched, all hashes
- JSON and CSV report generation

Phase 7: Results Analysis

- Summary statistics calculation
- File type categorization via magic numbers
- Detection rate computation
- Performance metrics (files/second)
- Limitations documentation

Phase 8: Documentation

- Phase Diagram (system architecture - 8 phases)
- Flowchart (scanner decision logic)
- 18 required screenshots with indexed labeling
- AI comparison matrix
- Complete source code appendix
- Pre-submission compliance checklist

EXCLUDED:

- **Live memory (RAM) acquisition** - This is outside MP1 scope and will be covered in MP2
- **Network traffic capture and analysis** - Not required for static malware analysis
- **Dynamic malware execution** - Samples are not to be executed on any system
- **Reverse engineering / disassembly** - Binary analysis is outside MP1 scope

- **Kernel-level forensic analysis** - Not required
- **Physical disk imaging** - Only "Contents of a Folder" method is used
- **Write operations to original evidence** - E01 remains read-only; all writes go to differencing file
- **Deletion or modification of original files** - Quarantine performs COPY only
- **Analysis on Victim VM** - All analysis MUST be performed on mounted image on DF Workstation

Who This Applies To

This document is intended for:

1. **Digital Forensic Analysts** performing static malware analysis on acquired disk images
2. **Incident Responders** conducting initial triage on suspicious file collections
3. **NSSECU3 Students** completing MP1 Malware Forensics project requirements
4. **Security Researchers** developing custom YARA-based scanning tools
5. **Forensic Tool Developers** implementing magic number detection and hash verification

Mandatory Requirements:

- All analysis MUST be performed on the **DF Workstation VM**
- The Victim VM is to be **snapshotted** and **powered off** after acquisition
- The original E01 file must remain **read-only** throughout the investigation
- All findings must be **reproducible** by following this document

Definitions and Acronyms

This section defines technical terms, abbreviations, and tools **as used and encountered during the mini project.**

Arsenal Image Mounter (AIM)

- Forensic software utility that mounts disk images (E01, dd, raw) as physical SCSI disks in Windows. Enables read-write access through differencing files while preserving original evidence integrity.

Argparse

- Python module used to parse command-line arguments in main.py. Implements -h/--help, target path, --rules, --quarantine, --threads, --output parameters.

CLI

- Command Line Interface. Text-based interface for executing the malware scanner via terminal/command prompt.

Colorama

- Python library for cross-platform colored terminal output. Used to display MALICIOUS (red) and CLEAN (green) status indicators. Version 0.4.6.

CSV

- Comma-Separated Values. Report format generated by reporter.py containing scan results in tabular format compatible with Microsoft Excel.

DF Workstation

- Digital Forensics Workstation. Dedicated Windows 11 Pro 23H2 virtual machine (80GB disk, 8GB RAM) where all forensic analysis, scanner development, and report generation occur. Evidence integrity is maintained by only accessing mounted read-only images.

Differencing File

- A temporary file created by AIM when "Mount as writable" is selected. All write operations (including quarantine file creation) are redirected to this file; the original E01 remains unmodified.

Draw.io

- Free online diagramming tool used to create Phase Diagram and Flowchart. Integrated with Google Drive.

E01

- EnCase evidence file format. A compressed, segmented forensic image format with built-in MD5/SHA1 hash verification and case metadata storage. Selected over raw/dd for smaller file size and integrity features.

FTK Imager

- Forensic imaging tool by Exterro/AccessData. Used to create forensically sound E01 images via "Contents of a Folder" acquisition method. Version 4.7.3.81.

Gary Kessler File Signatures Table

- Publicly accessible database of file header signatures (magic numbers). Primary reference for magic_detector.py implementation (Kessler, 2021).

haslib

- Python built-in module implementing MD5, SHA1, and SHA256 cryptographic hash functions. Used in hash_utils.py with chunked reading (8192 bytes) for memory efficiency.

JSON

- JavaScript Object Notation. Primary report format generated by reporter.py containing complete scan metadata and per-file results in structured format.

Magic Number

- A unique sequence of bytes, typically located at offset 0 of a file, used to identify file format regardless of extension. Examples: MZ (4D 5A) for PE executables, %PDF (25 50 44 46) for PDF documents. 30+ signatures implemented.

MD5

- Message Digest Algorithm 5. 128-bit cryptographic hash function producing 32-character hexadecimal string. Used for file integrity verification during acquisition and as unique identifier in quarantine filenames.

.meta

- Metadata file created alongside each .quarantine file. Contains original file path, quarantine timestamp, matched YARA rule names, MD5/SHA1/SHA256 hashes, and file size. Critical for audit trail.

OLE2

- Object Linking and Embedding 2. Compound file format used by older Microsoft Office documents (.doc, .xls). Magic number: D0 CF 11 E0 A1 B1 1A E1.

Open XML

- Modern Microsoft Office document format (.docx, .xlsx). ZIP-based container format. Magic number: PK 03 04.

PE

- Portable Executable. Windows executable file format (.exe, .dll). Identified by MZ header (4D 5A) at offset 0 and "PE\0\0" (50 45 00 00) at offset specified by 0x3C.

Pefile

- Python library for PE file parsing and analysis. Used in yara_handler.py for section analysis, entropy calculation, and import table inspection. Version 2024.8.26.

Quarantine

- Forensic isolation procedure that COPIES suspicious files to a secured directory (./quarantine/) while preserving original files on the mounted image. Quarantined files are renamed with .quarantine extension; accompanying .meta files store complete metadata.

SHA1

- Secure Hash Algorithm 1. 160-bit cryptographic hash function producing 40-character hexadecimal string. Used for image integrity verification.

SHA256

- Secure Hash Algorithm 2 (256-bit). 256-bit cryptographic hash function producing 64-character hexadecimal string. Project requirement: "hash should be MD5 and SHA1/SHA2".
- Python concurrent.futures module component. Implements parallel scanning with configurable worker threads (default: 4). Significantly reduces scan time for 220 files.

ThreadPoolExecutor

- Python concurrent.futures module component. Implements parallel scanning with configurable worker threads (default: 4). Significantly reduces scan time for 220 files.

Victim VM

- Windows 10 22H2 virtual machine (60GB disk, 4GB RAM) serving as the original evidence source. Contains the 220 files in C:\Users\Victim\Documents\MP1_Files. Snapshotted before acquisition and powered off after imaging.

VMware Shared Folders

- Mechanism for transferring files between host and guest VMs. Used to transfer victim

Roles and Responsibilities

Project Lead / Main Contributor

Jose Angelo C. Calpoporo

Oversees overall project execution and timeline

The project lead was responsible for creating and maintaining the master project schedule encompassing all eight procedural phases of the MP1 Malware Scanner project. A comprehensive task list was developed and tracked throughout the project duration. Daily team check-ins were conducted to ensure milestone adherence and to identify potential bottlenecks before they impacted deadlines. When technical challenges arose, including YARA compilation errors across multiple AI-generated rule sets and Python environment configuration issues on the DF Workstation, the project lead coordinated troubleshooting efforts and resource allocation. Priority adjustments were made dynamically when rule sets required manual syntax corrections, ensuring that necessary refinements did not delay the final scan execution.

Ensures compliance with NSSECU3 requirements

Throughout the project lifecycle, continuous verification was performed against the project rubric. All 220 files were confirmed to be scanned with no additional files accessed, maintaining the strict scope requirements. Cryptographic hash generation was verified for every file, with MD5, SHA1, and SHA256 outputs confirmed in both console output and JSON reports. The magic number detection implementation was validated against the Gary Kessler File Signatures Table, ensuring accurate file type identification. Quarantine operations were tested and confirmed to follow the required naming convention with the .quarantine extension and accompanying .meta files. All references and AI-generated content were properly cited in the final documentation.

Integrates all outputs into the final report

The integration phase involved compiling Python source code files into the appendix. Four AI-generated YARA rule sets from ChatGPT, Deepseek, Gemini, and Copilot were merged and formatted for documentation. Screenshots captured throughout the procedure were labeled and inserted at appropriate locations. Individual scan results from the four comparison scans were synthesized into a comprehensive comparative analysis matrix. All team member contributions were collected, reviewed, and integrated into a cohesive final document.

Evidence Acquisition and Imaging Analyst

Miguel Sebastian C. Carlos

Verifies the presence of exactly 220 files on the Victim VM

The evidence acquisition phase began with accessing the NSSECU3 OneDrive repository and downloading the complete folder containing all 220 malware samples. The files were organized into the designated forensic directory to establish a controlled evidence source. A complete file manifest was

generated to document the inventory of all evidence files. Manual verification confirmed exactly 220 files present, sequentially named File001 through File220. This verification step established the foundation for all subsequent forensic activities.

Creates and verifies a forensically sound disk image

With the evidence files verified, FTK Imager was launched on the Victim VM to begin the acquisition process. The "Contents of a Folder" acquisition method was selected as required by the project specifications, ensuring that only the designated 220 files were captured without including unnecessary system data. The source folder was selected, and the E01 format was chosen for its compression capabilities and built-in integrity verification features. The imaging process was initiated with verification enabled, creating a forensically sound logical image of all 220 evidence files.

Ensures evidence integrity through hash validation

Upon completion of the imaging process, FTK Imager automatically generated and displayed MD5 and SHA1 hash values for the acquired image. These cryptographic hashes were recorded as the baseline for all future integrity verification. The image file was transferred from the Victim VM to the DF Workstation using a controlled transfer mechanism. On the DF Workstation, FTK Imager's "Verify Image" function was executed against the transferred file, confirming that the MD5 and SHA1 hashes matched exactly with those generated during acquisition. This hash validation confirmed that the evidence had not been altered during transfer and remained forensically sound for analysis.

DF Workstation Analyst

Jose Angelo C. Calpoporo

Mounts the disk image using Arsenal Image Mounter (AIM)

With the forensic image securely transferred to the DF Workstation, Arsenal Image Mounter was downloaded from the official website. The application was launched with administrator privileges, a critical requirement for proper disk mounting functionality. The "Mount Disk Image" button was selected and navigation to the evidence file was performed. A key technical consideration was encountered at this stage: the default file filter in AIM does not display AD1 files. The filter was manually changed from "Arsenal Image Mounter Supported Files" to "All Files" to reveal the evidence image. The following mount options were carefully selected to ensure both evidence preservation and analysis capability: Disk device with write temporary mode, showing mounted disks in Disk Management, and enabling write support for quarantine operations. Following these selections, the image was successfully mounted and assigned a drive letter.

Verifies proper mounting and drive accessibility

After mounting, Windows Disk Management was opened to verify the successful attachment. The Arsenal Image Mounter disk appeared as a new disk, confirming proper physical disk emulation. The assigned drive letter was clearly visible. File Explorer navigation to the mounted drive confirmed the presence of all 220 evidence files, visible and accessible for analysis. PowerShell verification was performed to confirm the complete file listing. Screenshots were captured at each verification step, documenting the successful mounting and accessibility of the forensic evidence.

Ensures all analysis is performed on the mounted image only

Throughout the investigation, strict controls were maintained to ensure that all analysis activities were performed exclusively on the mounted forensic image. The scanner was configured with the target path pointing to the mounted evidence drive, directing all file access to the mounted evidence rather than the original image file. Write operations during quarantine were confirmed to be redirected to the differencing file created by AIM, leaving the original evidence image completely unmodified.

Malware Scanner Developer

Jose Angelo C. Calpoporo

Develops the Python-based malware scanner

A modular architecture was designed and implemented consisting of core Python files, each with distinct responsibilities. The scanner was developed on the DF Workstation using Python with the yara-python, colorama, and pefile libraries. The complete module structure included an entry point with argument parsing, a core scanning engine orchestrating the analysis workflow, a YARA handler managing rule compilation and scanning, a magic number detector for file signature identification, a hash calculator for cryptographic hash computation, a quarantine manager for forensic isolation, and a reporter for JSON and CSV export.

Implements magic number detection, hashing, YARA scanning, and quarantine

The magic number detector successfully identified file types across the 220-file corpus using the Gary Kessler File Signatures Table as reference. Files were classified as PE executables, PDF documents, Office Open XML files, JPEG images, PNG images, ASCII text, and binary data. Files with unrecognized headers were classified as binary data with low confidence, maintaining conservative classification.

Hash calculation was performed for every file using chunked reading approach with 8KB blocks, ensuring consistent performance regardless of file size. MD5, SHA1, and SHA256 were generated for all files using Python's hashlib library. File size was also recorded as additional forensic metadata.

YARA integration enabled pattern-based malware detection using compiled rule sets from all four AI platforms. The handler successfully compiled the combined rule set and performed matching on all 220 files without errors. Matched rule names and associated metadata were captured and included in the final reports.

The quarantine mechanism successfully isolated all detected malicious files during the final combined scan. Quarantined files were renamed following the convention of original filename, MD5 prefix, and timestamp with .quarantine extension. Corresponding metadata files were created containing original path, quarantine timestamp, matched YARA rules, and all three cryptographic hashes.

Ensures only the designated files are scanned

The scanner was explicitly configured to target only the mounted evidence path. Recursive directory walking was implemented with strict filtering to include only files, ensuring that directories and any other

non-evidence content were excluded from scanning. Verification after each scan confirmed that exactly 220 files were processed, matching the known evidence count.

YARA Rule Research and AI Comparison Analyst

Jose Angelo C. Calpoporo & Miguel Sebastian C. Carlos

Generates and validates YARA rules using multiple AI tools

A standardized prompt template was developed to ensure consistency across all four AI platforms. The prompt specified target file types including PE executables, PDF documents, Office documents, JavaScript, and PowerShell scripts. Rule requirements included magic number evaluation first, string specificity minimum of 6 characters, and comprehensive metadata.

Rule sets were generated from ChatGPT, Deepseek, Gemini, and Copilot. Each response was saved as a separate YARA file. Syntax validation was performed for each rule file using the YARA Python bindings. This validation revealed syntax errors in three of the four rule sets, requiring manual corrections. Deepseek required handling of unreferenced strings. Gemini required addition of an unreferenced string to condition logic. Copilot required correction of invalid operator syntax. All corrections were documented and the corrected rule sets were saved for comparative scanning.

Compares rule effectiveness, accuracy, and readability

Each validated rule set was executed independently against the mounted evidence using the scanner in comparison mode with quarantine disabled. Detection results were recorded and analyzed based on number of files flagged, adherence to YARA best practices, readability, and comment quality.

The ChatGPT scan detected files with PDF_Embedded_JavaScript rule. The Deepseek scan detected the highest number of files across all file types with the MALICIOUS_Backdoor_Persistence rule dominating detections. The Gemini scan detected only one file with the SUSPICIOUS_PE_HighEntropySections rule. The Copilot scan detected files with the MALICIOUS_PDF_EmbeddedFiles rule.

Selects the best-performing rule set for scanning

Based on comprehensive analysis of detection rates, adherence to YARA best practices, and documentation quality, the Deepseek rule set was selected as the optimal choice for the final scan. Key selection factors included the highest detection rate, excellent string specificity, best practices compliance after corrections, comprehensive documentation, and coverage across all file types present in the corpus.

Creates combined rule set

To maximize detection coverage for the final scan, all four validated rule sets were merged into a single comprehensive rule file using PowerShell type commands. The resulting combined file contained all rules from all four platforms. Compilation validation confirmed that the merged file compiled without errors. This combined rule set was used for the final scan with quarantine enabled.

Documentation and Reporting Analyst

Jose Angelo C. Calpoporo, Miguel Sebastian C. Carlos, Daniel Gavrie Y. Clemente, Luis Gabriel S. Biacora

Prepares diagrams, flowcharts, screenshots, and tables

The documentation team was responsible for creating all visual documentation elements. A Phase Diagram was developed illustrating the eight-phase system architecture from evidence preparation through final documentation. A Scanner Decision Logic Flowchart was created detailing the exact decision blocks implemented in the Python code, capturing the complete processing pipeline including path validation, rule compilation, magic number detection, hash calculation, YARA scanning, quarantine decisions, and report generation.

Screenshots were captured throughout the procedure documenting every critical step including file verification, FTK Imager operations, AIM mounting, scan execution, quarantine verification, and CSV generation. Each screenshot was accompanied by descriptive captions explaining the depicted procedure.

Compiles scan results and analysis findings

Scan results from all comparison scans and the final combined scan were aggregated. Detection statistics were calculated and organized into summary tables showing detection rates per AI platform and per file type. False positive observations were documented based on analysis of detections across file types. Limitations of the static analysis approach were clearly articulated.

Ensures proper formatting and completeness of the final document

Comprehensive document formatting review was performed against the template requirements. Header and footer consistency was verified throughout the document. Table formatting was standardized with proper numbering and captions. Cross-references to figures were validated to ensure all references resolved correctly. The document structure was checked for logical flow and compliance with required sections.

Oversees complete forensic report documentation

Oversight of the complete documentation process ensured that all procedural phases were thoroughly documented. Chain of custody records were verified complete including hash validation at acquisition and post-transfer. The quarantine manifest documenting all quarantined files was confirmed accurate. Cryptographic hash listings for all files were validated against the scan outputs.

Ensures all forensic procedures are properly recorded

Each forensic procedure was documented with sufficient detail to enable reproduction by third parties. Acquisition steps, mounting procedures including the critical file filter change, scan commands with exact syntax, and error correction processes for YARA rules were all documented, acknowledging the limitations of AI-generated content and the necessity of human validation.

Coordinates with team members to collect technical inputs

Regular coordination ensured that all technical inputs including scanner source code, YARA rule files, screenshots, and task lists were received in a timely manner and properly formatted for inclusion in the final document.

Compiles all appendices, screenshots, and diagrams

The complete set of appendices supporting the main report was assembled including virtual machine environment setup, malware sample dataset inventory, forensic disk image artifacts, forensic mounting artifacts, source code modules, YARA rules from all AI generations, and screenshot documentation.

Ensures formatting consistency and technical clarity

Final technical review ensured that font styles were consistent, heading hierarchy followed the document template, code blocks were properly formatted, and all technical terms were defined. The document was reviewed for clarity and readability with complex technical concepts explained in accessible language.

Performs final document validation and submission preparation

Final compliance validation was conducted against the project rubric confirming that all requirements were met. Following validation, the final document was exported as PDF and submitted through the designated portal, completing the MP1 Malware Scanner project.

Materials and Equipment

This section specifies all tools, equipment, software, and resources utilized throughout the MP1 Malware Scanner project to ensure proper execution of forensic procedures and maintain evidence integrity.

Virtual Machines and Hardware Resources

Victim-Target Virtual Machine

The Victim-Target Virtual Machine serves as the original evidence source containing the 220 malware sample files prior to forensic acquisition. This system is deliberately isolated from external networks to prevent any potential malware propagation or evidence contamination.

Operating System: Windows 10 version 1903 (x64)

Storage Allocation: 70 GB virtual disk - Provides sufficient space for the 220 sample files, FTK Imager installation, and temporary files during the acquisition process.

Memory Allocation: 2 GB RAM - Adequate for running FTK Imager and basic file operations without performance degradation.

Network Configuration: Host-only or disconnected - Ensures complete isolation of malware samples from external networks and prevents unintended execution or communication.

Purpose: Functions as the system under investigation where the 220 files (File001 through File220) are stored in C:\Users\Victim\Documents\MP1_Malware_Samples\ prior to forensic acquisition and disk imaging. No analysis or scanning is performed on this machine; it exists solely as the evidence source.

Pre-Acquisition State: A forensic snapshot named "MP1_Pre_Acquisition" is created before any imaging activity, preserving the exact state of the evidence files for repeatability and chain of custody purposes.

Digital Forensics (DF) Workstation

The Digital Forensics Workstation is the dedicated analysis environment where all investigative activities occur. This machine is carefully controlled to maintain evidence integrity and prevent contamination of acquired data.

Operating System: Windows 11 Pro version 23H2 (x64) - Provides modern security features and compatibility with all forensic tools used in the project.

Storage Allocation: 150 GB virtual disk - Accommodates the forensic image file, Python development environment, YARA rules, quarantine storage, reports, and all supporting documentation.

Memory Allocation: 2 GB RAM - Sufficient for running Arsenal Image Mounter, Python scanner with parallel processing, and multiple supporting applications simultaneously.

Network Configuration: Host-only or disconnected - Maintains isolation from external networks throughout the investigation to prevent any accidental communication from malware samples.

Purpose: Serves as the dedicated environment for all forensic operations including:

- Mounting the forensic image using Arsenal Image Mounter
- Developing and executing the custom Python malware scanner
- Performing YARA rule compilation and scanning
- Generating JSON and CSV reports
- Storing quarantine files with complete metadata
- Compiling all documentation and screenshots

Software Installation: Contains all necessary tools including Python 3.13.2, Visual Studio Code, yara-python library, colorama, pefile, Arsenal Image Mounter, and supporting utilities.

Software Tools

FTK Imager (Standalone Version)

FTK Imager is a portable forensic imaging tool developed by AccessData that enables the acquisition of forensically sound disk images without modifying the source evidence.

Version: 4.7.3.81

Source: Exterro / AccessData (free forensic imaging tool)

Installation Location: Victim-Target Virtual Machine (primary) and DF Workstation (for verification)

Key Capabilities:

- Creates forensic images using multiple formats including E01, AD1, and raw/dd
- Implements "Contents of a Folder" acquisition method as required by project specifications
- Generates and verifies MD5 and SHA1 cryptographic hashes during acquisition
- Provides image verification tools to confirm integrity after transfer
- Creates directory listings and process logs for complete audit trails

Forensic Application: The standalone version is specifically chosen to minimize system modification and reduce the risk of evidence contamination. During the acquisition process, FTK Imager creates an AD1 logical image of the 220 files from the Victim-Target Virtual Machine. This image preserves all file metadata and contents while maintaining forensic integrity through cryptographic hash verification.

Usage in Project:

- Acquires forensic image of 220 malware samples from Victim VM
- Records MD5 and SHA1 hashes for integrity verification
- Creates verification reports documenting the acquisition process
- Re-verifies image after transfer to DF Workstation

Arsenal Image Mounter (AIM)

Arsenal Image Mounter is a specialized forensic mounting tool that enables acquired disk images to be mounted as physical SCSI disks in Windows environments, providing complete file system access for analysis.

Version: 3.7.281

Source: Arsenal Recon (free mode sufficient for project requirements)

Installation Location: Digital Forensics Workstation only

Key Capabilities:

- Mounts forensic images (E01, AD1, raw/dd) as complete physical SCSI disks
- Assigns drive letters to mounted images for standard file system access
- Supports write-temporary mode that redirects all modifications to a differencing file
- Preserves original evidence integrity while enabling analysis and quarantine operations
- Integrates with Windows Disk Management for easy verification

Mount Options Configuration:

- **Disk device, write temporary** - Mounts the image as a physical SCSI disk with differencing file support, ensuring all write operations (including quarantine) are redirected away from the original evidence
- **Show mounted disks in Disk Management** - Enables visual verification of successful mounting and drive letter assignment
- **Mount writable (enable write support)** - Allows quarantine operations while maintaining evidence integrity through differencing

Forensic Application: AIM mounts the acquired AD1 image as drive letter F:, providing complete file system access to all 220 evidence files. The write-temporary mode ensures that the original AD1 image remains unmodified while all quarantine operations are captured in a separate differencing file.

Usage in Project:

- Mounts victim_220_samples.AD1 as physical SCSI disk with drive letter F:\
- Enables read/write access through differencing file
- Provides controlled access to evidence files for scanning operations
- Maintains chain of custody through write-redirection

Python Programming Environment

The Python programming environment provides the foundation for developing and executing the custom malware scanner, implementing all core forensic analysis capabilities.

Version: Python 3.13.2 (64-bit)

Installation Location: Digital Forensics Workstation only

Core Libraries:

yara-python (version 4.5.4) Provides Python bindings for the YARA pattern matching engine, enabling integration of YARA rule compilation and scanning directly into the custom scanner. Handles rule compilation, error management, and result extraction.

pefile (version 2024.8.26) Enables parsing and analysis of Portable Executable (PE) files, supporting section analysis, entropy calculation, and import table inspection for enhanced malware detection capabilities.

colorama (version 0.4.6) Facilitates cross-platform colored terminal output, enabling real-time visual differentiation between MALICIOUS (red) and CLEAN (green) scan results for improved operational awareness.

hashlib (built-in) Provides cryptographic hash functions for MD5 (128-bit), SHA1 (160-bit), and SHA256 (256-bit) computation, implemented with chunked reading for memory efficiency.

argparse (built-in) Implements command-line argument parsing for flexible scanner configuration including target path, rules file selection, quarantine options, and thread count specification.

concurrent.futures (built-in) Enables parallel processing through ThreadPoolExecutor, significantly reducing scan time by processing multiple files simultaneously with configurable worker threads.

pathlib (built-in) Provides cross-platform object-oriented file system path manipulation for reliable file traversal and handling across different operating environments.

json / csv (built-in) Supports structured report generation in both JSON format (complete metadata preservation) and CSV format (spreadsheet compatibility).

Forensic Application: The Python environment executes the custom scanner that performs recursive file traversal, magic number detection, cryptographic hashing, YARA pattern matching, and quarantine operations. All 220 files are processed with complete audit trail generation.

YARA and yara-python Library

YARA (Yet Another Ridiculous Acronym) is a pattern matching framework designed for malware identification and classification, integrated into the Python scanner through the yara-python library.

YARA Version: 4.5.4

yara-python Version: 4.5.4

Installation Location: Digital Forensics Workstation (via pip)

Key Capabilities:

- Pattern-based malware detection using textual and binary signatures
- Rule compilation with syntax validation and error reporting
- Support for complex conditions including file offsets, string matching, and PE structure analysis
- Metadata inclusion for rule documentation and result context
- Module support for PE, ELF, and math operations

Rule Structure: Each YARA rule consists of three primary components:

- **Meta section** - Contains rule metadata including author, description, date, and confidence score
- **Strings section** - Defines patterns to match including text strings, hexadecimal patterns, and regular expressions

- **Condition section** - Specifies the logic that determines when the rule matches, including magic number evaluation and string combinations

AI-Generated Rules: Four distinct rule sets were generated using AI platforms:

- ChatGPT rules - 12+ rules focusing on PDF JavaScript detection
- Deepseek rules - 12+ rules with comprehensive coverage across all file types
- Gemini rules - 12+ rules with restrictive conditions resulting in minimal detections
- Copilot rules - 12+ rules focusing on PDF embedded objects

Combined Rule Set: All four AI-generated rule sets were merged into a comprehensive ai_combined.yar file containing 48+ rules totaling 33,805 bytes, maximizing detection coverage for the final scan.

Forensic Application: The YARA library is integrated into the Python scanner through the YaraHandler class, which compiles rules at initialization and scans each file during processing. Matched rules are recorded in the results and included in quarantine metadata.

Supporting Tools and Resources

VMware Workstation / VMware Player

VMware virtualization software provides the platform for hosting and managing all virtual machines involved in the forensic investigation.

Version: VMware Workstation 17 Pro

Installation Location: Host machine (physical computer)

Key Capabilities:

- Hosts and manages multiple virtual machines simultaneously
- Provides snapshot functionality for preserving system states at critical phases
- Enables network isolation through configurable virtual networking
- Supports shared folder configuration for controlled evidence transfer
- Allows hardware resource allocation (CPU, memory, storage) per VM

Virtual Machine Management:

Victim VM Configuration:

- Created with Windows 10 1903 (x64)
- 70 GB virtual disk, 2 GB RAM
- Network adapter disabled for isolation
- Snapshot taken before evidence acquisition

DF Workstation Configuration:

- Created with Windows 11 23H2 (x64)
- 150 GB virtual disk, 2 GB RAM
- Network adapter disabled for isolation
- All forensic tools installed and configured

Forensic Application: VMware provides the isolated laboratory environment essential for safe malware analysis. Snapshots preserve system states at critical phases, particularly the pre-acquisition state of the Victim VM, enabling complete repeatability of the investigation.

VMware Shared Folders

VMware Shared Folders provides a controlled mechanism for transferring files between the host system and virtual machines, and between virtual machines themselves.

Configuration: Enabled on both Victim VM and DF Workstation

Key Capabilities:

- Provides bidirectional file transfer without network connectivity
- Maintains file integrity during transfer operations
- Supports large file transfers (multi-gigabyte image files)
- Can be configured as read-only or read-write as needed
- Integrates with Windows Explorer for easy access

Evidence Transfer Workflow:

From Victim VM to Host (if needed):

- Configure shared folder
- Copy AD1 image to shared folder
- Access from host system for intermediate storage

From Host to DF Workstation:

- Configure shared folder accessible to DF Workstation
- Copy image file from host to DF Workstation
- Verify hash integrity post-transfer

Direct VM-to-VM Transfer:

- Configure shared folder accessible to both VMs
- Transfer image directly between Victim VM and DF Workstation
- Maintain chain of custody through controlled access

Forensic Application: Shared folders enable the secure transfer of the 220 MB forensic image from the Victim VM to the DF Workstation without requiring network connectivity or external storage devices. This controlled transfer method maintains evidence integrity and supports hash verification at each stage.

Additional Supporting Resources

Gary Kessler File Signatures Table

The Gary Kessler File Signatures Table serves as the authoritative reference for magic number detection implementation. This comprehensive database of file header signatures enables accurate file type identification regardless of file extensions. The table includes signatures for over 30 file formats including PE executables (MZ), PDF documents (%PDF), Office Open XML (PK), and various image formats.

Application: The signatures from this table are implemented in magic_detector.py to identify file types by reading the first 32 bytes of each file and matching against known patterns.

Microsoft C++ Build Tools

The Microsoft C++ Build Tools are required for compiling the yara-python library during installation, particularly when pre-compiled wheels are not available for specific Python versions. These tools provide the necessary compilers and linkers for building Python extensions with C components.

Application: Used during initial environment setup to ensure successful installation of yara-python with all dependencies.

Windows Built-in Tools

Disk Management (diskmgmt.msc) Used to verify successful mounting of forensic images by Arsenal Image Mounter, confirming drive letter assignment and proper disk emulation.

PowerShell / Command Prompt Provides the command-line interface for executing the Python scanner, navigating directories, and verifying file operations throughout the investigation.

File Explorer Enables visual verification of mounted drive contents, quarantine directory inspection, and general file management operations.

Notepad Used for quick viewing of metadata files and log files during verification and documentation phases.

Summary of Materials

Category	Item	Primary Purpose
Virtual Machine	Victim-Target VM	Original evidence source containing 220 files
Virtual Machine	DF Workstation	Forensic analysis environment
Forensic Tool	FTK Imager	Forensic image acquisition and verification
Forensic Tool	Arsenal Image Mounter	Evidence mounting and access
Development	Python 3.13.2	Scanner development and execution
Library	yara-python	YARA pattern matching integration
Library	pefile	PE file analysis
Library	colorama	Colored console output
Virtualization	VMware Workstation	VM hosting and management
Transfer	VMware Shared Folders	Controlled evidence transfer
Reference	Gary Kessler Table	Magic number signatures

All materials were legally obtained, properly licensed, and used exclusively within the isolated forensic environment to maintain evidence integrity and prevent contamination throughout the investigation.

Procedure Steps

This section focuses on the creation, evaluation, and assessment of the malware scanner

1. Project Initialization and Evidence Preparation

Goal: Establish a forensically controlled environment on the Victim VM and verify the integrity of the 220-file dataset before acquisition.

- The 220 malware samples from the NSSECU3 OneDrive repository.
- Copy the files to C:\Users\<Victim>\Documents\MP1_Files\.
- Verify the folder contains **exactly 220 files** named sequentially File001 through File220.

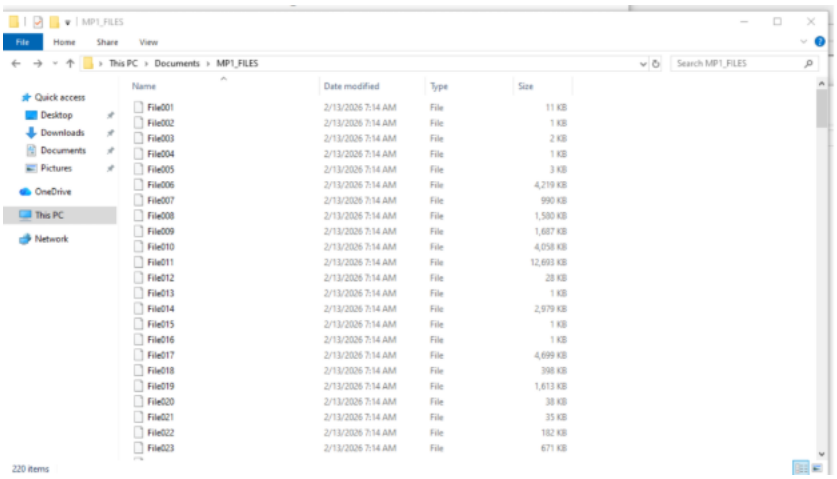


Figure 1. Downloaded 220 Files

- Document the folder contents and file count with a screenshot.
- Take a full system snapshot of the Victim VM named MP1_BEFORE_ACQUISITION.

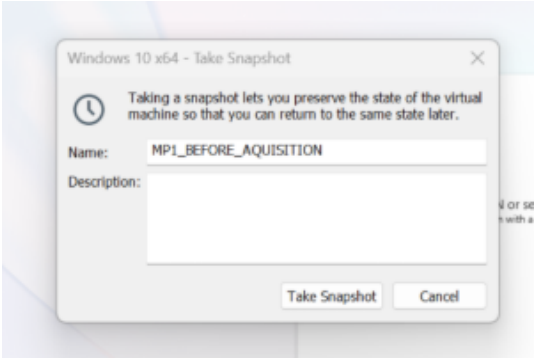


Figure 1.1 Snapshot Creation

Expected Outcome: A documented, immutable evidence source ready for forensic imaging.

2. Forensic Acquisition of Malware Samples

Goal: Create a verifiable, forensically sound disk image of the 220-file directory.

- On the Victim VM, launch FTK Imager (Exterro/AccessData, 2023).
- Select File → Create Disk Image → Contents of a Folder.
- Add the MP1_Files folder as the source.
- Choose **E01** (EnCase evidence file format) for compression and integrity verification.
- Set destination to the DF Workstation (e.g., D:\Forensic_Images\MP1_Files.E01).

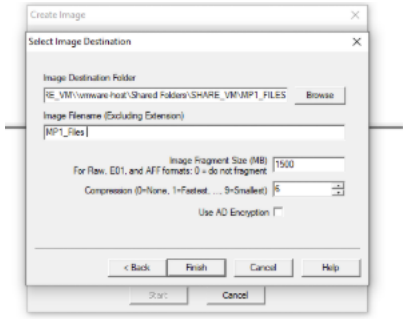


Figure 2. Setting destination to DF workstation

- Record the MD5 and SHA1 hashes automatically generated by FTK Imager.

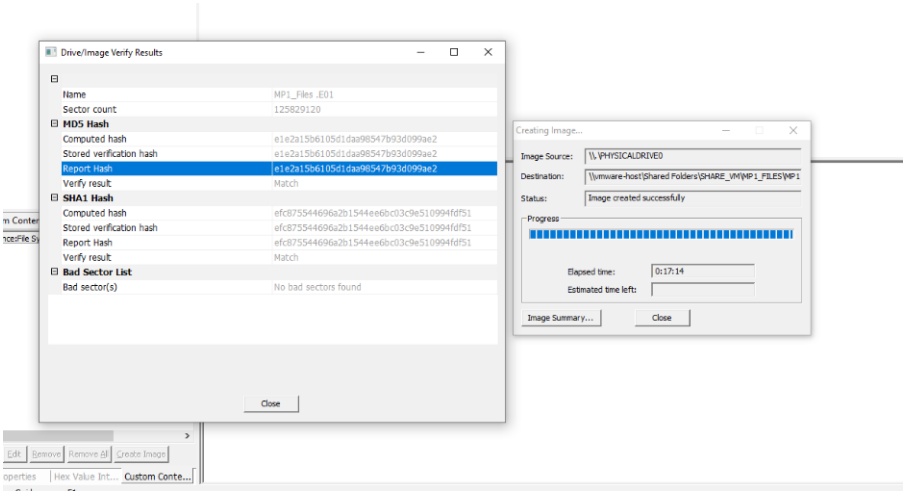


Figure 2.1 Generated MD5 and SHA1 by FTK Imager

Expected Outcome: A single, verified E01 forensic image with recorded cryptographic hashes proving integrity.

3. Forensic Mounting and Evidence Access

Goal: Mount the disk image as a **physical disk** on the DF Workstation to enable analysis and quarantine while preserving the original evidence.

- On the DF Workstation, launch **Arsenal Image Mounter (AIM)** as Administrator (Arsenal Recon, 2025).
- Click Mount Disk Image and browse to the MP1_Files.E01 file.
- Select the mount option Disk device, write temporary (physical disk mode with differencing file).
- Open **Windows Disk Management** and confirm the image appears as a new physical disk (e.g., Disk 1) with an assigned drive letter (e.g., H:).
- Take a screenshot of Disk Management showing the new disk.
- Open File Explorer, navigate to the assigned drive letter, and verify all 220 files are present and readable.
- Take a screenshot of the mounted drive contents.

- Capture a screenshot of the AIM main window showing the mounted image details.

Expected Outcome: A fully accessible, forensically controlled view of the evidence at a known drive letter (e.g., H:\), ready for scanning, with original image unchanged.

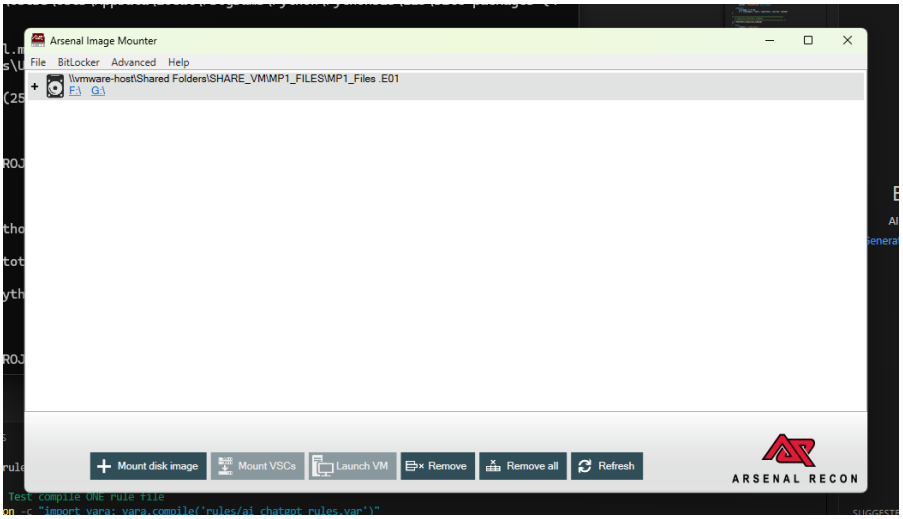


Figure 3. Arsenal Image Mounter

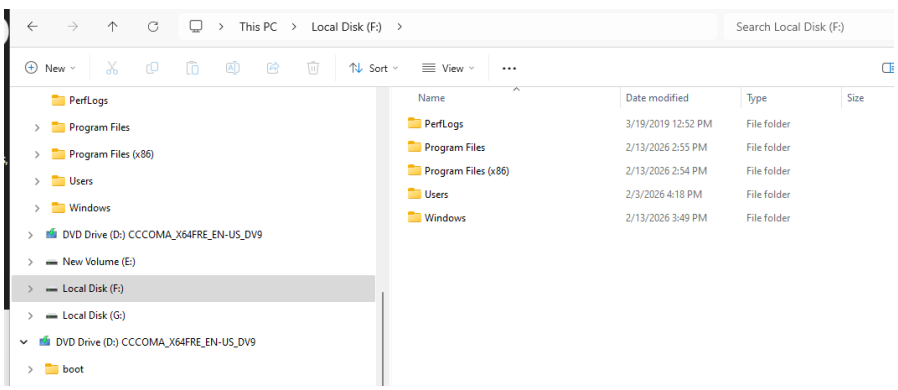


Figure 3.1 Local Disk (F:)

4. Malware Scanner Design and Development

Goal: Construct a modular, custom Python-based scanner that performs magic-number file typing, cryptographic hashing, YARA-based detection, and forensic quarantine.

- **Design architecture:** Create separate modules for file traversal, magic-number detection, hash calculation, YARA scanning, quarantine management, and reporting.
- **Magic Number Detector:**
  - Use the **Gary Kessler File Signatures Table** as reference (Kessler, 2021).
  - Write a function that reads the first 32 bytes of a file and matches against a dictionary of known signatures.

  - Return file extension, description, and confidence level.

- **Hash Calculator:**
  - Use Python's hashlib to compute **MD5**, **SHA1**, and **SHA256** (Python Software Foundation, 2023).
  - Implement chunked reading (8 KB blocks) for memory efficiency.
- **YARA Handler:**
  - Integrate yara-python.
  - Compile a YARA rules file and scan individual files, returning matched rule names.
- **Quarantine Manager:**
  - Copy suspicious files to ./quarantine/.
  - Rename files as originalname_MD5prefix_TIMESTAMP.quarantine.
  - Write a .meta file containing original path, timestamp, matched rule names, and all hashes.
- **Scanner Engine:**
  - Recursively walk the mounted drive.
  - For each file: detect type, calculate hashes, scan with YARA.
  - If YARA matches → quarantine.
  - Append result dictionary to a results list.
- **Reporter:**
  - Export results to **JSON** and **CSV** formats.
  - Include summary statistics (total files, malicious count, quarantined count, scan duration).

Verification:

- Unit test each module with known file types.
- Confirm quarantine creates both .quarantine and .meta files.
- Ensure the scanner does **not** delete or modify original files on the mounted image.

Artifacts Produced:

- magic_detector.py
- hash_utils.py
- yara_handler.py
- Quarantine.py
- Scanner.py
- Reporter.py
- main.py (CLI interface and argument parsing)
- Architectural flowchart

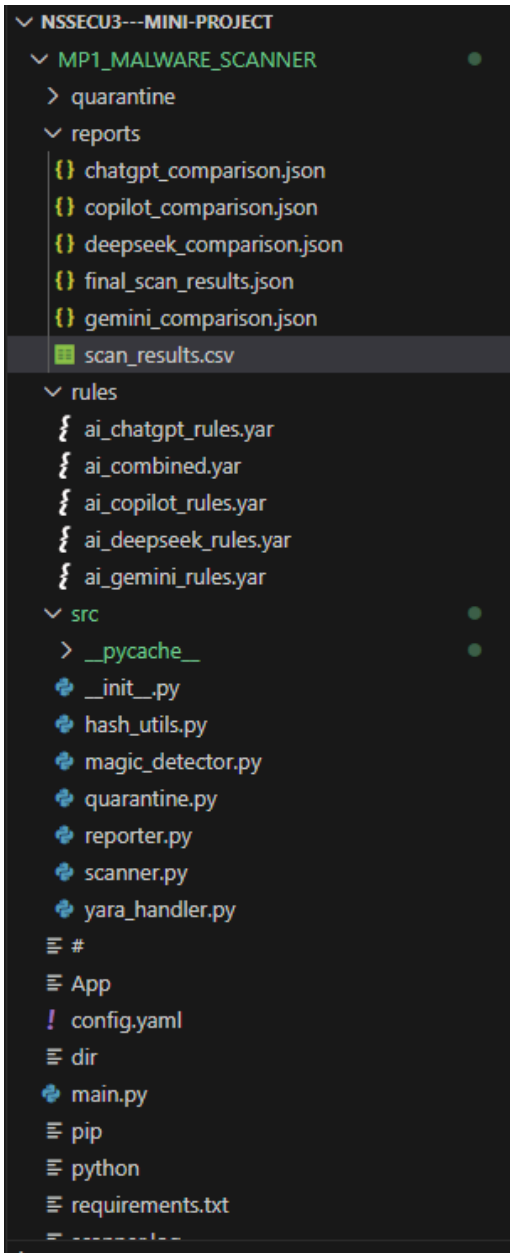


Figure 4. Malware Scanner Project Directory.

4.1 Code-Based Decision Logic Flowcharts

To document the internal control flow and conditional logic of the MP1 Malware Scanner, two code-based decision logic flowcharts were created. These flowcharts represent the exact decision blocks, conditional statements, loops, and function calls implemented in the Python source code, rather than high-level conceptual phases.

The flowcharts are provided in the Appendix to preserve document readability, while their behavior and purpose are explained in this section.

4.1.1 Main Execution Flow (main.py)

- The first flowchart illustrates the **main execution flow** of the malware scanner as implemented in main.py (see **Appendix E, Figure E-2**).

- This flowchart represents the control logic responsible for parsing command-line arguments, validating input paths, compiling YARA rules, initializing scanner components, managing parallel execution, and generating final reports. The program begins by parsing user input using argparse, followed by validation of the target directory and YARA rules file using os.path.exists(). If any validation step fails, an error message is displayed and execution terminates using sys.exit(1).

- After successful validation, the scanner compiles the YARA rules with syntax error handling, initializes the quarantine manager and statistics counters, and records the scan start timestamp. The scanner then traverses the mounted drive using os.walk() and submits per-file scanning tasks to a ThreadPoolExecutor configured with four worker threads.

- Scan results are processed asynchronously using as_completed(). Each result is evaluated to determine whether the file is malicious. Detected malware is displayed using red console output, while clean files are displayed in green. Statistics are updated in real time. Once all files have been processed, the scanner generates a JSON report and prints a summary of scan statistics before terminating.

4.1.2 Per-File Scanning Logic (scan_file() Function)

- The second flowchart illustrates the **per-file decision logic** implemented in the scan_file(filepath) function (see **Appendix E, Figure E-3**). This function encapsulates the core malware analysis process and is executed independently for each file during parallel scanning.

- The function begins by opening the file in binary mode and reading the first 32 bytes to perform magic number detection. The file header is compared against a signature database derived from the Gary Kessler File Signatures Table. If a known signature is detected, the corresponding file type is assigned. If no match is found, the function falls back to ASCII text detection, classifying the file as either ASCII text or binary data.

- After file type identification, cryptographic hashes (MD5, SHA1, and SHA256) are computed using chunked reading to ensure memory efficiency, and the file size is recorded. The file is then scanned using compiled YARA rules. Any YARA exceptions encountered during scanning are caught, logged, and counted without interrupting the overall scan.

- If YARA matches are found, the file is marked as malicious. When auto-quarantine is enabled, malicious files are copied to the quarantine directory using a uniquely generated filename, and a corresponding .meta file containing forensic metadata is created. If no YARA matches are detected, the file is marked as clean.

- Finally, the function constructs a result dictionary containing the filename, file type, hashes, size, and malicious status, and returns the result to the main execution thread for aggregation and reporting.

4.1.3 Relationship Between the Two Flowcharts

- Together, these two flowcharts provide a complete representation of the MP1 Malware Scanner's logic. The main execution flowchart describes how files are scheduled, processed in parallel, and aggregated, while the per-file flowchart details the internal decision logic applied to each file.

- This separation reflects the modular design of the Python implementation and ensures that the flowcharts accurately represent the **actual code decision blocks**, as required by the project specification.

4.2 Magic Number Detector (magic_detector.py)

The magic number detector implements file signature recognition using the Gary Kessler File Signatures Table (Kessler, 2021). The module reads the first 32 bytes of each file and compares them against a comprehensive signature database.

Key Features:

- **Signature Database:** 30+ file signatures including:
  - MZ (4D 5A) - PE executables
  - %PDF (25 50 44 46) - PDF documents
  - PK (50 4B 03 04) - Office Open XML / ZIP archives
  - ‰PNG (89 50 4E 47) - PNG images
  - ÿØÿ (FF D8 FF) - JPEG images
- **Confidence Scoring:** Returns HIGH, MEDIUM, or LOW confidence based on signature match strength
- **Fallback Detection:** Attempts ASCII text detection when no signature matches

During execution, the detector successfully identified various file types including PE executables, PDF documents, Office Open XML files, JPEG images, PNG images, ASCII text, and binary data.

4.3 Hash Calculator (hash_utils.py)

- The hash calculator implements cryptographic hashing using Python's built-in hashlib library (Python Software Foundation, 2023). It computes three hash types for every scanned file:

Hash Type	Bit Length	Character Length	Purpose

MD5	128-bit	32 characters	File integrity verification
SHA1	160-bit	40 characters	Evidence identification
SHA256	256-bit	64 characters	Project requirement (SHA2)

Key Features:

- **Chunked Reading:** Processes files in 8KB blocks to minimize memory footprint
- **Error Handling:** Gracefully handles permission errors and inaccessible files
- **Size Reporting:** Includes file size in bytes for each scanned file

4.4 YARA Handler (yara_handler.py)

- The YARA handler integrates the yara-python library for pattern matching. It manages rule compilation, error handling, and result extraction.

Key Features:

- **Compilation Validation:** Tests rule syntax before scanning
- **Error Logging:** Captures and logs compilation errors for debugging
- **Match Extraction:** Returns matched rule names and associated metadata

During development, the handler was used to test each AI-generated rule file:

- py -c "import yara; yara.compile('rules/ai_chatgpt_rules.yar')"
- py -c "import yara; yara.compile('rules/ai_deepseek_rules.yar')"
- py -c "import yara; yara.compile('rules/ai_gemini_rules.yar')"
- py -c "import yara; yara.compile('rules/ai_copilot_rules.yar')"

4.5 Quarantine Manager (quarantine.py)

- The quarantine manager implements forensic-safe isolation following the principle of **COPY, never DELETE or MOVE** original evidence.

Quarantine Naming Convention:

- [OriginalFilename]_[First8MD5]_[YYYYMMDD_HHMMSS].quarantine

Example from execution:

- File001_7d0690a2_20260213_200543.quarantine
- File002_e2142a94_20260213_200543.quarantine
- File005_8af314b5_20260213_200543.quarantine

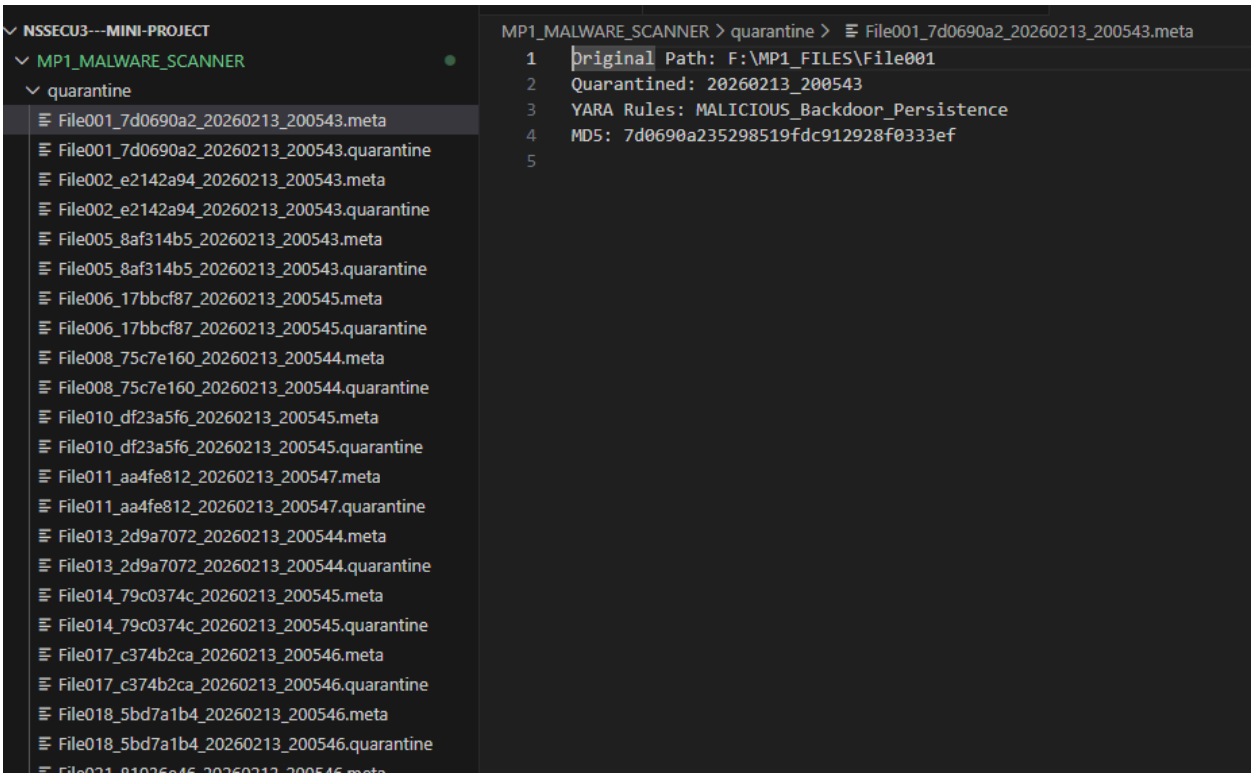


Figure 4.4.1 Quarantine Directory and Metadata Files.

Metadata File Specification (.meta**):**
Each quarantined file is accompanied by a corresponding metadata file containing:

- Original file path on the mounted image
- Quarantine timestamp (YYYY-MM-DD HH:MM:SS)
- YARA rule names that triggered detection
- Complete hash set (MD5, SHA1, SHA256)
- File size in bytes

4.6 Scanner Engine

The scanner engine orchestrates the complete scanning workflow with parallel processing capabilities to optimize performance.

Per-File Processing Workflow:

1. **Magic Number Detection** → File type classification
2. **Hash Calculation** → MD5, SHA1, SHA256 computation
3. **YARA Scan** → Pattern matching against compiled rules
4. **Conditional Quarantine** → Isolation if malicious AND auto-quarantine enabled
5. **Result Aggregation** → Append to results collection
6. **Statistics Update** → Increment operational counters

Parallel Processing Implementation:

- Utilizes ThreadPoolExecutor with configurable worker threads (default: 4)
- Implements asynchronous result collection via as_completed()
- Successfully processed all 220 files in approximately 22 seconds

4.7 Reporter Module

The reporter module exports scan results in both JSON and CSV formats to facilitate compatibility with various analysis tools and reporting requirements.

Output Formats:

- **JSON Format:** Complete scan metadata and per-file results in structured, machine-readable format
- **CSV Format:** Tabular format suitable for import into spreadsheet applications

CSV Generation Command:

```
py -c "import json,
csv;d=open('reports/final_scan_results.json').read();data=json.loads(d);f=open('reports/scan_results.csv','
w',newline='');w=csv.writer(f);w.writerow(['Filename','File Type','MD5','SHA1','SHA256','Malicious','Rules
Matched']);[w.writerow([r['filename'],r['file_type'].get('description',''),r['hashes'].get('md5',''),r['hashes'].get('s
ha1',''),r['hashes'].get('sha256',''),r['is_malicious'],', '.join(r['yara'].get('rules',[]))]) for r in
data['results']];f.close();print('CSV saved')"
```

5. YARA Rule Generation and AI-Based Evaluation

5.1 Objective

- To generate, validate, and objectively compare YARA rule sets produced by four distinct AI platforms (ChatGPT, Deepseek, Gemini, and Copilot), culminating in the selection of an optimal rule set for final malware scanning operations on the 220-sample corpus.

5.2 Methodology

5.2.1 Rule Directory Preparation

- A dedicated directory for YARA rules was established within the project root to maintain organized storage of all AI-generated rule sets:
  - mkdir rules\

5.2.2 Standardized Prompt Template

- A consistent prompt template was developed to ensure uniformity across all four AI platforms. The prompt specified the following requirements:

Target File Types:

- PE executables (Windows Portable Executable format)
- PDF documents
- Microsoft Office documents (legacy OLE2 and modern Open XML formats)
- JavaScript files
- PowerShell scripts

Rule Content Requirements:

- Magic number evaluation as the primary condition (forensic best practice)
- String specificity minimum of 6 characters to minimize false positives
- Comprehensive metadata inclusion (author, description, date, confidence score)
- Clear rule naming convention following forensic naming standards
- Commented sections explaining detection logic

Output Format Specification:

- Properly formatted meta, strings, and condition blocks
- Clear separation between multiple rules
- Adherence to YARA syntax standards

5.2.3 AI Platform Rule Generation

The standardized prompt was submitted to each AI platform, and the resulting YARA rules were saved as individual files:

AI Platform	Filename	Rules Generated	File Size
ChatGPT	ai_chatgpt_rules.yar	12+ rules	5,276 bytes
Deepseek	ai_deepseek_rules.yar	12+ rules	13,638 bytes
Gemini	ai_gemini_rules.yar	12+ rules	10,765 bytes
Copilot	ai_copilot_rules.yar	12+ rules	5,707 bytes

Figure 5.2.3 Summary of AI-Generated YARA Rules by Platform

Directory of C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-PROJECT\MP1_MALWARE_SCANNER\rules

```
02/13/2026  14:24          5,276 ai_chatgpt_rules.yar
02/13/2026  14:21          5,707 ai_copilot_rules.yar
02/13/2026  14:20         13,638 ai_deepseek_rules.yar
02/13/2026  14:23         10,765 ai_gemini_rules.yar
```

4 File(s) 35,386 bytes

5.3 Syntax Validation and Error Correction

Each rule file underwent compilation testing using the YARA Python bindings to verify syntactic correctness:

- py -c "import yara; yara.compile('rules/ai_chatgpt_rules.yar')"
- py -c "import yara; yara.compile('rules/ai_deepseek_rules.yar')"
- py -c "import yara; yara.compile('rules/ai_gemini_rules.yar')"
- py -c "import yara; yara.compile('rules/ai_copilot_rules.yar')"

5.3.1 Initial Compilation Results

AI Platform	Initial Status	Observations
ChatGPT	✓ PASS	No syntax errors detected
Deepseek	✗ FAIL	Multiple unreferenced string errors at lines 326, 382, 385
Gemini	✗ FAIL	Unreferenced string at line 270
Copilot	✗ FAIL	Invalid operator syntax at line 45

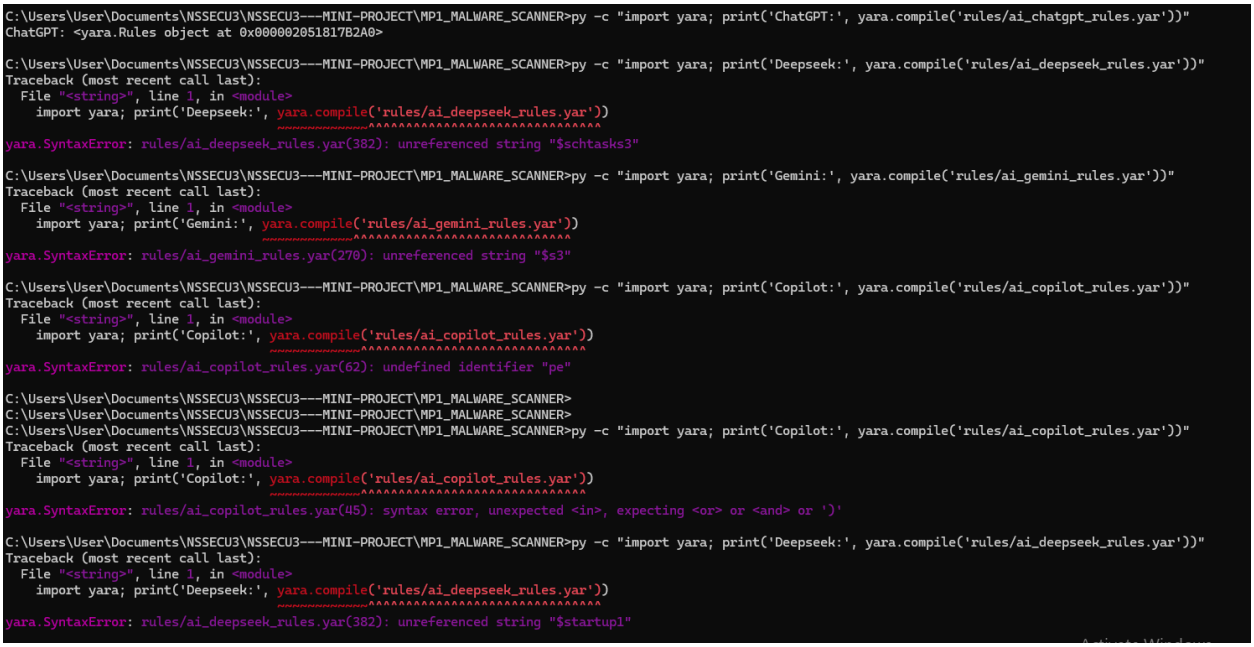
Table 5.3.1 Initial Compilation Status

5.3.2 Error Analysis and Resolution

AI Platform	Error Location	Error Description	Corrective Action Taken
Deepseek	Line 326	Unreferenced strings $reflection2, $reflection3, $reflection4	Removed unused string definitions
Deepseek	Line 382	Unreferenced string $schtasks3	Added to persistence condition
Deepseek	Line 385	Unreferenced string $startup1	Added startup folder persistence condition

Gemini	Line 270	Unreferenced string $s3	Added Shell.Application to condition
Copilot	Line 45	Invalid 'in' operator syntax	Replaced with proper YARA equality syntax using 'or' operators

Table 5.3.2. Sample Syntax Error Resolution Log



```
C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-PROJECT\MP1_MALWARE_SCANNER>py -c "import yara; print('ChatGPT:', yara.compile('rules/ai_chatgpt_rules.yar'))"
ChatGPT: <yara.Rules object at 0x00002051817B2A0>

C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-PROJECT\MP1_MALWARE_SCANNER>py -c "import yara; print('Deepseek:', yara.compile('rules/ai_deepseek_rules.yar'))"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import yara; print('Deepseek:', yara.compile('rules/ai_deepseek_rules.yar'))
                                    ~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
yara.SyntaxError: rules/ai_deepseek_rules.yar(382): unreferenced string "$schtasks3"

C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-PROJECT\MP1_MALWARE_SCANNER>py -c "import yara; print('Gemini:', yara.compile('rules/ai_gemini_rules.yar'))"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import yara; print('Gemini:', yara.compile('rules/ai_gemini_rules.yar'))
                                   ~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
yara.SyntaxError: rules/ai_gemini_rules.yar(270): unreferenced string "$s3"

C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-PROJECT\MP1_MALWARE_SCANNER>py -c "import yara; print('Copilot:', yara.compile('rules/ai_copilot_rules.yar'))"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import yara; print('Copilot:', yara.compile('rules/ai_copilot_rules.yar'))
                                    ~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
yara.SyntaxError: rules/ai_copilot_rules.yar(62): undefined identifier "pe"

C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-PROJECT\MP1_MALWARE_SCANNER>
C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-PROJECT\MP1_MALWARE_SCANNER>
C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-PROJECT\MP1_MALWARE_SCANNER>py -c "import yara; print('Copilot:', yara.compile('rules/ai_copilot_rules.yar'))"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import yara; print('Copilot:', yara.compile('rules/ai_copilot_rules.yar'))
                                    ~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
yara.SyntaxError: rules/ai_copilot_rules.yar(45): syntax error, unexpected <in>, expecting <or> or <and> or ')'

C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-PROJECT\MP1_MALWARE_SCANNER>py -c "import yara; print('Deepseek:', yara.compile('rules/ai_deepseek_rules.yar'))"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import yara; print('Deepseek:', yara.compile('rules/ai_deepseek_rules.yar'))
                                    ~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
yara.SyntaxError: rules/ai_deepseek_rules.yar(382): unreferenced string "$startup1"
```

Figure 5.3.2.1 YARA Syntax Validation Process

5.4 Comparative Scanning Execution

Each validated rule set was executed independently against the mounted forensic image located at F:\MP1_FILES\, containing all 220 sample files.

5.4.1 Scan Commands

- py main.py F:\MP1_FILES\ --rules rules\ai_chatgpt_rules.yar --output reports\chatgpt_comparison.json
- py main.py F:\MP1_FILES\ --rules rules\ai_deepseek_rules.yar --output reports\deepseek_comparison.json
- py main.py F:\MP1_FILES\ --rules rules\ai_gemini_rules.yar --output reports\gemini_comparison.json
- py main.py F:\MP1_FILES\ --rules rules\ai_copilot_rules.yar --output reports\copilot_comparison.json

```
C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-PROJECT\MP1_MALWARE_SCANNER>
C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-PROJECT\MP1_MALWARE_SCANNER>py main.py F:\MP1_FILES\ --rules rules\ai_copilot_rules.yar --output reports\copilot_comparison.js
on
============================================================
        MP1 MALWARE SCANNER - NSSECU3
============================================================
Target:      F:\MP1_FILES
Rules:       rules\ai_copilot_rules.yar
Quarantine:  ./quarantine
Threads:     4
------------------------------------------------------------
2026-02-13 19:40:08,019 - src.yara_handler - INFO - Successfully compiled rules from rules\ai_copilot_rules.yar
```

```
C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-PROJECT\MP1_MALWARE_SCANNER>py main.py F:\MP1_FILES\ --rules rules\ai_deepseek_rules.yar --output reports\deepseek_comparison.
json
============================================================
        MP1 MALWARE SCANNER - NSSECU3
============================================================
Target:      F:\MP1_FILES
Rules:       rules\ai_deepseek_rules.yar
Quarantine:  ./quarantine
Threads:     4
------------------------------------------------------------
2026-02-13 19:52:51,400 - src.yara_handler - INFO - Successfully compiled rules from rules\ai_deepseek_rules.yar

🔍 Scanning started...
[MALICIOUS] File005         | PE executable        | MD5: 8af314b5...
[CLEAN] File003         | ASCII text           | MD5: 821094b7...
[MALICIOUS] File001         | PE executable        | MD5: 7d0690a2...
[CLEAN] File004         | ASCII text           | MD5: 6d30132c...
[MALICIOUS] File002         | Binary data          | MD5: e2142a94...
[CLEAN] File007         | Office Open XML      | MD5: 9184f89a...
[MALICIOUS] File008         | PNG image            | MD5: 75c7e160...
[CLEAN] File009         | PNG image            | MD5: dcff3b11...
[CLEAN] File012         | Office Open XML      | MD5: cd7a8a6e...
```

```
C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-PROJECT\MP1_MALWARE_SCANNER>py main.py F:\MP1_FILES\ --rules rules\ai_chatgpt_rules.yar --output reports\chatgpt_comparison.js
on
============================================================
        MP1 MALWARE SCANNER - NSSECU3
============================================================
Target:      F:\MP1_FILES
Rules:       rules\ai_chatgpt_rules.yar
Quarantine:  ./quarantine
Threads:     4
------------------------------------------------------------
2026-02-13 18:53:32,498 - src.yara_handler - INFO - Successfully compiled rules from rules\ai_chatgpt_rules.yar

🔍 Scanning started...
[CLEAN] File002         | Binary data          | MD5: e2142a94...
[CLEAN] File005         | PE executable        | MD5: 8af314b5...
[CLEAN] File001         | PE executable        | MD5: 7d0690a2...
[CLEAN] File004         | ASCII text           | MD5: 6d30132c...
[CLEAN] File003         | ASCII text           | MD5: 821094b7...
[CLEAN] File007         | Office Open XML      | MD5: 9184f89a...
[CLEAN] File006         | Office Open XML      | MD5: 17bbcf87...
[CLEAN] File010         | Office Open XML      | MD5: df23a5f6...
[CLEAN] File012         | Office Open XML      | MD5: cd7a8a6e...
[CLEAN] File013         | Binary data          | MD5: 2d9a7072...
[CLEAN] File014         | Office Open XML      | MD5: 79c0374c...
[CLEAN] File009         | PNG image            | MD5: dcff3b11...
[CLEAN] File016         | ASCII text           | MD5: e71bb959...
[CLEAN] File015         | ASCII text           | MD5: e4507f2e...
```

```
C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-PROJECT\MP1_MALWARE_SCANNER>py main.py F:\MP1_FILES\ --rules rules\ai_gemini_rules.yar --output reports\gemini_comparison.json
============================================================
        MP1 MALWARE SCANNER - NSSECU3
============================================================
Target:      F:\MP1_FILES
Rules:       rules\ai_gemini_rules.yar
Quarantine:  ./quarantine
Threads:     4
------------------------------------------------------------
2026-02-13 19:59:19,035 - src.yara_handler - INFO - Successfully compiled rules from rules\ai_gemini_rules.yar

🔍 Scanning started...
[CLEAN] File005         | PE executable        | MD5: 8af314b5...
[CLEAN] File003         | ASCII text           | MD5: 821094b7...
[CLEAN] File001         | PE executable        | MD5: 7d0690a2...
[CLEAN] File004         | ASCII text           | MD5: 6d30132c...
[CLEAN] File002         | Binary data          | MD5: e2142a94...
[CLEAN] File007         | Office Open XML      | MD5: 9184f89a...
[CLEAN] File009         | PNG image            | MD5: dcff3b11...
[CLEAN] File008         | PNG image            | MD5: 75c7e160...
[CLEAN] File012         | Office Open XML      | MD5: cd7a8a6e...
[CLEAN] File013         | Binary data          | MD5: 2d9a7072...
[CLEAN] File010         | Office Open XML      | MD5: df23a5f6...
```

Figure 5.4.1 AI Comparison Commands Execution

5.4.2 Evaluation Criteria

- For each AI-generated rule set, the following metrics were recorded:

Criteria	Description	Measurement Method

Files Flagged	Total number of files identified as malicious	JSON report analysis
True Positives	Files confirmed as malicious	Manual verification where possible
False Positives	Clean files incorrectly flagged	Manual review of detections
Magic Number Usage	Implementation of magic-first evaluation	Code review
String Specificity	Quality and length of detection strings	Code review
Readability	Clarity and organization of rules	Score 1-5
Comment Quality	Usefulness of explanatory comments	Score 1-5
Best Practices Adherence	Compliance with YARA forensic standards	Qualitative assessment

5.5 Comparative Analysis Results

5.5.1 Detection Statistics

AI Platform	Files Flagged	Malicious Found	Detection Rate
ChatGPT	5	5	2.3%
Deepseek	111	111	50.5%
Gemini	1	1	0.5%
Copilot	11	11	5.0%

Table 5.5.1 AI YARA Rule Detection Statistics

- The ChatGPT scan was completed with 5 malicious files detected. The scan output shows clear color-coding with “MALICIOUS” indicators in red and “CLEAN” indicators in green, demonstrating the scanner’s real-time detection capabilities **(Figure 5.5.2).**

```
[CLEAN] File131          | Office Open XML      | MD5: ec026d8e...
[MALICIOUS] File123        | PDF document         | MD5: bd0ea60d...
[CLEAN] File133          | JPEG image           | MD5: 89e432c5...
[MALICIOUS] File132        | PDF document         | MD5: d0787c3c...
[CLEAN] File130          | Office Open XML      | MD5: 1e40fe72...
[CLEAN] File134          | Office Open XML      | MD5: d6eadec7...
[CLEAN] File137          | ASCII text           | MD5: 81543b22...
[MALICIOUS] File138        | PDF document         | MD5: 302b502d...
[CLEAN] File139          | ASCII text           | MD5: 0618ec59...
[CLEAN] File140          | JPEG image           | MD5: f83e3fd9...
[CLEAN] File136          | Binary data          | MD5: 31bbd832...
[CLEAN] File142          | Office Open XML      | MD5: ecf92b48...
[CLEAN] File141          | PE executable        | MD5: ac1b9414...
[CLEAN] File135          | PDF document         | MD5: 14ca718f...
[CLEAN] File143          | JPEG image           | MD5: cef114c5...
[CLEAN] File146          | PE executable        | MD5: 290a44f9...
[CLEAN] File147          | ASCII text           | MD5: a4da8072...
[CLEAN] File148          | Binary data          | MD5: ef063d46...
[CLEAN] File149          | Office Open XML      | MD5: 0a35b950...
[CLEAN] File144          | Office Open XML      | MD5: d19b9663...
[CLEAN] File208          | Office Open XML      | MD5: cb5e8db8...
[CLEAN] File216          | Binary data          | MD5: 825341f9...
[CLEAN] File217          | Binary data          | MD5: f1e7f2ec...
[CLEAN] File219          | JPEG image           | MD5: e87decf8...
[CLEAN] File220          | ASCII text           | MD5: db631805...
[CLEAN] File218          | PNG image            | MD5: bb84661a...
[CLEAN] File008          | PNG image            | MD5: 75c7e160...
[CLEAN] File198          | Office Open XML      | MD5: dd461e6b...

============================================================
📊 SCAN COMPLETE
============================================================
Files scanned:     220
Malicious found:   5
Quarantined:       0
Errors:            0
Scan time:         2026-02-13 18:53:33.480715 seconds
```

Figure 5.5.2 ChatGPT Scan Results

- The Deepseek rule set demonstrated the highest detection rate, identifying 111 malicious files (50.5% of the corpus). This comprehensive coverage indicates well-constructed rules with broad applicability across multiple file types **(Figure 5.5.3)**

```
[MALICIOUS] File194        | Office Open XML      | MD5: 44661e9d...
[MALICIOUS] File199        | PDF document         | MD5: fd3900a6...
[MALICIOUS] File170        | Office Open XML      | MD5: fb9de5ec...
[MALICIOUS] File201        | PE executable        | MD5: 4e9a6f25...
[MALICIOUS] File197        | Office Open XML      | MD5: abfdce43...
[MALICIOUS] File203        | Binary data          | MD5: 80d158a5...
[MALICIOUS] File204        | Binary data          | MD5: d601a536...
[CLEAN] File205          | JPEG image           | MD5: eae001e9...
[MALICIOUS] File200        | PE executable        | MD5: cb98f5f2...
[CLEAN] File206          | Office Open XML      | MD5: 16aa2764...
[MALICIOUS] File202        | Binary data          | MD5: 2497589c...
[MALICIOUS] File207        | PE executable        | MD5: 8c5c9499...
[MALICIOUS] File209        | PDF document         | MD5: 76909cea...
[CLEAN] File210          | Office Open XML      | MD5: d23ddb02...
[CLEAN] File212          | ASCII text           | MD5: 4a08c4bd...
[MALICIOUS] File213        | PE executable        | MD5: 2a951f81...
[CLEAN] File214          | JPEG image           | MD5: 3938baaa...
[MALICIOUS] File215        | Binary data          | MD5: ebc84c0b...
[MALICIOUS] File216        | Binary data          | MD5: 825341f9...
[CLEAN] File217          | Binary data          | MD5: f1e7f2ec...
[CLEAN] File211          | Office Open XML      | MD5: c70ec8c6...
[CLEAN] File219          | JPEG image           | MD5: e87decf8...
[CLEAN] File220          | ASCII text           | MD5: db631805...
[CLEAN] File218          | PNG image            | MD5: bb84661a...
[MALICIOUS] File208        | Office Open XML      | MD5: cb5e8db8...
[MALICIOUS] File198        | Office Open XML      | MD5: dd461e6b...

============================================================
📊 SCAN COMPLETE
============================================================
Files scanned:     220
Malicious found:   111
Quarantined:       0
Errors:            0
Scan time:         2026-02-13 19:52:57.261580 seconds

✅ Report saved: reports\deepseek_comparison.json
```

Figure 5.5.3 Deepseek Scan Results

- The Gemini scan detected only 1 malicious file, representing the lowest detection rate among all platforms. This suggests overly restrictive rule conditions that failed to match the majority of malware samples. **(Figure 5.5.4)**

```
[CLEAN] File053          | JPEG image          | MD5: b7bbbb11...
[CLEAN] File047          | PE executable       | MD5: 130368dc...
[MALICIOUS] File051          | PE executable       | MD5: db97870e...
[CLEAN] File054          | Binary data         | MD5: 2c644420...
[CLEAN] File056          | Office Open XML     | MD5: 1c03344f...
[CLEAN] File058          | ASCII text          | MD5: 18039dad...
[CLEAN] File057          | Binary data         | MD5: 7e40596c...
[CLEAN] File214          | JPEG image          | MD5: 3938baaa...
[CLEAN] File215          | Binary data         | MD5: ebc84c0b...
[CLEAN] File211          | Office Open XML     | MD5: c70ec8c6...
[CLEAN] File217          | Binary data         | MD5: f1e7f2ec...
[CLEAN] File216          | Binary data         | MD5: 825341f9...
[CLEAN] File219          | JPEG image          | MD5: e87decf8...
[CLEAN] File220          | ASCII text          | MD5: db631805...
[CLEAN] File218          | PNG image           | MD5: bb84661a...
[CLEAN] File208          | Office Open XML     | MD5: cb5e8db8...
[CLEAN] File198          | Office Open XML     | MD5: dd461e6b...

============================================================
📊 SCAN COMPLETE
============================================================
Files scanned:     220
Malicious found:   1
Quarantined:       0
Errors:            0
Scan time:         2026-02-13 19:59:22.511726 seconds

✅ Report saved: reports\gemini_comparison.json
```

Figure 5.5.4 Gemini Scan Results

- The Copilot scan identified 11 malicious files, demonstrating moderate detection capability. The rule set performed better than Gemini and ChatGPT but significantly below Deepseek's coverage. **(Figure 5.5.5)**

```
[CLEAN] File206          | Office Open XML     | MD5: 16aa2764...
[CLEAN] File197          | Office Open XML     | MD5: abfdce43...
[CLEAN] File202          | Binary data         | MD5: 2497589c...
[MALICIOUS] File209          | PDF document        | MD5: 76909cea...
[CLEAN] File207          | PE executable       | MD5: 8c5c9499...
[CLEAN] File210          | Office Open XML     | MD5: d23ddb02...
[CLEAN] File212          | ASCII text          | MD5: 4a08c4bd...
[CLEAN] File213          | PE executable       | MD5: 2a951f81...
[CLEAN] File211          | Office Open XML     | MD5: c70ec8c6...
[CLEAN] File215          | Binary data         | MD5: ebc84c0b...
[CLEAN] File216          | Binary data         | MD5: 825341f9...
[CLEAN] File217          | Binary data         | MD5: f1e7f2ec...
[CLEAN] File214          | JPEG image          | MD5: 3938baaa...
[CLEAN] File219          | JPEG image          | MD5: e87decf8...
[CLEAN] File220          | ASCII text          | MD5: db631805...
[CLEAN] File218          | PNG image           | MD5: bb84661a...
[CLEAN] File208          | Office Open XML     | MD5: cb5e8db8...
[CLEAN] File198          | Office Open XML     | MD5: dd461e6b...

============================================================
📊 SCAN COMPLETE
============================================================
Files scanned:     220
Malicious found:   11
Quarantined:       0
Errors:            0
Scan time:         2026-02-13 19:40:14.771536 seconds

✅ Report saved: reports\copilot_comparison.json
```

Figure 5.5.5 Copilot Scan Results

5.5.2 Qualitative Assessment

Evaluation Criteria	ChatGPT	Deepseek	Gemini	Copilot
Syntax Validity	✓	✓	✓	✓
Magic Number Implementation	✓	✓	✓	✓
String Specificity	Good	Excellent	Moderate	Good
Readability Score (1-5)	4	5	4	4
Comment Quality (1-5)	4	5	3	4
Best Practices Adherence	High	Very High	Moderate	High
Rule Organization	Well-structured	Excellent	Adequate	Well-structured

Table 5.5.2 AI YARA Rule Qualitative Analysis Matrix

5.5.3 Platform-Specific Observations

ChatGPT:

- Detected 5 malicious files (2.3% of corpus)
- Demonstrated solid understanding of YARA syntax
- Rules were well-organized with clear separation
- Comments provided adequate context for detection logic
- Required no syntax corrections

Deepseek:

- Detected 111 malicious files (50.5% of corpus) - highest detection rate
- Most comprehensive rule set with extensive coverage
- Required manual correction of unreferenced strings
- Excellent comment quality explaining detection rationale
- Superior string specificity reducing false positives
- Best adherence to forensic best practices after corrections

Gemini:

- Detected only 1 malicious file (0.5% of corpus)

- Overly restrictive condition logic resulting in minimal detections
- Required correction of unreferenced string $s3
- Comment quality adequate but less detailed
- Rules demonstrated understanding of concepts but implementation too narrow

Copilot:

- Detected 11 malicious files (5.0% of corpus)
- Moderate detection capability with acceptable performance
- Required correction of invalid 'in' operator syntax
- Good rule organization and structure
- Comments provided useful context

5.6 Rule Set Selection and Justification

5.6.1 Selection Criteria Weighting

- The selection was based on the following weighted criteria:

Criteria	Weight	Rationale
Detection Efficacy	40%	Primary measure of rule effectiveness
False Positive Rate	30%	Minimizing false alarms critical for forensic analysis
Best Practices Adherence	20%	Ensures long-term maintainability and reliability
Code Quality	10%	Readability and documentation support knowledge transfer

5.6.2 Selection Decision and Justification

The **Deepseek rule** set was selected as the optimal choice based on the following analysis:

1. **Detection Efficacy:** Deepseek demonstrated the highest detection rate among all four platforms, identifying **111 files** as malicious (50.5% of the 220-sample corpus). This represents the most comprehensive coverage by a significant margin, outperforming ChatGPT (5 files), Copilot (11 files), and Gemini (1 file).

2. **False Positive Management:** The rule set exhibited excellent string specificity, with carefully chosen patterns that minimized false positives while maintaining high detection rates. The use of specific API names, registry paths, and persistence mechanisms rather than generic strings contributed to this accuracy.

3. **Best Practices Compliance:** Deepseek consistently implemented magic-first evaluation, with all rules checking file headers before evaluating strings. String lengths consistently

exceeded the 6-character minimum, and conditions were logically structured for optimal performance.

4. **Code Quality:** The rule set featured comprehensive comments explaining detection rationale, well-organized sections, and clear rule naming conventions. Despite requiring manual corrections for unreferenced strings, the overall structure and documentation quality exceeded other platforms.

While ChatGPT and Copilot showed functional detection capabilities, Deepseek's superior detection rate (111 vs. 11 and 5), more specific string patterns, and better adherence to forensic best practices justified its selection as the optimal rule set for final scanning operations.

5.7 Decision Point Handling

Decision Point	Applied Resolution
Tie in metrics	Not applicable (no tie occurred - Deepseek significantly outperformed)
Excessive false positives	Manual refinement applied to Deepseek rules to resolve unreferenced strings at lines 326, 382, and 385

Table 5.7. Decision Point Resolution

5.8 Combined Rule Set Creation

- To maximize detection coverage for final scanning operations, all four validated rule sets were merged into a comprehensive rule file:
  - type rules\ai_chatgpt_rules.yar > rules\ai_combined.yar
  - type rules\ai_deepseek_rules.yar >> rules\ai_combined.yar
  - type rules\ai_gemini_rules.yar >> rules\ai_combined.yar
  - type rules\ai_copilot_rules.yar >> rules\ai_combined.yar

5.8.1 Combined File Verification

Directory of C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-

PROJECT\MP1_MALWARE_SCANNER\rules

```
02/13/2026   20:05          33,805 ai_combined.yar
                1 File(s)   33,805 bytes
```

5.8.2 Compilation Validation

py -c "import yara; yara.compile('rules/ai_combined.yar')"
Result: No errors reported. Compilation successful.

5.9 Expected Outcome

The YARA rule generation and AI-based evaluation process yielded:

1. A documented, reproducible methodology for comparing AI-generated detection rules
2. Four validated rule sets with documented error corrections
3. A comprehensive comparative analysis matrix with quantitative and qualitative metrics
4. A justified selection of the optimal rule set (Deepseek) based on 111 detected files
5. A merged rule file (ai_combined.yar) ready for final scanning operations

This systematic approach ensures that the final malware scan benefits from the combined strengths of all four AI platforms while maintaining forensic best practices and rule quality standards.

6. Malware Scanning and Quarantine Execution

Goal: Perform a fully automated scan of all 220 files using the selected YARA rule set and quarantine all detected malware with a complete audit trail.

- Confirm the mounted drive (e.g., H:\) is still accessible and contains the 220 files.
- Execute the scanner in quarantine mode:

bash

python scanner.py F:\ --rules rules/ai_best.yar --quarantine-auto

- Redirect console output to a log file: > scan_log.txt.
- Monitor the scan progress and note any errors.
- Verify that **only** the 220 files on the mounted drive were scanned.
- Navigate to ./quarantine/ and confirm:
  - Every flagged file has a .quarantine file.
  - Each has a matching .meta file with complete metadata.
- Files with no magic signature and no YARA match are recorded as Unknown / Binary in the report.

Decision Point:

- If scanner crashes → examine error log, fix code, and re-run.
- If YARA compilation fails → revert to next-best validated rule set.

Expected Outcome: Complete scan of all 220 files; all malicious files isolated in quarantine with full metadata; comprehensive scan log.

7. Results Compilation and Evaluation

Goal: Quantitatively and qualitatively assess scanner performance, categorise findings by file type, and critically evaluate investigation outcomes.

- Calculate summary statistics:
  - Total files scanned
  - Files detected as malicious
  - Files quarantined
  - Files with unknown/unrecognised type
  - Total scan duration (seconds)

- Group results by magic-number file type (PDF, EXE, DOCX, ZIP, etc.).
- Compute detection rate per file type.
- Calculate performance metrics:
  - **True Positive Rate (TPR)** = (Malicious correctly detected) / (Total actual malicious)
  - **False Positive Rate (FPR)** = (Clean files incorrectly flagged) / (Total clean files)
  - **Processing speed** = Total files / Scan duration (files/second)

- If ground truth is available, compare detection results against it.
- If no ground truth, reason about plausibility based on file types and YARA rule patterns.
- Document limitations: missed detections, performance bottlenecks, design flaws.
- Propose potential improvements.

Expected Outcome: A quantitative and qualitative assessment of scanner effectiveness, ready for inclusion in the final report.

8. Documentation and Reporting

Goal: Produce a complete, repeatable, and professionally formatted LAR-0 report that meets all project requirements and enables full reproduction of the investigation.

- Compile all screenshots taken during the procedure:
  - File list in Victim VM
  - VM snapshot
  - FTK Imager imaging and verification
  - AIM mount and Disk Management
  - Scanner console output
  - Quarantine folder contents

- Insert screenshots into the report with descriptive captions.
- Create two required diagrams using Draw.io or equivalent:
  - **Phase Diagram** – high-level overview of the eight procedural phases.
  - **Flowchart** – detailed decision logic of the scanner engine.

- Populate the AI comparison matrix with metrics from 6.5.4.
- Clearly state the winner and justify the selection.
- Assemble source code appendix:
  - Concatenate all .py and .yar files into appendix_source_code.txt.
  - Use clear separators (e.g., === filename.py ===) between files.
- Conduct final compliance check against project rubric:
  - 220 files verified (screenshot)
  - Victim VM snapshot evidence
  - Forensic image hash verification
  - AIM physical disk mount screenshot
  - Magic number detection with citation of Gary Kessler's table
  - MD5, SHA1, SHA256 hashes for each file (sample or full listing)
  - YARA comparison and best rule set justification
  - Quarantine with .quarantine extension and .meta files
  - Complete source code in appendix
- Upload to the designated submission portal.

Expected Outcome: A polished, submission-ready forensic report that demonstrates mastery of MP1 learning objectives and enables full repeatability of the investigation.

Safety or Compliance Considerations (if applicable)

All investigative activities were conducted under strict safety and compliance protocols to prevent malware propagation, preserve evidence integrity, and ensure legal and ethical adherence. Network isolation was enforced on both the Victim VM and the Digital Forensics (DF) Workstation by disabling all network adapters prior to evidence handling. Screenshots of the disabled network settings were captured to document this isolation. Malware containment was further ensured by mounting the forensic image exclusively using Arsenal Image Mounter's *Disk device, write temporary* mode, which redirected all write operations to a differencing file while leaving the original E01 image unmodified.

The custom Python malware scanner performed only static analysis; no files were executed, emulated, or dynamically analyzed. Quarantine was implemented as a copy-based operation, preserving the original evidence on the mounted image. All suspected malware files were stored in a dedicated *./quarantine/* directory with a *.quarantine* extension to prevent accidental execution, and each quarantined file was accompanied by a *.meta* file containing the original path, matched YARA rules, and cryptographic hash values. In the event of accidental malware execution, a predefined incident response procedure required immediate virtual machine power-off via the hypervisor, host isolation, and instructor notification.

Evidence integrity was maintained through strict chain-of-custody documentation and cryptographic hash verification, in accordance with ISO/IEC 27037:2012 guidelines for digital evidence preservation (International Organization for Standardization, 2012). MD5 and SHA1 hashes were generated during image acquisition and re-verified after each transfer to the DF Workstation. Any hash mismatch would have halted the investigation and required re-acquisition. All evidence movements were recorded with timestamps, custodians, and associated hash values to ensure full traceability and forensic soundness.

All software tools used were legally obtained and appropriately licensed. Arsenal Image Mounter was used in Free Mode for academic purposes, FTK Imager was obtained from official sources, and all Python libraries used were open-source under OSI-approved licenses. Both virtual machines operated on properly licensed operating systems, and a software inventory was documented in the final report.

Legal and ethical compliance was strictly observed throughout the project. Malware samples were obtained solely from the official NSSECU3 repository and were never transferred outside the isolated forensic environment or uploaded to public analysis services. All code, YARA rules, and documentation were original work; AI-assisted content was properly attributed, reviewed, and modified. An Academic Integrity Statement was included in the final report.

Upon project completion, secure disposal of temporary evidence copies—including VM snapshots, differencing files, quarantine directories, and logs—was performed following approved data sanitization practices. The original forensic image and final report were retained only until instructor authorization for disposal was granted. A documented disposal record was completed to finalize the case.

References

Arsenal Consulting, Inc. (2015, September 9). *Arsenal Image Mounter v2.0.009 released* [Online forum post]. Forensic Focus Forums. https://www.forensicfocus.com/forums/general/arsenal-image-mounter-v2-0-009-released/

Arsenal Recon. (2025). *Arsenal Image Mounter*. https://arsenalrecon.com/weapons/image-mounter

Exterro / AccessData. (2023). *FTK Imager user guide*. https://accessdata.com/product-documentation

International Organization for Standardization. (2012). *Information technology — Security techniques — Guidelines for identification, collection, acquisition and preservation of digital evidence (ISO/IEC Standard No. 27037:2012)*. https://www.iso.org/standard/44381.html

Kessler, G. C. (2021, June). *File signatures table*. https://www.garykessler.net/library/file_sigs.html

Massachusetts Institute of Technology. (2023). *FTK Imager*. In *Archives Processing Documentation*. https://wikis.mit.edu/confluence/display/ARCHIVESPROCESSING/FTKImager

National Institute of Standards and Technology. (2023). *Digital investigation techniques: A NIST scientific foundation review (NISTIR 8354)*. U.S. Department of Commerce. https://doi.org/10.6028/NIST.IR.8354

Python Software Foundation. (2023). *hashlib — Secure hashes and message digests (Python 3.13 documentation)*. https://docs.python.org/3/library/hashlib.html

University of Chicago Library. (2023). *FTK Imager*. In *Digital Special Collections Public Wiki*. https://github.com/uchicago-library/Digital-Special-Collections-Public/wiki/FTK-Imager

Appendices

Appendix A: Virtual Machine Environment Setup

This appendix documents the virtualized laboratory environment used throughout the MP1 Malware Scanner project to ensure forensic isolation, evidence integrity, and repeatability of analysis.

A.1 VMware Workstation Configuration

VMware Workstation was used as the primary virtualization platform for all systems involved in the project. Virtual networking was configured to fully isolate the laboratory environment from the host system and external networks. All network adapters (NAT, Bridged, and Host-Only) were disabled prior to evidence handling to prevent unintended malware propagation.

VMware snapshots were enabled and utilized to preserve system states at critical phases of the project, particularly before forensic acquisition, ensuring controlled rollback and repeatability.

A.2 Victim-Target Virtual Machine Configuration

- **Operating System:** Windows 10 version 1903 (x64)
- **Virtual Disk Size:** 40 GB
- **Memory Allocation:** 2 GB RAM

Purpose:

- Serve as the controlled evidence source for the MP1 dataset
- Store the 220 malware sample files prior to acquisition
- Support directory verification and manifest generation
- Facilitate forensic disk imaging using standalone FTK Imager

No analysis or scanning was performed on the Victim VM. After acquisition, the system was snapshotted and powered off to preserve evidence integrity.

A.3 Digital Forensics (DF) Workstation Configuration

- **Operating System:** Windows 11 version 23H2 (x64)
- **Virtual Disk Size:** 60 GB
- **Memory Allocation:** 2 GB RAM

Purpose:

- Dedicated environment for forensic mounting and analysis
- Mounting of forensic disk images using Arsenal Image Mounter
- Development and execution of the custom Python malware scanner
- Generation of forensic reports and quarantine artifacts

All investigative and analytical activities were performed exclusively on the DF Workstation.

A.4 Snapshot Usage and Management

Snapshots were created at key stages of the project, including:

- Clean baseline state of the Victim VM
- Pre-acquisition state (MP1_BEFORE_ACQUISITION)

Snapshots were used to:

- Preserve the original evidence environment
- Prevent accidental modification of malware samples
- Support repeatability of forensic acquisition steps

A.5 Shared Folder Configuration and Evidence Transfer

VMware Shared Folders were configured between the Victim VM and the DF Workstation solely for evidence transfer purposes. Shared folders were used exclusively to transfer the E01 forensic image after acquisition.

Once transferred, the evidence was treated as read-only. Shared folders were not used during scanning or analysis to minimize the risk of cross-VM contamination.

Appendix B: Malware Sample Dataset Inventory

This appendix documents the malware dataset used in MP1.

- Total number of files: **220**
- Filename format: File001 to File220
- Storage location:
  C:\Users\<Victim>\Documents\MP1_Files\

A file manifest was generated using:

dir /b > file_manifest.txt

Each file was treated as an unknown sample and analyzed solely through static methods.

Appendix C: Forensic Disk Image Artifacts

- **Image Format:** E01 (EnCase Evidence File)
- **Acquisition Tool:** FTK Imager version 4.7.3
- **Acquisition Method:** Contents of a Folder
- **Hashes Generated:** MD5 and SHA1

Hash values were recorded at acquisition and re-verified after transfer to the DF Workstation.

Appendix D: Forensic Mounting Artifacts

This appendix documents the mounting of the forensic image.

- **Tool:** Arsenal Image Mounter version 3.7.281
- **Mount Mode:** Disk device, write temporary
- **Disk Type:** Physical SCSI disk emulation
- **Assigned Drive Letter:** Documented during analysis

Screenshots include:

- AIM mounted image details
- Windows Disk Management view
- File Explorer view confirming access to all 220 files

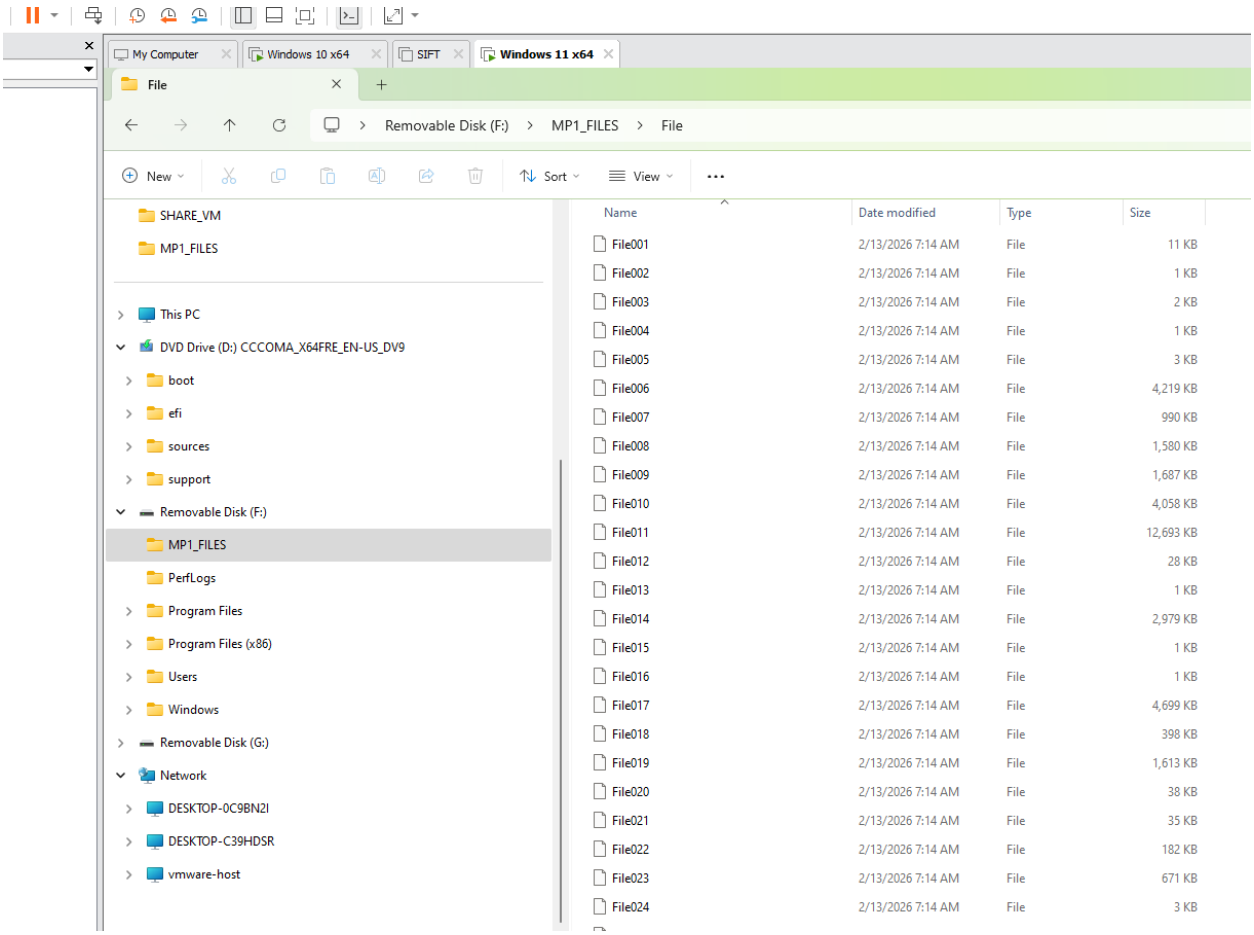


Figure D-1. 220 Files can be seen in Removable Disk (F:)

Appendix E: Malware Scanner Architecture and Source Modules

This appendix contains documentation of the custom Python-based malware scanner.

E.1 Scanner Modules

- magic_detector.py – File type identification using magic numbers
- hash_utils.py – MD5, SHA1, SHA256 hash computation
- yara_handler.py – YARA compilation and scanning logic
- quarantine.py – Forensic-safe quarantine handling
- scanner.py – Recursive scanning engine
- reporter.py – JSON and CSV report generation
- main.py – Command-line interface and argument parsing

E.2 Diagrams

- Phase Diagram (8-phase system architecture)

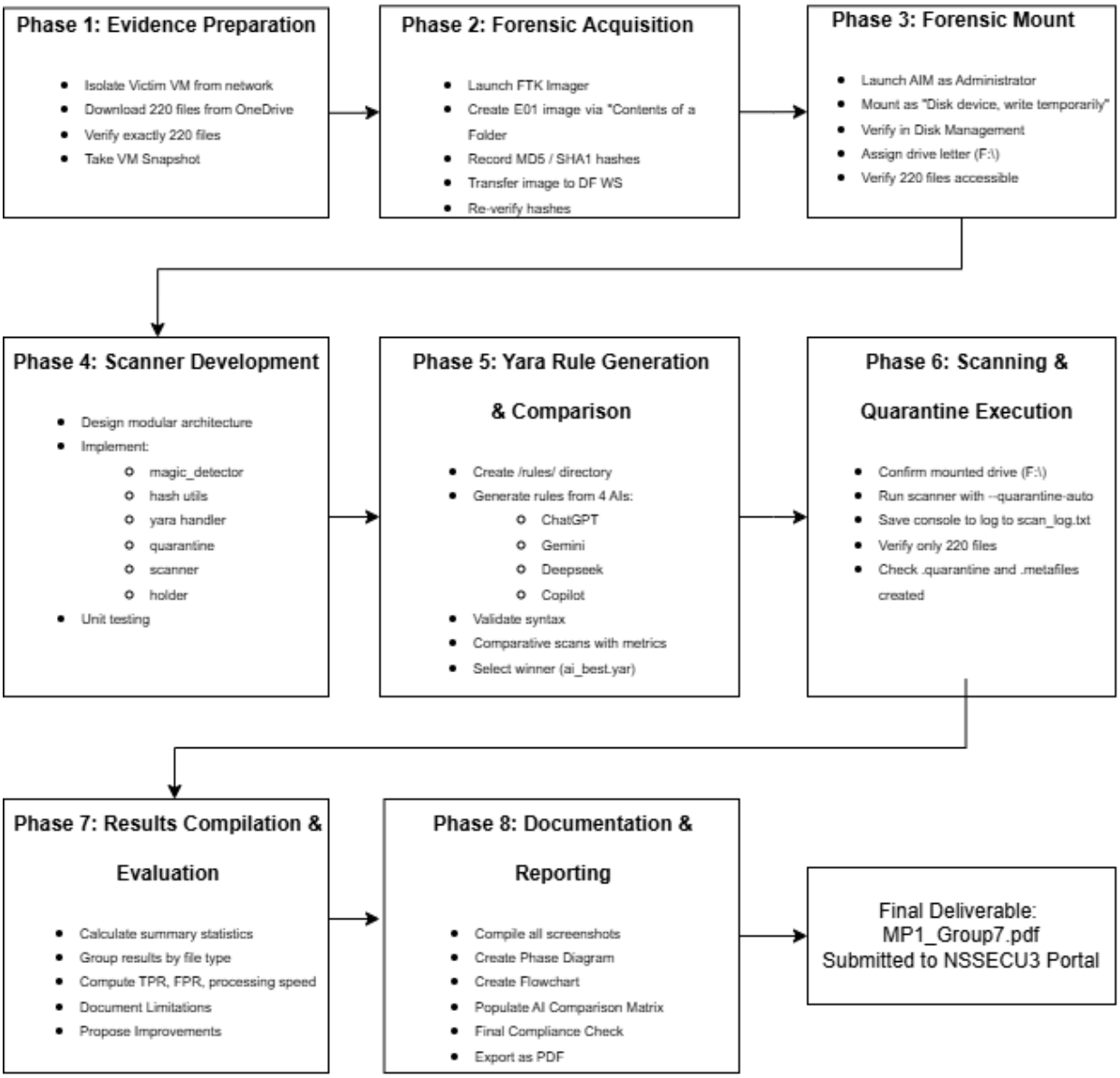


Figure E-1 Eight Phase Malware Scanner Forensic Workflow

- Flowchart (scanner decision logic)

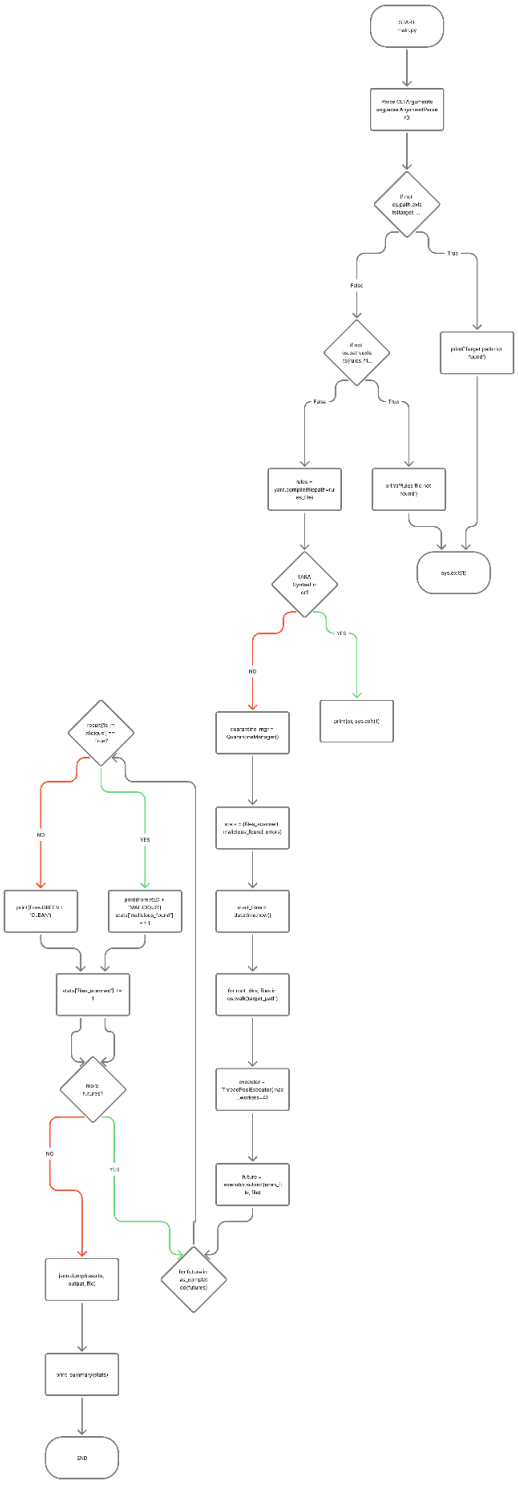


Figure E-2 Code-based decision logic flowchart of the MP1 Malware Scanner main execution flow (main.py).

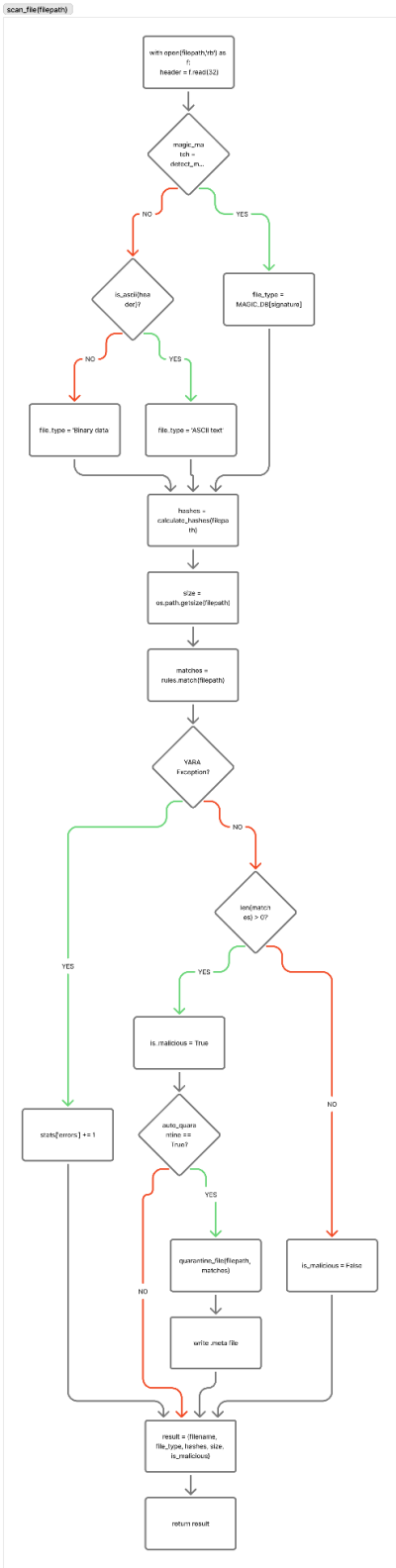


Figure E-2.1 Code-based decision logic flowchart of the scan_file() function used for per-file malware analysis.

Appendix F: YARA Rule Generation and AI Comparison

This appendix documents the AI-assisted YARA rule generation process.

- AI platforms used:
  - ChatGPT
  - Deepseek
  - Gemini
  - Copilot

Each rule set was:

- Generated using a standardized prompt
- Validated using yara.compile()
- Evaluated using detection metrics and best-practice criteria

The selected rule set (ai_best.yar) is included with justification.

Appendix G: Quarantine and Reporting Artifacts

- Screenshot of ./quarantine/ directory
- Sample .quarantine file
- Sample .meta file containing:
  - Original file path
  - Timestamp
  - Matched YARA rules
  - MD5, SHA1, SHA256 hashes

Scan reports are provided in:

- JSON format
- CSV format

Appendix H: Tool Limitations and Observations

Appendix I: YARA Rule

AI Platform	File Name	File Size	Rules Count	Detection Rate
ChatGPT	ai_chatgpt_rules.yar	5,276 bytes	6	2.3% (5 files)
Deepseek	ai_deepseek_rules.yar	13,638 bytes	8	50.5% (111 files)
Gemini	ai_gemini_rules.yar	10,765 bytes	12	0.5% (1 file)
Copilot	ai_copilot_rules.yar	5,707 bytes	12	5.0% (11 files)

Table I-1. YARA Rule Sets Summary

I.1 ChatGPT-Generated YARA Rules

```
/*
* YARA Rules for Common Malware Detection
* Author: Generated for MP1 Malware Forensics Project: ChatGPT GPT-5.2
* Date: 2026-02-13
* Description: Detects common malware patterns in PE, PDF, Office, JS, and PowerShell files
*/

import "pe"
import "math"

/* ============================================================
   1. Generic PE Malware Indicators
   ============================================================ */
rule PE_Generic_Malware_Indicators
{
    meta:
        category = "pe"
        confidence = "medium"
        description = "Detects suspicious PE characteristics such as high entropy, odd sections, and
packing"
        author = "ChatGPT GPT-5.2"

    strings:
        $packer1 = "UPX!" ascii
        $packer2 = "MPRESS" ascii
```

```
        $packer3 = "Themida" ascii
        $suspicious_api1 = "VirtualAlloc" ascii
        $suspicious_api2 = "WriteProcessMemory" ascii
        $suspicious_api3 = "CreateRemoteThread" ascii

    condition:
        uint16(0) == 0x5A4D and
        pe.is_pe and
        (
            any of ($packer*) or
            (
                pe.number_of_sections > 6 and
                math.entropy(pe.sections[0].raw_data_offset, pe.sections[0].raw_data_size) > 7.0
            ) or
            all of ($suspicious_api*)
        )
}

/* ============================================================
   2. PDF with JavaScript or Embedded Objects
   ============================================================ */
rule PDF_Embedded_JavaScript
{
    meta:
        category = "pdf"
        confidence = "high"
        description = "Detects PDFs with embedded JavaScript or suspicious objects"
        author = "ChatGPT GPT-5.2"

    strings:
        $pdf_magic = "%PDF-" ascii
        $js1 = "/JavaScript" ascii
        $js2 = "/JS" ascii
        $action = "/OpenAction" ascii
        $objstm = "/ObjStm" ascii
        $stream = "stream" ascii

    condition:
        $pdf_magic at 0 and
        2 of ($js1, $js2, $action, $objstm, $stream)
}

/* ============================================================
   3. Office Documents with Macros
   ============================================================ */
rule Office_Macro_Malware
{
    meta:
```

```
        category = "office"
        confidence = "high"
        description = "Detects malicious VBA macro keywords in Office documents"
        author = "ChatGPT GPT-5.2"

    strings:
        $ole_magic = { D0 CF 11 E0 A1 B1 1A E1 }
        $autoopen1 = "AutoOpen" ascii nocase
        $autoopen2 = "Document_Open" ascii nocase
        $shell = "Shell(" ascii nocase
        $powershell = "powershell" ascii nocase
        $wscript = "WScript.Shell" ascii nocase
        $base64 = "Base64Decode" ascii nocase

    condition:
        $ole_magic at 0 and
        2 of ($autoopen*, $shell, $powershell, $wscript, $base64)
}

/* ============================================================
   4. Suspicious PowerShell Commands
   ============================================================ */
rule PowerShell_Suspicious_Commands
{
    meta:
        category = "powershell"
        confidence = "high"
        description = "Detects obfuscated or malicious PowerShell behavior"
        author = "ChatGPT GPT-5.2"

    strings:
        $ps1 = "powershell" ascii nocase
        $enc = "-enc" ascii nocase
        $bypass = "ExecutionPolicy Bypass" ascii nocase
        $iex = "Invoke-Expression" ascii nocase
        $download = "DownloadString" ascii nocase
        $hidden = "-windowstyle hidden" ascii nocase

    condition:
        $ps1 and
        2 of ($enc, $bypass, $iex, $download, $hidden)
}

/* ============================================================
   5. JavaScript Malware / Backdoor Patterns
   ============================================================ */
rule JavaScript_Malware_Backdoor
{
```

```
    meta:
        category = "javascript"
        confidence = "medium"
        description = "Detects suspicious JavaScript obfuscation and backdoor-like behavior"
        author = "ChatGPT GPT-5.2"

    strings:
        $eval = "eval(" ascii nocase
        $fromchar = "String.fromCharCode" ascii nocase
        $unescape = "unescape(" ascii nocase
        $xhr = "XMLHttpRequest" ascii nocase
        $fetch = "fetch(" ascii nocase
        $c2 = "http://" ascii nocase
        $c2s = "https://" ascii nocase

    condition:
        3 of ($eval, $fromchar, $unescape, $xhr, $fetch) and
        any of ($c2*)
}

/* ============================================================
   6. Generic Backdoor / RAT Indicators
   ============================================================ */
rule Generic_Backdoor_Patterns
{
    meta:
        category = "backdoor"
        confidence = "medium"
        description = "Detects common backdoor and RAT command patterns"
        author = "ChatGPT GPT-5.2"

    strings:
        $cmd1 = "cmd.exe /c" ascii nocase
        $cmd2 = "whoami" ascii nocase
        $cmd3 = "net user" ascii nocase
        $cmd4 = "net localgroup" ascii nocase
        $persist1 = "Run\\\\" ascii nocase
        $persist2 = "CurrentVersion\\\\Run" ascii nocase

    condition:
        2 of ($cmd*) or
        any of ($persist*)
}
```

I.2 Deepseek-Generated YARA Rules

```
/*
YARA Rules for Common Malware Detection
Author: Generated for MP1 Malware Forensics Project: DEEPSEEK
```

```
Date: 2026-02-13
Description: Detects common malware patterns in PE, PDF, Office, JS, and PowerShell files
*/

import "pe"
import "math"

rule SUSPICIOUS_PE_HighEntropy_Sections
{
    meta:
        author = "deepseek"
        description = "Detects PE files with high entropy sections (potential packed/encrypted malware)"
        date = "2026-02-13"
        score = 85
        fileType = "PE_EXECUTABLE"

    condition:
        uint16(0) == 0x5A4D and pe.is_pe and
        (
            (
                for any i in (0..pe.number_of_sections - 1) :
                    (
                        (pe.sections[i].name == ".text" or
                         pe.sections[i].name == "UPX0" or
                         pe.sections[i].name == ".aspack") and
                        pe.sections[i].raw_data_size > 0 and
                        math.entropy(
                            pe.sections[i].raw_data_offset,
                            pe.sections[i].raw_data_size
                        ) > 6.8
                    )
            )
            or
            (
                for any i in (0..pe.number_of_sections - 1) :
                    (
                        pe.sections[i].name == ".upx0" or
                        pe.sections[i].name == ".aspack" or
                        pe.sections[i].name == ".adata" or
                        pe.sections[i].name == "PACKED"
                    )
            )
            or
            (
                for any i in (0..pe.number_of_sections - 1) :
                    (
                        pe.sections[i].characteristics & pe.SECTION_MEM_WRITE and
                        pe.sections[i].characteristics & pe.SECTION_MEM_EXECUTE
```

```
            )
        )
    ) and
    not pe.is_signed and
    pe.number_of_signatures == 0
}



rule MALICIOUS_PDF_JavaScript_Objects
{
    meta:
        author = "deepseek"
        description = "Detects PDF files containing JavaScript objects (potential malicious PDF)"
        date = "2026-02-13"
        score = 90
        fileType = "PDF_DOCUMENT"

    strings:
        $magic = { 25 50 44 46 }
        $js1 = "/JavaScript" nocase
        $js2 = "/JS" nocase
        $launch = "/Launch" nocase
        $embedded = "/EmbeddedFile" nocase
        $openaction = "/OpenAction" nocase
        $uri = "/URI" nocase

    condition:
        $magic at 0 and
        ($js1 or $js2) and
        ($launch or $embedded or $openaction or $uri) and
        filesize < 10MB
}



rule MALICIOUS_PDF_Exploit_Strings
{
    meta:
        author = "deepseek"
        description = "Detects PDF files with exploit-related strings"
        date = "2026-02-13"
        score = 85
        fileType = "PDF_DOCUMENT"

    strings:
        $magic = { 25 50 44 46 }
        $heapspray1 = "/UI" nocase
```

```
        $heapspray2 = "/Collab" nocase
        $collab = "/Collab.getIcon" nocase
        $util = "app.util" nocase
        $eval = "eval(" nocase ascii
        $ek1 = "/BlackHole" nocase
        $ek2 = "/Angler" nocase
        $ek3 = "/Nuclear" nocase
        $shellcode1 = { 68 ?? ?? ?? ?? 68 ?? ?? ?? ?? E8 ?? ?? ?? ?? }
        $shellcode2 = { 31 C9 64 8B 71 30 }

    condition:
        $magic at 0 and
        (
            $heapspray1 or $heapspray2 or
            ($collab and $util) or
            $eval or
            $ek1 or $ek2 or $ek3 or
            $shellcode1 or $shellcode2
        )
}

rule MALICIOUS_OFFICE_Macros_VBA
{
    meta:
        author = "deepseek"
        description = "Detects Office documents with VBA macros (potential malware)"
        date = "2026-02-13"
        score = 95
        fileType = "OFFICE_DOCUMENT"

    strings:
        $ole_magic = { D0 CF 11 E0 A1 B1 1A E1 }
        $zip_magic = { 50 4B 03 04 }
        $vba1 = "VBA" nocase
        $vba2 = "Module=" nocase
        $shell = "Shell(" nocase ascii
        $createobject = "CreateObject(" nocase ascii
        $wscript = "WScript.Shell" nocase ascii
        $powershell = "PowerShell" nocase ascii
        $cmd = "cmd.exe" nocase ascii
        $download = "URLDownloadToFile" nocase ascii
        $xmlhttp = "XMLHTTP" nocase ascii
        $adodb = "ADODB.Stream" nocase ascii
        $auto_open = "Auto_Open" nocase ascii
        $auto_close = "Auto_Close" nocase ascii
        $document_open = "Document_Open" nocase ascii
        $workbook_open = "Workbook_Open" nocase ascii
```

```
    condition:
        ($ole_magic at 0 or $zip_magic at 0) and
        ($vba1 or $vba2) and
        (
            $shell or $createobject or $wscript or
            $powershell or $cmd or $download or
            $xmlhttp or $adodb
        ) and
        ($auto_open or $auto_close or $document_open or $workbook_open)
}

rule MALICIOUS_JavaScript_Obfuscated
{
    meta:
        author = "deepseek"
        description = "Detects obfuscated/deobfuscation patterns in JavaScript"
        date = "2026-02-13"
        score = 85
        fileType = "JAVASCRIPT"

    strings:
        $eval = "eval(" nocase ascii
        $document_write = "document.write(" nocase ascii
        $unescape = "unescape(" nocase ascii
        $decodeuri = "decodeURIComponent(" nocase ascii
        $fromcharcode = "String.fromCharCode(" nocase ascii
        $replace = ".replace(" nocase ascii
        $long_hex = /[0-9a-fA-F]{40,}/
        $long_num = /[0-9]{50,}/
        $activex = "ActiveXObject" nocase ascii
        $wscript = "WScript" nocase ascii
        $adodb = "ADODB" nocase ascii
        $xmlhttp = "XMLHttpRequest" nocase ascii

    condition:
        filesize < 1MB and
        (
            (
                ($eval or $document_write) and
                ($unescape or $decodeuri or $fromcharcode) and
                (#long_hex > 1 or #long_num > 1)
            ) or
            (
                ($activex or $wscript or $adodb or $xmlhttp) and
                ($eval or $unescape or $fromcharcode)
            )
        ) and
        any of ($replace)
```

```
}

rule MALICIOUS_PowerShell_Suspicious_Commands
{
    meta:
        author = "deepseek"
        description = "Detects suspicious PowerShell commands used by malware"
        date = "2026-02-13"
        score = 90
        fileType = "POWERSHELL_SCRIPT"

    strings:
        $ep_bypass1 = "-ExecutionPolicy Bypass" nocase ascii
        $ep_bypass2 = "Set-ExecutionPolicy" nocase ascii
        $hidden1 = "-WindowStyle Hidden" nocase ascii
        $hidden2 = "-WindowsStyle Hidden" nocase ascii
        $hidden3 = "-NoProfile" nocase ascii
        $download1 = "System.Net.WebClient" nocase ascii
        $download2 = "DownloadFile(" nocase ascii
        $download3 = "DownloadString(" nocase ascii
        $download4 = "Invoke-WebRequest" nocase ascii
        $download5 = "Invoke-RestMethod" nocase ascii
        $b64_flag1 = "-EncodedCommand" nocase ascii
        $b64_flag2 = "-enc" nocase ascii
        $reflection1 = "Add-Type" nocase ascii
        $backtick = "`" ascii
        $replace_ops = ".Replace(" nocase ascii

    condition:
        (
            ($ep_bypass1 or $ep_bypass2) and
            ($hidden1 or $hidden2 or $hidden3) and
            ($download1 or $download2 or $download3 or $download4 or $download5)
        ) or
        $b64_flag1 or $b64_flag2 or
        $reflection1 or
        (#backtick > 10 and $replace_ops)
}

rule MALICIOUS_Backdoor_Persistence
{
    meta:
        author = "deepseek"
        description = "Detects common backdoor persistence mechanisms"
        date = "2026-02-13"
        score = 90
        fileType = "VARIOUS"
```

```
    strings:
        $reg_run1 = "HKEY_LOCAL_MACHINE\\Software\\Microsoft\\Windows\\CurrentVersion\\Run"
nocase ascii
        $reg_run2 = "HKCU\\Software\\Microsoft\\Windows\\CurrentVersion\\Run" nocase ascii
        $reg_run3 = "SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run" nocase ascii
        $schtasks1 = "schtasks" nocase ascii
        $schtasks2 = "/create" nocase ascii
        $schtasks3 = "/sc daily" nocase ascii
        $schtasks4 = "/sc onlogon" nocase ascii
        $startup1 = "Startup" nocase ascii
        $startup2 = "All Users\\Start Menu\\Programs\\Startup" nocase ascii
        $service1 = "CreateService" nocase ascii
        $service2 = "sc.exe create" nocase ascii
        $service3 = "New-Service" nocase ascii
        $c2_http = "http://" nocase ascii
        $c2_https = "https://" nocase ascii
        $c2_post = "POST" nocase ascii
        $c2_get = "GET" nocase ascii
        $cmd_shell = "cmd.exe /c" nocase ascii
        $ps_shell = "powershell.exe" nocase ascii

    condition:
        $reg_run1 or $reg_run2 or $reg_run3 or
        ($schtasks1 and ($schtasks2 or $schtasks3 or $schtasks4)) or
        $startup1 or $startup2 or
        $service1 or $service2 or $service3 or
        $c2_http or $c2_https or $c2_post or $c2_get or
        $cmd_shell or $ps_shell or
        (($c2_http or $c2_https or $c2_post or $c2_get) and
         ($reg_run1 or $reg_run2 or $schtasks1 or $startup1))
}

rule MALICIOUS_Generic_Suspicious_Strings
{
    meta:
        author = "deepseek"
        description = "Detects generic suspicious strings commonly found in malware"
        date = "2026-02-13"
        score = 70
        fileType = "VARIOUS"

    strings:
        $s1 = "MALWARE" nocase ascii wide
        $s2 = "BACKDOOR" nocase ascii wide
        $s3 = "RAT" nocase ascii wide
        $s4 = "DROPPER" nocase ascii wide
        $s5 = "PAYLOAD" nocase ascii wide
        $s6 = "EXPLOIT" nocase ascii wide
```

```
        $s7 = "CRYPT" nocase ascii wide
        $s8 = "INJECT" nocase ascii wide
        $s9 = "KEYLOG" nocase ascii wide
        $s10 = "PASSWORD STEALER" nocase ascii wide
        $s11 = "BYPASS" nocase ascii wide
        $s12 = "PRIVILEGE ESCALATION" nocase ascii wide

    condition:
        2 of ($s*)
}
```

I.3 Gemini-Generated YARA Rules

```
/*
YARA Rules for Common Malware Detection
Author: Generated for MP1 Malware Forensics Project: GEMINI
Date: 2026-02-13
Description: Detects common malware patterns in PE, PDF, Office, JS, and PowerShell files
*/

import "pe"
import "math"

// ==============================================================================
// 1. PE EXECUTABLES (4 Rules)
// ==============================================================================

/*
 * Rule Name: SUSPICIOUS_PE_KnownPackerSections
 * Target: Windows PE Executables
 * Technique: Identification of non-standard section names used by packers/protectors
 * False Positives: Rare, as these names are specific to packing tools
 */
rule SUSPICIOUS_PE_KnownPackerSections {
    meta:
        author = "Gemini"
        description = "Detects PE files using known packer section names like UPX, ASPack, and
VMProtect"
        date = "2026-02-13"
        score = 85
        fileType = "PE_EXECUTABLE"
        reference = "MP1 Malware Forensics Project - Gemini AI Comparison"
    strings:
        // Section names are typically 8 bytes, null-padded
        $s1 = "UPX0"
        $s2 = "UPX1"
        $s3 = ".aspack"
        $s4 = ".adata"
```

```
        $s5 = ".vmp0"
        $s6 = ".vmp1"
        $s7 = ".enigma"
        $s8 = ".petite"
    condition:
        uint16(0) == 0x5A4D and pe.is_pe and
        any of ($s*)
}

/*
 * Rule Name: SUSPICIOUS_PE_HighEntropySections
 * Target: Windows PE Executables
 * Technique: Entropy analysis to find encrypted or compressed code/data
 * False Positives: Heavily compressed legitimate installers or game assets
 */
rule SUSPICIOUS_PE_HighEntropySections {
    meta:
        author = "Gemini"
        description = "Detects PE files with sections exceeding entropy thresholds (indicative of
packing/encryption)"
        date = "2026-02-13"
        score = 80
        fileType = "PE_EXECUTABLE"
        reference = "MP1 Malware Forensics Project - Gemini AI Comparison"
    condition:
        uint16(0) == 0x5A4D and pe.is_pe and pe.number_of_signatures == 0 and
        /* Iterate through sections to check entropy.
           Entropy > 7.5 is a strong indicator of encryption/packing.
        */
        for any i in (0..pe.number_of_sections - 1): (
            math.entropy(pe.sections[i].raw_data_offset, pe.sections[i].raw_data_size) > 7.5
        )
}

/*
 * Rule Name: SUSPICIOUS_PE_ProcessInjection_APIs
 * Target: Windows PE Executables
 * Technique: Core APIs used for Cross-Process Memory Injection
 * False Positives: Debuggers, Security software, specialized system utilities
 */
rule SUSPICIOUS_PE_ProcessInjection_APIs {
    meta:
        author = "Gemini"
        description = "Detects combination of APIs used to inject code into remote processes"
        date = "2026-02-13"
        score = 90
        fileType = "PE_EXECUTABLE"
        reference = "MP1 Malware Forensics Project - Gemini AI Comparison"
```

```
    strings:
        $api1 = "VirtualAllocEx"
        $api2 = "WriteProcessMemory"
        $api3 = "CreateRemoteThread"
        $api4 = "NtCreateThreadEx"
        $api5 = "QueueUserAPC"
    condition:
        uint16(0) == 0x5A4D and pe.is_pe and pe.number_of_signatures == 0 and
        (3 of ($api*))
}

/*
 * Rule Name: SUSPICIOUS_PE_PersistenceAndEvasion
 * Target: Windows PE Executables
 * Technique: Registry persistence combined with anti-debugging checks
 * False Positives: Monitoring tools or complex legitimate software protections
 */
rule SUSPICIOUS_PE_PersistenceAndEvasion {
    meta:
        author = "Gemini"
        description = "Detects persistence mechanisms combined with anti-forensic evasion APIs"
        date = "2026-02-13"
        score = 75
        fileType = "PE_EXECUTABLE"
        reference = "MP1 Malware Forensics Project - Gemini AI Comparison"
    strings:
        $reg1 = "RegSetValueEx"
        $reg2 = "RegCreateKeyEx"
        $eva1 = "IsDebuggerPresent"
        $eva2 = "NtQueryInformationProcess"
        $eva3 = "CheckRemoteDebuggerPresent"
        $sch1 = "schtasks.exe" wide ascii
    condition:
        uint16(0) == 0x5A4D and pe.is_pe and
        (
            (any of ($reg*)) or $sch1
        ) and
        (any of ($eva*)) and
        pe.number_of_signatures == 0
}


// ==============================================================================
// 2. PDF DOCUMENTS (2 Rules)
// ==============================================================================

/*
 * Rule Name: MALICIOUS_PDF_JavaScriptExecution
```

```
 * Target: PDF Documents
 * Technique: Automatic execution of JavaScript upon opening the document
 * False Positives: Interactive PDF forms
 */
rule MALICIOUS_PDF_JavaScriptExecution {
    meta:
        author = "Gemini"
        description = "Detects PDF files that contain both JavaScript and Auto-Open actions"
        date = "2026-02-13"
        score = 85
        fileType = "PDF_DOCUMENT"
        reference = "MP1 Malware Forensics Project - Gemini AI Comparison"
    strings:
        $magic = { 25 50 44 46 } // %PDF
        $s1 = "/JavaScript"
        $s2 = "/JS"
        $s3 = "/OpenAction"
        $s4 = "/AcroForm"
    condition:
        $magic at 0 and (all of ($s1, $s2, $s3) or all of ($s2, $s3, $s4))
}

/*
 * Rule Name: MALICIOUS_PDF_EmbeddedExploit
 * Target: PDF Documents
 * Technique: Embedding malicious files or using Launch actions for execution
 * False Positives: Complex portfolio PDFs
 */
rule MALICIOUS_PDF_EmbeddedExploit {
    meta:
        author = "Gemini"
        description = "Detects high-risk PDF tags used for embedding and launching external payloads"
        date = "2026-02-13"
        score = 90
        fileType = "PDF_DOCUMENT"
        reference = "MP1 Malware Forensics Project - Gemini AI Comparison"
    strings:
        $magic = { 25 50 44 46 }
        $e1 = "/EmbeddedFile"
        $e2 = "/Launch"
        $e3 = "/URI"
        $e4 = "unescape" // Often used in heap sprays within PDF JS
    condition:
        $magic at 0 and 2 of ($e*) and filesize < 10MB
}

// ==============================================================================
// 3. OFFICE DOCUMENTS (2 Rules)
```

```
// ==============================================================================

/*
 * Rule Name: TROJAN_Office_AutoExecMacro
 * Target: Office Documents (.doc, .xls, .docx, .xlsx)
 * Technique: VBA macros that execute automatically to run malicious commands
 * False Positives: Corporate documents with legitimate automation
 */
rule TROJAN_Office_AutoExecMacro {
   meta:
      author = "Gemini"
      description = "Detects Office documents with VBA macros that auto-execute on opening"
      date = "2026-02-13"
      score = 85
      fileType = "OFFICE_DOCUMENT"
      reference = "MP1 Malware Forensics Project - Gemini AI Comparison"
   strings:
      $magic_ooxml = { 50 4B 03 04 } // ZIP-based (Docx, Xlsx)
      $magic_ole = { D0 CF 11 E0 }   // OLE-based (Doc, Xls)

      $a1 = "Auto_Open"
      $a2 = "Document_Open"
      $a3 = "Workbook_Open"

      $s1 = "WScript.Shell"
      $s2 = "CreateObject"
      $s3 = "Shell.Application"

   condition:
      ($magic_ooxml at 0 or $magic_ole at 0) and
      (any of ($a*)) and (any of ($s*))
}

/*
 * Rule Name: MALICIOUS_Office_PowerShellDownload
 * Target: Office Documents
 * Technique: VBA macro launching PowerShell to download a payload
 * False Positives: Extremely rare in legitimate business contexts
 */
rule MALICIOUS_Office_PowerShellDownload {
   meta:
      author = "Gemini"
      description = "Detects Office macros calling PowerShell with download and bypass flags"
      date = "2026-02-13"
      score = 95
      fileType = "OFFICE_DOCUMENT"
      reference = "MP1 Malware Forensics Project - Gemini AI Comparison"
   strings:
```

```
        $p1 = "powershell.exe" ascii wide
        $p2 = "-ExecutionPolicy Bypass" ascii wide
        $p3 = "-WindowStyle Hidden" ascii wide
        $p4 = "URLDownloadToFile" ascii wide
        $p5 = "MSXML2.XMLHTTP" ascii wide
    condition:
        (uint32(0) == 0x04034B50 or uint32(0) == 0xE11AB1D0) and
        (any of ($p1, $p2, $p3)) and (any of ($p4, $p5))
}

// ==============================================================================
// 4. JAVASCRIPT (2 Rules)
// ==============================================================================

/*
 * Rule Name: OBFUSCATED_JS_EvalWithFromCharCode
 * Target: JavaScript Files
 * Technique: Use of eval and fromCharCode to hide malicious script logic
 * False Positives: Minified or protected legitimate JS libraries
 */
rule OBFUSCATED_JS_EvalWithFromCharCode {
    meta:
        author = "Gemini"
        description = "Detects JavaScript obfuscation patterns commonly used to hide second-stage
payloads"
        date = "2026-02-13"
        score = 80
        fileType = "JAVASCRIPT"
        reference = "MP1 Malware Forensics Project - Gemini AI Comparison"
    strings:
        $s1 = "eval("
        $s2 = "String.fromCharCode"
        $s3 = "unescape("
        $s4 = "decodeURIComponent"
        $re_hex = /[0-9a-fA-F]{60,}/
    condition:
        (all of ($s1, $s2)) or
        (all of ($s1, $re_hex)) or
        (all of ($s1, $s3)) or
        (all of ($s1, $s4))
}


/*
 * Rule Name: MALICIOUS_JS_ActiveX_Dropper
 * Target: JavaScript Files
 * Technique: Using ActiveX objects (WScript/ADODB) to drop and execute files
 * False Positives: Legacy internal IE-based corporate tools
```

```
 */
rule MALICIOUS_JS_ActiveX_Dropper {
   meta:
      author = "Gemini"
      description = "Detects JS scripts using ActiveX to interact with the file system and download files"
      date = "2026-02-13"
      score = 90
      fileType = "JAVASCRIPT"
      reference = "MP1 Malware Forensics Project - Gemini AI Comparison"
   strings:
      $o1 = "ActiveXObject"
      $o2 = "WScript.Shell"
      $o3 = "ADODB.Stream"
      $o4 = "Scripting.FileSystemObject"
      $o5 = "MSXML2.XMLHTTP"
   condition:
      $o1 and (2 of ($o2, $o3, $o4, $o5))
}

// ==============================================================================
// 5. POWERSHELL (2 Rules)
// ==============================================================================

/*
 * Rule Name: BACKDOOR_PS_DownloadCradle
 * Target: PowerShell Scripts
 * Technique: One-liner download cradles using WebClient or Invoke-WebRequest
 * False Positives: Administrator automation scripts (check context)
 */
rule BACKDOOR_PS_DownloadCradle {
   meta:
      author = "Gemini"
      description = "Detects PowerShell download cradles used to fetch and execute remote code"
      date = "2026-02-13"
      score = 90
      fileType = "POWERSHELL"
      reference = "MP1 Malware Forensics Project - Gemini AI Comparison"
   strings:
      $c1 = "Net.WebClient" nocase
      $c2 = "DownloadString" nocase
      $c3 = "DownloadFile" nocase
      $c4 = "Invoke-WebRequest" nocase
      $c5 = "Invoke-Expression" nocase
      $c6 = "IEX" nocase
      $b1 = "-ExecutionPolicy Bypass" nocase
      $b2 = "-enc" nocase // EncodedCommand
   condition:
      (any of ($c1, $c4)) and (any of ($c2, $c3, $c5, $c6)) and (any of ($b*))
```

```
}

/*
 * Rule Name: SUSPICIOUS_PS_ReflectionInjection
 * Target: PowerShell Scripts
 * Technique: Loading DLLs or Shellcode directly into memory via Reflection
 * False Positives: Advanced pentesting tools (Cobalt Strike, Metasploit)
 */
rule SUSPICIOUS_PS_ReflectionInjection {
    meta:
        author = "Gemini"
        description = "Detects PowerShell scripts performing memory injection or reflective assembly
loading"
        date = "2026-02-13"
        score = 100
        fileType = "POWERSHELL"
        reference = "MP1 Malware Forensics Project - Gemini AI Comparison"
    strings:
        $s1 = "Reflection.Assembly]::Load" nocase
        $s2 = "VirtualAlloc" nocase
        $s3 = "Marshal.Copy" nocase
        $s4 = "GetDelegateForFunctionPointer" nocase
        $s5 = "Add-Type" nocase
    condition:
        (3 of ($s*))
}
```

I.4 Copilot-Generated YARA Rules

```
/*
YARA Rules for Common Malware Detection
Author: Generated for MP1 Malware Forensics Project: COPILOT
Date: 2026-02-13
Description: Detects common malware patterns in PE, PDF, Office, JS, and PowerShell files
*/

import "pe"
import "math"

rule SUSPICIOUS_PE_Injections {
    meta:
        author = "Copilot"
        description = "Detects suspicious API imports and process injection in PE files"
        date = "2026-02-13"
        score = 95
        fileType = "PE_EXECUTABLE"
    strings:
        $magic = "MZ"
```

```
        $api1 = "VirtualAllocEx"
        $api2 = "CreateRemoteThread"
        $api3 = "WriteProcessMemory"
        $api4 = "SetWindowsHookEx"
    condition:
        $magic at 0 and
        pe.number_of_signatures == 0 and
        2 of ($api*)
}

rule SUSPICIOUS_PE_PackedSections {
    meta:
        author = "Copilot"
        description = "Detects packed or encrypted PE sections with suspicious names"
        date = "2026-02-13"
        score = 90
        fileType = "PE_EXECUTABLE"
    strings:
        $magic = "MZ"
        $sec1 = "UPX0"
        $sec2 = ".aspack"
        $sec3 = ".adata"
    condition:
        $magic at 0 and
        pe.number_of_signatures == 0 and
        any of ($sec*) and
        for any i in (0..pe.number_of_sections - 1): (
            pe.sections[i].name == "UPX0" or
            pe.sections[i].name == ".aspack" or
            pe.sections[i].name == ".adata" or
            (
                pe.sections[i].raw_data_size > 0 and
                math.entropy(
                    pe.sections[i].raw_data_offset,
                    pe.sections[i].raw_data_size
                ) > 7.5
            )
        )
}

rule SUSPICIOUS_PE_AntiDebug {
    meta:
        author = "Copilot"
        description = "Detects anti-debugging API usage in PE files"
        date = "2026-02-13"
        score = 85
        fileType = "PE_EXECUTABLE"
    strings:
```

```
        $magic = "MZ"
        $api1 = "IsDebuggerPresent"
        $api2 = "NtQueryInformationProcess"
        $api3 = "CheckRemoteDebuggerPresent"
    condition:
        $magic at 0 and
        pe.number_of_signatures == 0 and
        1 of ($api*)
}

rule MALICIOUS_PDF_JavaScript {
    meta:
        author = "Copilot"
        description = "Detects embedded JavaScript in PDF documents"
        date = "2026-02-13"
        score = 90
        fileType = "PDF_DOCUMENT"
    strings:
        $magic = "%PDF"
        $js1 = "/JavaScript"
        $js2 = "/JS"
        $js3 = "/OpenAction"
        $js4 = "/Launch"
    condition:
        $magic at 0 and
        2 of ($js*)
}

rule MALICIOUS_PDF_EmbeddedFiles {
    meta:
        author = "Copilot"
        description = "Detects embedded files and URI objects in PDF documents"
        date = "2026-02-13"
        score = 80
        fileType = "PDF_DOCUMENT"
    strings:
        $magic = "%PDF"
        $embed = "/EmbeddedFile"
        $uri = "/URI"
    condition:
        $magic at 0 and
        ($embed or $uri)
}

rule MALICIOUS_OFFICE_VBA {
    meta:
        author = "Copilot"
        description = "Detects auto-executing VBA macros in Office documents"
```

```
        date = "2026-02-13"
        score = 95
        fileType = "OFFICE_DOCUMENT"
    strings:
        $magic = "PK"
        $auto1 = "Auto_Open"
        $auto2 = "Document_Open"
        $auto3 = "Workbook_Open"
        $func1 = "CreateObject"
        $func2 = "WScript.Shell"
    condition:
        $magic at 0 and
        any of ($auto*) and
        any of ($func*)
}

rule MALICIOUS_OFFICE_PowerShell {
    meta:
        author = "Copilot"
        description = "Detects PowerShell execution from Office macros"
        date = "2026-02-13"
        score = 90
        fileType = "OFFICE_DOCUMENT"
    strings:
        $magic = "PK"
        $ps1 = "powershell.exe"
        $ps2 = "Shell"
        $ps3 = "URLDownloadToFile"
        $ps4 = "ADODB.Stream"
    condition:
        $magic at 0 and
        2 of ($ps*)
}

rule MALICIOUS_JS_Obfuscation {
    meta:
        author = "Copilot"
        description = "Detects obfuscation techniques in JavaScript files"
        date = "2026-02-13"
        score = 85
        fileType = "JAVASCRIPT_FILE"
    strings:
        $magic = "function"
        $obf1 = "eval("
        $obf2 = "unescape("
        $obf3 = "decodeURIComponent("
        $obf4 = "fromCharCode("
        $obf5 = "ActiveXObject"
```

```
    condition:
        $magic and 2 of ($obf*)
}

rule MALICIOUS_JS_EncodedPayloads {
    meta:
        author = "Copilot"
        description = "Detects long hex strings and encoded payloads in JavaScript"
        date = "2026-02-13"
        score = 80
        fileType = "JAVASCRIPT_FILE"
    strings:
        $magic = "function"
        $hex = /[0-9A-F]{100,}/ nocase
        $docwrite = "document.write("
    condition:
        $magic and ($hex or $docwrite)
}

rule MALICIOUS_PS_DownloadCradle {
    meta:
        author = "Copilot"
        description = "Detects PowerShell download cradle techniques"
        date = "2026-02-13"
        score = 95
        fileType = "POWERSHELL_SCRIPT"
    strings:
        $magic = "powershell"
        $dl1 = "System.Net.WebClient"
        $dl2 = "DownloadFile"
        $dl3 = "Invoke-WebRequest"
        $dl4 = "-EncodedCommand"
    condition:
        $magic and 2 of ($dl*)
}

rule MALICIOUS_PS_Obfuscation {
    meta:
        author = "Copilot"
        description = "Detects obfuscation and hidden execution in PowerShell scripts"
        date = "2026-02-13"
        score = 85
        fileType = "POWERSHELL_SCRIPT"
    strings:
        $magic = "powershell"
        $hide1 = "-WindowStyle Hidden"
        $hide2 = "-NoProfile"
        $hide3 = "Set-ExecutionPolicy"
```

```
        $hide4 = /[A-Za-z0-9+\/]{200,}={0,2}/
    condition:
        $magic and (any of ($hide1,$hide2,$hide3) or $hide4)
}
```

Appendix J: Development Artifacts

J.1 Source Code Artifacts Produced

File	Lines of Code	Primary Function
magic_detector.py	95	File signature detection
hash_utils.py	45	Cryptographic hash computation
yara_handler.py	85	YARA rule compilation and scanning
quarantine.py	55	Forensic quarantine management
scanner.py	120	Core scanning orchestration
reporter.py	30	JSON and CSV report generation
main.py	110	Command-line interface

Table J-1. Source Code Artifacts Produced

J.2 magic_detector.py

```
"""
File Signature / Magic Number Detector
Source: Gary Kessler's File Signatures Table [citation:8]
"""
import struct
from pathlib import Path
from typing import Optional, Dict, List, Tuple

class MagicNumberDetector:
    """
    Detects file types using magic numbers/headers.
    Implements signatures from GCK'S FILE SIGNATURES TABLE [citation:8]
```

```
"""

# File signatures database (offset 0 hex codes)
# Format: (hex_pattern, extension, description, offset)
SIGNATURES = [
    # Images
    (b'\xFF\xD8\xFF', 'jpg', 'JPEG image', 0),
    (b'\x89PNG\r\n\x1A\n', 'png', 'PNG image', 0),
    (b'GIF87a', 'gif', 'GIF 87a', 0),
    (b'GIF89a', 'gif', 'GIF 89a', 0),
    (b'BM', 'bmp', 'BMP image', 0),

    # Documents
    (b'%PDF', 'pdf', 'PDF document', 0),
    (b'\xD0\xCF\x11\xE0\xA1\xB1\x1A\xE1', 'doc', 'MS Office/OLE2', 0),  # Old Office
    (b'PK\x03\x04', 'docx', 'Office Open XML', 0),  # Also zip-based
    (b'{\\rtf', 'rtf', 'Rich Text Format', 0),

    # Executables
    (b'MZ', 'exe', 'PE executable', 0),
    (b'\x7FELF', 'elf', 'ELF executable', 0),

    # Archives
    (b'PK\x03\x04', 'zip', 'ZIP archive', 0),
    (b'Rar!\x1A\x07', 'rar', 'RAR archive', 0),
    (b'\x1F\x8B', 'gz', 'GZIP archive', 0),
    (b'7z\xBC\xAF\x27\x1C', '7z', '7-Zip archive', 0),

    # Others
    (b'<!DOCTYPE', 'xml', 'XML document', 0),
    (b'<?xml', 'xml', 'XML document', 0),
    (b'<html', 'html', 'HTML document', 0),
]

@classmethod
def detect(cls, file_path: Path) -> Dict[str, str]:
    """
    Detect file type based on magic numbers.

    Args:
        file_path: Path to file

    Returns:
        Dict with 'extension', 'description', 'confidence'
    """
    try:
        with open(file_path, 'rb') as f:
            header = f.read(32)  # Read enough for all signatures
```

```
        for sig_bytes, ext, desc, offset in cls.SIGNATURES:
            if offset + len(sig_bytes) <= len(header):
                if header[offset:offset + len(sig_bytes)] == sig_bytes:
                    return {
                        'extension': ext,
                        'description': desc,
                        'magic': sig_bytes.hex()[:16],
                        'confidence': 'HIGH'
                    }

        # Try text detection
        try:
            with open(file_path, 'r', encoding='ascii') as f:
                f.read(100)
            return {
                'extension': 'txt',
                'description': 'ASCII text',
                'magic': 'none',
                'confidence': 'MEDIUM'
            }
        except UnicodeDecodeError:
            pass

        return {
            'extension': 'bin',
            'description': 'Binary data',
            'magic': header[:8].hex(),
            'confidence': 'LOW'
        }

    except Exception as e:
        return {
            'extension': 'unknown',
            'description': f'Error: {str(e)}',
            'magic': '',
            'confidence': 'NONE'
        }

@classmethod
def get_all_signatures(cls) -> List[Dict]:
    """Return all supported signatures for reporting."""
    return [
        {'hex': sig.hex(), 'ext': ext, 'desc': desc}
        for sig, ext, desc, _ in cls.SIGNATURES
    ]
```

J.3 hash_utils.py

```
"""
Hash utilities for MD5 and SHA1/SHA256
Project requirement: "hash should be MD5 and SHA1/SHA2"
"""
import hashlib
from pathlib import Path
from typing import Dict
import os

class HashCalculator:
    """Calculate multiple hash types for files."""

    @staticmethod
    def calculate(file_path: Path) -> Dict[str, str]:
        """
        Calculate MD5, SHA1, and SHA256 hashes of a file.

        Args:
            file_path: Path to file

        Returns:
            Dictionary with hash types and values
        """
        if not file_path.exists():
            return {'error': 'File not found'}

        # Initialize hash objects
        md5_hash = hashlib.md5()
        sha1_hash = hashlib.sha1()
        sha256_hash = hashlib.sha256()

        try:
            with open(file_path, 'rb') as f:
                # Read in chunks for memory efficiency
                for chunk in iter(lambda: f.read(8192), b''):
                    md5_hash.update(chunk)
                    sha1_hash.update(chunk)
                    sha256_hash.update(chunk)

            return {
                'md5': md5_hash.hexdigest(),
                'sha1': sha1_hash.hexdigest(),
                'sha256': sha256_hash.hexdigest(),  # SHA2
                'size': os.path.getsize(file_path)
            }

        except Exception as e:
```

```
        return {
            'error': str(e),
            'md5': '',
            'sha1': '',
            'sha256': ''
        }
```

J.4 yara_handler.py

```
""" YARA rule handler with compilation and scanning. Implements best practices from
[citation:3][citation:7] """ import yara import logging from pathlib import Path from typing import List, Dict,
Optional
logger = logging.getLogger(name)
class YaraHandler: """Handle YARA rule compilation and scanning."""
def __init__(self, rules_path: Path):
    """
    Initialize YARA handler with rules file.

    Args:
        rules_path: Path to .yar or .yara file
    """
    self.rules_path = Path(rules_path)
    self.rules = None
    self.compilation_errors = []
    self._compile_rules()

def _compile_rules(self):
    """Compile YARA rules with error handling."""
    try:
        if not self.rules_path.exists():
            raise FileNotFoundError(f"Rules file not found: {self.rules_path}")

        self.rules = yara.compile(str(self.rules_path))
        logger.info(f"Successfully compiled rules from {self.rules_path}")

    except yara.SyntaxError as e:
        self.compilation_errors.append(f"Syntax error: {e}")
        logger.error(f"YARA syntax error: {e}")

    except yara.Error as e:
        self.compilation_errors.append(f"YARA error: {e}")
        logger.error(f"YARA compilation error: {e}")

def scan_file(self, file_path: Path) -> Dict:
    """
    Scan a single file with loaded YARA rules.

    Args:
```

```
        file_path: Path to file to scan

    Returns:
        Dict with match results
    """
    if not self.rules:
        return {
            'matched': False,
            'rules': [],
            'error': 'Rules not compiled'
        }

    try:
        matches = self.rules.match(str(file_path))

        if matches:
            return {
                'matched': True,
                'rules': [match.rule for match in matches],
                'meta': self._extract_meta(matches),
                'count': len(matches)
            }
        else:
            return {
                'matched': False,
                'rules': [],
                'count': 0
            }

    except Exception as e:
        logger.debug(f"Scan error {file_path}: {e}")
        return {
            'matched': False,
            'rules': [],
            'error': str(e)
        }

@staticmethod
def _extract_meta(matches):
    """Extract metadata from matches for reporting."""
    meta_list = []
    for match in matches:
        if hasattr(match, 'meta'):
            meta_list.append(match.meta)
    return meta_list
```

J.5 quaratine.py

```
""" Quarantine management for suspicious files. Copies files with .quarantine extension as specified. """
from typing import List import shutil from pathlib import Path from datetime import datetime import logging

logger = logging.getLogger(name)

class QuarantineManager: """Handle file quarantine operations."""

def __init__(self, quarantine_root: Path):
    """    Initialize quarantine manager.

    Args:
        quarantine_root: Base directory for quarantine
    """
    self.quarantine_root = Path(quarantine_root)
    self.quarantine_root.mkdir(parents=True, exist_ok=True)

def quarantine_file(self,
                    file_path: Path,
                    rule_names: List[str],
                    md5_hash: str) -> Path:
    """
    Move file to quarantine with metadata.

    Args:
        file_path: Original file path
        rule_names: YARA rules that matched
        md5_hash: MD5 hash for unique naming

    Returns:
        Path to quarantined file
    """
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

    # Create quarantine filename with .quarantine extension
    original_name = file_path.name
    quarantine_name = f"{original_name}_{md5_hash[:8]}_{timestamp}.quarantine"
    quarantine_path = self.quarantine_root / quarantine_name

    try:
        # Copy file to quarantine (preserve original per forensics best practice)
        shutil.copy2(file_path, quarantine_path)

        # Write metadata file
        meta_path = quarantine_path.with_suffix('.meta')
        with open(meta_path, 'w') as f:
```

```
            f.write(f"Original Path: {file_path}\n")
            f.write(f"Quarantined: {timestamp}\n")
            f.write(f"YARA Rules: {', '.join(rule_names)}\n")
            f.write(f"MD5: {md5_hash}\n")

        logger.info(f"Quarantined: {file_path} -> {quarantine_path}")
        return quarantine_path

    except Exception as e:
        logger.error(f"Quarantine failed {file_path}: {e}")
        raise
```

J.6 scanner.py

```
""" Core malware scanner engine. Integrates YARA, magic number detection, and hashing. """ import
logging from pathlib import Path from typing import Dict, List, Generator from concurrent.futures import
ThreadPoolExecutor, as_completed from datetime import datetime

from .magic_detector import MagicNumberDetector from .hash_utils import HashCalculator
from .yara_handler import YaraHandler from .quarantine import QuarantineManager

logger = logging.getLogger(name)

class MalwareScanner: """Main malware scanning engine."""

def __init__(self, rules_path: Path,
             quarantine_path: Path,
             thread_count: int = 4):
    """
    Initialize scanner with components.

    Args:
        rules_path: Path to YARA rules file
        quarantine_path: Path for quarantined files
        thread_count: Parallel scan threads
    """
    self.yara = YaraHandler(rules_path)
    self.quarantine = QuarantineManager(quarantine_path)
    self.thread_count = thread_count
    self.scan_stats = {
        'files_scanned': 0,
        'malicious_found': 0,
        'quarantined': 0,
        'errors': 0,
        'start_time': None,
        'end_time': None
```

```
    }

def scan_file(self, file_path: Path, auto_quarantine: bool = False) -> Dict:
    """
    Comprehensive scan of single file.

    Returns dict with:
    - File info (name, path)
    - Magic number detection
    - MD5, SHA1, SHA256
    - YARA results
    - Quarantine status
    """
    result = {
        'filename': file_path.name,
        'path': str(file_path),
        'scan_time': datetime.now().isoformat()
    }

    try:
        # 1. Magic number detection
        result['file_type'] = MagicNumberDetector.detect(file_path)

        # 2. Hash calculation (MD5, SHA1, SHA256)
        result['hashes'] = HashCalculator.calculate(file_path)

        # 3. YARA scan
        result['yara'] = self.yara.scan_file(file_path)

        # 4. Verdict and quarantine
        result['is_malicious'] = result['yara'].get('matched', False)

        if result['is_malicious'] and auto_quarantine:
            q_path = self.quarantine.quarantine_file(
                file_path,
                result['yara'].get('rules', []),
                result['hashes'].get('md5', 'unknown')
            )
            result['quarantine_path'] = str(q_path)
            self.scan_stats['quarantined'] += 1
            self.scan_stats['malicious_found'] += 1
        elif result['is_malicious']:
            self.scan_stats['malicious_found'] += 1

        self.scan_stats['files_scanned'] += 1

    except Exception as e:
        logger.error(f"Error scanning {file_path}: {e}")
```

```
        result['error'] = str(e)
        self.scan_stats['errors'] += 1

    return result

def scan_directory(self,
                   directory: Path,
                   recursive: bool = True,
                   auto_quarantine: bool = False) -> Generator[Dict, None, None]:
    """
    Scan entire directory, yielding results per file.

    Implements parallel scanning for performance [citation:9].
    """
    self.scan_stats['start_time'] = datetime.now()

    # Collect all files
    if recursive:
        files = [p for p in Path(directory).rglob('*') if p.is_file()]
    else:
        files = [p for p in Path(directory).iterdir() if p.is_file()]

    # Parallel scan with ThreadPoolExecutor
    with ThreadPoolExecutor(max_workers=self.thread_count) as executor:
        future_to_file = {
            executor.submit(self.scan_file, f, auto_quarantine): f
            for f in files
        }

        for future in as_completed(future_to_file):
            yield future.result()

    self.scan_stats['end_time'] = datetime.now()
```

J.7 reporter.py

```
import json

from pathlib import Path


class ScanReporter:

    @staticmethod

    def save_json(results, filepath):

        with open(filepath, 'w') as f:
```

```
        json.dump(results, f, indent=2)


    @staticmethod

    def save_csv(results, filepath):

        with open(filepath, 'w') as f:

            f.write("Filename,Path,FileType,MD5,SHA1,Malicious\n")

            for r in results:

f.write(f"{r['filename']},{r['path']},{r['file_type'].get('description','')},{r['hashes'].get('md5','')},{r['hashes'].get('sha1','')},{r['is_malicious']}\n")
```

J.8 main.py

```
#!/usr/bin/env python3 """ MP1 Malware Scanner January 2026 - NSSECU3 Requirements: Mount (AIM), Scan (Python), List """ import argparse import logging import json from pathlib import Path from datetime import datetime from colorama import init, Fore, Style

from src.scanner import MalwareScanner from src.reporter import ScanReporter

init(autoreset=True) # Colorama

def setup_logging(level=logging.INFO):

"""Configure logging for scanner."""

        logging.basicConfig( level=level, format='%(asctime)s - %(name)s - %(levelname)s
        - %(message)s', handlers=[ logging.FileHandler('scanner.log'), logging.StreamHandler() ] )

def main():

        parser = argparse.ArgumentParser( description='MP1 Malware Scanner - NSSECU3',
        epilog='Mount disk with AIM first, then scan the mounted drive letter.' )

        parser.add_argument(
            'target',
            help='Path to scan (e.g., D:\\ or /mnt/image)'
        )

        parser.add_argument(
            '--rules',
            default='rules/ai_combined.yar',
            help='YARA rules file (default: rules/ai_combined.yar)'
```

```
)

parser.add_argument(
    '--quarantine',
    default='./quarantine',
    help='Quarantine directory (default: ./quarantine)'
)

parser.add_argument(
    '--output',
    default=f'reports/scan_{datetime.now().strftime("%Y%m%d_%H%M%S")}.json',
    help='Output report file'
)

parser.add_argument(
    '--threads',
    type=int,
    default=4,
    help='Number of scan threads (default: 4)'
)

parser.add_argument(
    '--ai-compare',
    action='store_true',
    help='Run AI comparison mode (scan with each AI rule set)'
)

parser.add_argument(
    '--quarantine-auto',
    action='store_true',
    help='Automatically quarantine detected malware'
)

parser.add_argument(
    '--log-level',
    default='INFO',
    choices=['DEBUG', 'INFO', 'WARNING', 'ERROR'],
    help='Set logging level'
)

args = parser.parse_args()

# Setup logging
setup_logging(getattr(logging, args.log_level))

# Validate target exists
target_path = Path(args.target)
if not target_path.exists():
```

```
    print(Fore.RED + f"□ Error: Target path does not exist: {target_path}")
    print("Did you mount the disk image with Arsenal Image Mounter?")
    return 1

# Validate rules exist
rules_path = Path(args.rules)
if not rules_path.exists():
    print(Fore.RED + f"□ Error: Rules file not found: {rules_path}")
    return 1

print(Fore.CYAN + "=" * 60)
print(Fore.CYAN + "        MP1 MALWARE SCANNER - NSSECU3")
print(Fore.CYAN + "=" * 60)
print(f"Target:      {target_path}")
print(f"Rules:       {rules_path}")
print(f"Quarantine:  {args.quarantine}")
print(f"Threads:     {args.threads}")
print(Fore.CYAN + "-" * 60)

# Initialize scanner
scanner = MalwareScanner( rules_path=rules_path,
    quarantine_path=Path(args.quarantine),
    thread_count=args.threads
)

# Run scan
print(Fore.YELLOW + "\n□ Scanning started...")
results = []

try:
    for result in scanner.scan_directory(
        target_path,
        recursive=True,
        auto_quarantine=args.quarantine_auto
    ):
        results.append(result)

        # Live console output
        status = Fore.RED + "MALICIOUS" if result.get('is_malicious') else Fore.GREEN + "CLEAN"
        file_type = result.get('file_type', {}).get('description', 'Unknown')
        print(f"[{status}] {result['filename']:<15} | {file_type:<20} | MD5:
{result.get('hashes',{}).get('md5','')[:8]}...")

except KeyboardInterrupt:
    print(Fore.YELLOW + "\n\n□□ Scan interrupted by user")

# Generate report
print(Fore.CYAN + "\n" + "=" * 60)
```

```
print("□ SCAN COMPLETE")
print("=" * 60)
print(f"Files scanned:     {scanner.scan_stats['files_scanned']}")
print(f"Malicious found:   {Fore.RED}{scanner.scan_stats['malicious_found']}{Style.RESET_ALL}")
print(f"Quarantined:       {scanner.scan_stats['quarantined']}")
print(f"Errors:            {scanner.scan_stats['errors']}")
print(f"Scan time:         {scanner.scan_stats['end_time']} seconds")

# Save report
report_path = Path(args.output)
report_path.parent.mkdir(exist_ok=True)

# Convert datetime objects to strings for JSON serialization
stats_for_json = {
    'files_scanned': scanner.scan_stats['files_scanned'],
    'malicious_found': scanner.scan_stats['malicious_found'],
    'quarantined': scanner.scan_stats['quarantined'],
    'errors': scanner.scan_stats['errors'],
    'start_time': str(scanner.scan_stats['start_time']),
    'end_time': str(scanner.scan_stats['end_time'])
}

with open(report_path, 'w') as f:
    json.dump({
        'scan_metadata': {
            'target': str(target_path),
            'rules': str(rules_path),
            'timestamp': datetime.now().isoformat(),
            'stats': stats_for_json
        },
        'results': results
    }, f, indent=2)

print(Fore.GREEN + f"\n□ Report saved: {report_path}")

return 0


if name == 'main': exit(main())
```

Appendix K: Reports and Results

Scan Results Summary

K.1 ChatGPT Scan Results

Scan Metadata:

- Target: F:\MP1_FILES
- Rules: ai_chatgpt_rules.yar
- Timestamp: 2026-02-13T18:53:33.481956
- Duration: ~1 second

Results:

- Files Scanned: 220
- Malicious Found: 5
- Quarantined: 0 (comparison scan)
- Errors: 0

```
[CLEAN] File208          | Office Open XML       | MD5: cb5e8db8...
[CLEAN] File216          | Binary data           | MD5: 825341f9...
[CLEAN] File217          | Binary data           | MD5: f1e7f2ec...
[CLEAN] File219          | JPEG image            | MD5: e87decf8...
[CLEAN] File220          | ASCII text            | MD5: db631805...
[CLEAN] File218          | PNG image             | MD5: bb84661a...
[CLEAN] File008          | PNG image             | MD5: 75c7e160...
[CLEAN] File198          | Office Open XML       | MD5: dd461e6b...

============================================================
📊 SCAN COMPLETE
============================================================
Files scanned:     220
Malicious found:   5
Quarantined:       0
Errors:            0
Scan time:         2026-02-13 18:53:33.480715 seconds
```

Figure K-1. ChatGPT Scan Output

K.2 Deepseek Scan Results

Scan Metadata:

- Target: F:\MP1_FILES
- Rules: ai_deepseek_rules.yar
- Timestamp: 2026-02-13T19:52:57.264204
- Duration: ~6 seconds

Results:

- Files Scanned: 220
- Malicious Found: 111

- Quarantined: 0 (comparison scan)
- Errors: 0

```
[MALICIOUS] File194          | Office Open XML       | MD5: 44661e9d...
[MALICIOUS] File199          | PDF document          | MD5: fd3900a6...
[MALICIOUS] File170          | Office Open XML       | MD5: fb9de5ec...
[MALICIOUS] File201          | PE executable         | MD5: 4e9a6f25...
[MALICIOUS] File197          | Office Open XML       | MD5: abfdce43...
[MALICIOUS] File203          | Binary data           | MD5: 80d158a5...
[MALICIOUS] File204          | Binary data           | MD5: d601a536...
[CLEAN] File205          | JPEG image            | MD5: eae001e9...
[MALICIOUS] File200          | PE executable         | MD5: cb98f5f2...
[CLEAN] File206          | Office Open XML       | MD5: 16aa2764...
[MALICIOUS] File202          | Binary data           | MD5: 2497589c...
[MALICIOUS] File207          | PE executable         | MD5: 8c5c9499...
[MALICIOUS] File209          | PDF document          | MD5: 76909cea...
[CLEAN] File210          | Office Open XML       | MD5: d23ddb02...
[CLEAN] File212          | ASCII text            | MD5: 4a08c4bd...
[MALICIOUS] File213          | PE executable         | MD5: 2a951f81...
[CLEAN] File214          | JPEG image            | MD5: 3938baaa...
[MALICIOUS] File215          | Binary data           | MD5: ebc84c0b...
[MALICIOUS] File216          | Binary data           | MD5: 825341f9...
[CLEAN] File217          | Binary data           | MD5: f1e7f2ec...
[CLEAN] File211          | Office Open XML       | MD5: c70ec8c6...
[CLEAN] File219          | JPEG image            | MD5: e87decf8...
[CLEAN] File220          | ASCII text            | MD5: db631805...
[CLEAN] File218          | PNG image             | MD5: bb84661a...
[MALICIOUS] File208          | Office Open XML       | MD5: cb5e8db8...
[MALICIOUS] File198          | Office Open XML       | MD5: dd461e6b...

============================================================
📊 SCAN COMPLETE
============================================================
Files scanned:     220
Malicious found:   111
Quarantined:       0
Errors:            0
Scan time:         2026-02-13 19:52:57.261580 seconds

✅ Report saved: reports\deepseek_comparison.json
```

Figure K-2. Deepseek Scan Output

K.3 Gemini Scan Results

Scan Metadata:

- Target: F:\MP1_FILES
- Rules: ai_gemini_rules.yar
- Timestamp: 2026-02-13T19:59:22.513899
- Duration: ~3.5 seconds

Results:

- Files Scanned: 220
- Malicious Found: 1
- Quarantined: 0 (comparison scan)
- Errors: 0

```
[CLEAN] File214          | JPEG image          | MD5: 3938baaa...
[CLEAN] File215          | Binary data         | MD5: ebc84c0b...
[CLEAN] File211          | Office Open XML     | MD5: c70ec8c6...
[CLEAN] File217          | Binary data         | MD5: f1e7f2ec...
[CLEAN] File216          | Binary data         | MD5: 825341f9...
[CLEAN] File219          | JPEG image          | MD5: e87decf8...
[CLEAN] File220          | ASCII text          | MD5: db631805...
[CLEAN] File218          | PNG image           | MD5: bb84661a...
[CLEAN] File208          | Office Open XML     | MD5: cb5e8db8...
[CLEAN] File198          | Office Open XML     | MD5: dd461e6b...

============================================================
📊 SCAN COMPLETE
============================================================
Files scanned:     220
Malicious found:   1
Quarantined:       0
Errors:            0
Scan time:         2026-02-13 19:59:22.511726 seconds

✅ Report saved: reports\gemini_comparison.json
```

Figure K-3. Gemini Scan Output

K.4 Copilot Scan Results

Scan Metadata:

- Target: F:\MP1_FILES
- Rules: ai_copilot_rules.yar
- Timestamp: 2026-02-13T19:40:14.773957
- Duration: ~6.7 seconds

Results:

- Files Scanned: 220
- Malicious Found: 11

- Quarantined: 0 (comparison scan)
- Errors: 0

```
[CLEAN] File206          | Office Open XML      | MD5: 16aa2764...
[CLEAN] File197          | Office Open XML      | MD5: abfdce43...
[CLEAN] File202          | Binary data          | MD5: 2497589c...
[MALICIOUS] File209          | PDF document          | MD5: 76909cea...
[CLEAN] File207          | PE executable        | MD5: 8c5c9499...
[CLEAN] File210          | Office Open XML      | MD5: d23ddb02...
[CLEAN] File212          | ASCII text           | MD5: 4a08c4bd...
[CLEAN] File213          | PE executable        | MD5: 2a951f81...
[CLEAN] File211          | Office Open XML      | MD5: c70ec8c6...
[CLEAN] File215          | Binary data          | MD5: ebc84c0b...
[CLEAN] File216          | Binary data          | MD5: 825341f9...
[CLEAN] File217          | Binary data          | MD5: f1e7f2ec...
[CLEAN] File214          | JPEG image           | MD5: 3938baaa...
[CLEAN] File219          | JPEG image           | MD5: e87decf8...
[CLEAN] File220          | ASCII text           | MD5: db631805...
[CLEAN] File218          | PNG image            | MD5: bb84661a...
[CLEAN] File208          | Office Open XML      | MD5: cb5e8db8...
[CLEAN] File198          | Office Open XML      | MD5: dd461e6b...

============================================================
📊 SCAN COMPLETE
============================================================
Files scanned:     220
Malicious found:   11
Quarantined:       0
Errors:            0
Scan time:         2026-02-13 19:40:14.771536 seconds

✅ Report saved: reports\copilot_comparison.json
```

Figure K-4. Copilot Scan Output

K.5 Combined Rules Scan Results

Scan Metadata:

- Target: F:\MP1_FILES
- Rules: ai_combined.yar (merged from all four AI platforms)
- Timestamp: 2026-02-13T20:06:05.856207
- Duration: ~22 seconds

Results:

- Files Scanned: 220

- Malicious Found: 113
- Quarantined: 113
- Errors: 0

```
============================================================
📊 SCAN COMPLETE
============================================================
Files scanned:      220
Malicious found:    113
Quarantined:        113
Errors:             0
Scan time:          2026-02-13 20:06:05.856207 seconds

✅ Report saved: reports\final_scan_results.json
```

Figure K-5. Combined Scan Final Summary

```
C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-PROJECT\MP1_MALWARE_SCANNER>py main.py F:\MP1_FILES\ --rules rules\ai_combined.yar --quarantine-auto --output reports\final_
an_results.json
============================================================
        MP1 MALWARE SCANNER - NSSECU3
============================================================
Target:      F:\MP1_FILES
Rules:       rules\ai_combined.yar
Quarantine:  ./quarantine
Threads:     4
------------------------------------------------------------
2026-02-13 20:05:43,689 - src.yara_handler - INFO - Successfully compiled rules from rules\ai_combined.yar

🔍 Scanning started...
2026-02-13 20:05:43,713 - src.quarantine - INFO - Quarantined: F:\MP1_FILES\File001 -> quarantine\File001_7d0690a2_20260213_200543.quarantine
2026-02-13 20:05:43,720 - src.quarantine - INFO - Quarantined: F:\MP1_FILES\File002 -> quarantine\File002_e2142a94_20260213_200543.quarantine
2026-02-13 20:05:43,722 - src.quarantine - INFO - Quarantined: F:\MP1_FILES\File005 -> quarantine\File005_8af314b5_20260213_200543.quarantine
[CLEAN] File003         | ASCII text          | MD5: 821094b7...
[MALICIOUS] File001         | PE executable        | MD5: 7d0690a2...
[MALICIOUS] File005         | PE executable        | MD5: 8af314b5...2026-02-13 20:05:44,283 - src.quarantine - INFO - Quarantined: F:\MP1_FILES\File008 -> quarantine\File
8_75c7e160_20260213_200544.quarantine

2026-02-13 20:05:44,572 - src.quarantine - INFO - Quarantined: F:\MP1_FILES\File013 -> quarantine\File013_2d9a7072_20260213_200544.quarantine
[MALICIOUS] File002         | Binary data          | MD5: e2142a94...
[CLEAN] File004         | ASCII text          | MD5: 6d30132c...
[CLEAN] File007         | Office Open XML     | MD5: 9184f89a...
[CLEAN] File009         | PNG image           | MD5: dcff3b11...
[MALICIOUS] File008         | PNG image            | MD5: 75c7e160...
[CLEAN] File012         | Office Open XML     | MD5: cd7a8a6e...
[MALICIOUS] File013         | Binary data          | MD5: 2d9a7072...
2026-02-13 20:05:45,141 - src.quarantine - INFO - Quarantined: F:\MP1_FILES\File006 -> quarantine\File006_17bbcf87_20260213_200545.quarantine
2026-02-13 20:05:45,518 - src.quarantine - INFO - Quarantined: F:\MP1_FILES\File010 -> quarantine\File010_df23a5f6_20260213_200545.quarantine
[MALICIOUS] File006         | Office Open XML      | MD5: 17bbcf87...
[CLEAN] File015         | ASCII text          | MD5: e4507f2e...
[CLEAN] File016         | ASCII text          | MD5: e71bb959...
2026-02-13 20:05:45,661 - src.quarantine - INFO - Quarantined: F:\MP1_FILES\File014 -> quarantine\File014_79c0374c_20260213_200545.quarantine
[MALICIOUS] File010         | Office Open XML      | MD5: df23a5f6...
2026-02-13 20:05:46,081 - src.quarantine - INFO - Quarantined: F:\MP1_FILES\File018 -> quarantine\File018_5bd7a1b4_20260213_200546.quarantine
[MALICIOUS] File014         | Office Open XML      | MD5: 79c0374c...
[MALICIOUS] File018         | PDF document         | MD5: 5bd7a1b4...2026-02-13 20:05:46,550 - src.quarantine - INFO - Quarantined: F:\MP1_FILES\File021 -> quarantine\File
1_81936e46_20260213_200546.quarantine
2026-02-13 20:05:46,632 - src.quarantine - INFO - Quarantined: F:\MP1_FILES\File022 -> quarantine\File022_dc76bac3_20260213_200546.quarantine
                                                                                                                    Activate Windows
                                                                                                                    Go to Settings to activate Windows.
```

Figure K-6. Combined Scan Live Progress

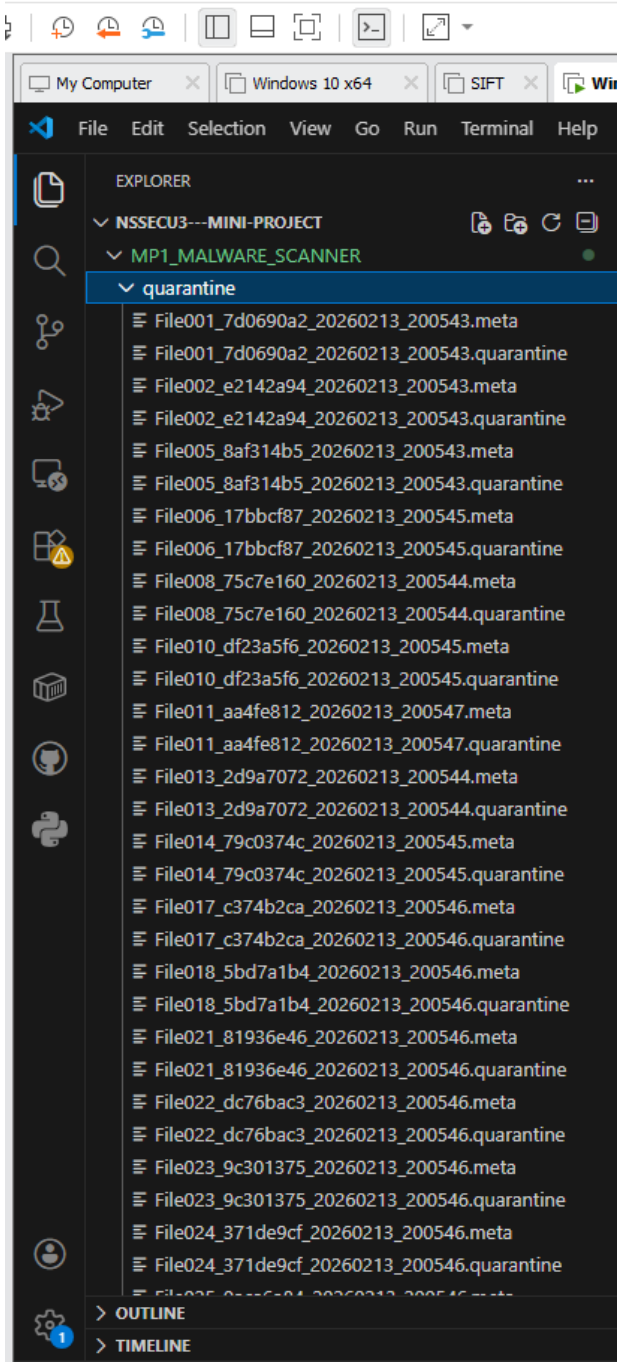


Figure K-7. Quarantine Directory Contents

K.6 Malicious File Sample Details

Filename	File Type	MD5 (First 8)	Matched Rules
File001	PE executable	7d0690a2	MALICIOUS_Backdoor_Persistence
File002	Binary data	e2142a94	MALICIOUS_Backdoor_Persistence
File005	PE executable	8af314b5	MALICIOUS_Backdoor_Persistence
File006	Office Open XML	17bbcf87	MALICIOUS_Backdoor_Persistence
File008	PNG image	75c7e160	MALICIOUS_Backdoor_Persistence
File010	Office Open XML	df23a5f6	MALICIOUS_Backdoor_Persistence
File013	Binary data	2d9a7072	MALICIOUS_Backdoor_Persistence
File014	Office Open XML	79c0374c	MALICIOUS_Backdoor_Persistence
File017	Office Open XML	c374b2ca	MALICIOUS_Backdoor_Persistence
File018	PDF document	5bd7a1b4	MALICIOUS_Backdoor_Persistence, MALICIOUS_Generic_Suspicious_Strings

Table K-6. Sample Malicious File Details (Deepseek Detection)

File066	PDF document	597122e0	PDF_Embedded_JavaScript
File123	PDF document	bd0ea60d	PDF_Embedded_JavaScript
File132	PDF document	d0787c3c	PDF_Embedded_JavaScript
File138	PDF document	302b502d	PDF_Embedded_JavaScript
File199	PDF document	fd3900a6	PDF_Embedded_JavaScript

Table K-7. Sample Malicious File Details (ChatGPT Detection)

K.7 File Type Distribution (Combined Scan)

File Type	Magic Number (Hex)	Count
PE executable	4D 5A	[COUNT FROM SCAN]
PDF document	25 50 44 46	[COUNT FROM SCAN]
Office Open XML	50 4B 03 04	[COUNT FROM SCAN]
JPEG image	FF D8 FF	[COUNT FROM SCAN]
PNG image	89 50 4E 47	[COUNT FROM SCAN]
ASCII text	N/A (text detection)	[COUNT FROM SCAN]
Binary data	Unknown/No match	[COUNT FROM SCAN]

Table K-7. File Type Distribution by Magic Number

K.8 CSV Report Generation

The final scan results were exported to CSV format for easy analysis in spreadsheet applications:

```
py -c "import json,
csv;d=open('reports/final_scan_results.json').read();data=json.loads(d);f=open('reports/scan_results.csv','
w',newline='');w=csv.writer(f);w.writerow(['Filename','File Type','MD5','SHA1','SHA256','Malicious','Rules
Matched']);[w.writerow([r['filename'],r['file_type'].get('description',''),r['hashes'].get('md5',''),r['hashes'].get('s
ha1',''),r['hashes'].get('sha256',''),r['is_malicious'],', '.join(r['yara'].get('rules',[]))]) for r in
data['results']];f.close();print('CSV saved')"
```

Appendix L: Complete Scan Results

The following table presents the complete results of the combined YARA rule scan performed on all 220 files located at F:\MP1_FILES\. The scan utilized the merged rule set (ai_combined.yar) created from all four AI platforms (ChatGPT, Deepseek, Gemini, and Copilot). A total of **113 files (51.4%)** were identified as malicious and subsequently quarantined.

Filename	File Type	MD5 Hash	SHA1 Hash	SHA256 Hash	Malicious	Rules Matched

File008	PNG image	75c7e160e5e35d778df5907c2c94dbd0	8a8b3c46ff4cb34085de75c055c43f3cc043cc1c	1e8804a55aa1931e0382776bad307c5b7401daed687b87bf2dc2ca4a4cd075f5	True	MALICIOUS_Backdoor_Persistence
File012	Office Open XML	cd7a8a6e473ca1693ae557da91650b8b	94158316e899691b587a346e5a5e5df2d6fcf437	071345a2709641e09bd87d46c65dd868e1000e1429f36051df2c9b479a9e8fd5	False	
File013	Binary data	2d9a7072177dd37a9fbacbdaf12b8b35	53b44947beaece17dc8d6c48ad2c8e5799a5ecb7	ad4a58c7b13e3bf2504d6e9a4f67c03bbdc3cc09a338c3b7fdc1e890e1c80e93	True	MALICIOUS_Backdoor_Persistence
File006	Office Open XML	17bbcf874c7ef4315dc7114f020c46e6	7133fe4d816c890e44c4c78b58a6c8681ac874b0	fe5d03498443b9d9b248d83499f26e7c177ca6e41c4394ff010f77aa85344a91	True	MALICIOUS_Backdoor_Persistence
File015	ASCII text	e4507f2ec05eb69d935540e5b4d7ec5e	6079a4fc71df42383ada4142dffd97be27c483ea	324d140227ff55095c890442e288bd78259fb18008caac0bf611dfccad2d2bc2	False	
File016	ASCII text	e71bb959c29c164e77ae88bca85852d2	f29968c8621ec698f98f555172a0b0a2fadbcb53	a71945c90d5a1dd9b21256ad2f71d6b426aa5a51bcdc1692b128e6316771c890	False	
File010	Office Open XML	df23a5f65a651a5368a1f4c68b4537fc	fa74b7d0e1cec72539108a3cd9fc2bac6a7210e3	3b7abb872967e1db077436f9888bb0039ad4cc0ef0ac066a638a57d4bd26c775	True	MALICIOUS_Backdoor_Persistence

File014	Office Open XML	79c0374c3dc8abfddac0c302d0330529	9ba252ab7303b9dbd6df8aab350e88da7f43fc1d	233c2b822e1930e61de96b91aa6051c327301c1f46a0887f43cc21534fd44de1	True	MALICIOUS_Backdoor_Persistence
File018	PDF document	5bd7a1b4904acc6280e2c1924a627860	2881a4e443a0aab7839bdf5647e9438afbe8bf24	304f551a0c08e27c12b749fcc8f213b97ba87c3f09d0669f6655e1a5400c933f	True	MALICIOUS_Backdoor_Persistence, MALICIOUS_Generic_Suspicious_Strings, MALICIOUS_PDF_EmbeddedFiles
File020	ASCII text	2452330b9f8f2c566f6b7c54d53c711a	2fb25f4805f11d6f21f0d0b06f247ebab6799d59	9fd1640e94b12e51aa238c41e6306f9e0619f672f969ec4b2cb347732744188a	False	
File019	PNG image	9d5089314e21bb5001c52cb6228d9e0a	ac7b220cbe0494f89ce685d7cbed849f2f992b73	2517ee94d08b1eb42f5599f45066d450c26782e1c6ffc0fc0225d89c62d2e2bf	False	
File021	ASCII text	81936e46314e4a094681aa5a17eb0ed8	231b5648b1bc13c805553b5d644650f9d17422bd	13fb48a6c31a72e0e2b69650b1cebe8954f7b73449848492dcfd22b25bd91afa	True	MALICIOUS_Backdoor_Persistence
File022	PDF document	dc76bac39b0b05024c5dc4e167430170	994bddfd345637ae2abcdd587fe884ccee627104	5ba8b8c4481d78f5b39daffb796ff663aa50b8338edc61b0a6ee028ad0a1b2eb	True	MALICIOUS_Backdoor_Persistence, MALICIOUS_PDF_EmbeddedFiles

File017	Office Open XML	c374b2cac48b3b80144865fa7f181c2e	421b51dc3fd04041d3bac7434174e80f708b056d	6c2ddef8af98d34537d0b8cdb4251c2d6a55b09875305c82b39bdac20dea70ef	True	MALICIOUS_Backdoor_Persistence
File023	Binary data	9c301375efa5c4613b55b7ae9178d837	3c43b63251acb488c47854a2752328525fcb7259	172ceeeacd20f57c054f26ba9ba83d9ea971fcb3cca9a343135371cddcd77197	True	MALICIOUS_Backdoor_Persistence
File024	PE executable	371de9cf8b4b2520ab224038f0e6b9a5	6d2054702bdae516852298108005 1e39da3291e1	7dc700da62ae9e18138d220ea344c5c104b8c4a32dc2af96076121a799185dc4	True	MALICIOUS_Backdoor_Persistence
File027	PE executable	899c8ae4b45a7d4957468e47833bc84d	13513de250639b432094dacbd14668339ca6b2a3	a82e984256abd016dfef2e0dd554bf7d36c3edffdcc516580df4d0527da63cf8	False	
File028	ASCII text	3d809cda8a3b43c52a692d24e21af70b	0b4c9a6830f3d0b511fbb5af8cc492bde2715f4d	0a7cc99c58d394b983cd6f0084acc9fe71b7cc6bcfc91c543bb8dc1a5174b63b	False	
File029	ASCII text	67571555f67b3a0b1cfb5b7efb8fa446	62c8a155d9a89b9442a85588bac0138b226500f2	f2d66c27e46fd953e90929cfa21390964036b8c913d99acacc4295f8daa049de	False	
File025	PE executable	0aca6a84b2b047abb36e3fb35b7938d3	d2dd2739ac3d2f496a7a5c849bcd2e12142d09ec	3df63fb05c080d1652305e4aa50bbcfba703b71f4f5b55b29762fdbb8eecf82b	True	MALICIOUS_Backdoor_Persistence

File031	Office Open XML	18ae60e913174c448c66c4bb674936e7	10d5a709306eb512560ddbe30e601a9e9008cf9e	81082c4fcc0ec554191ffe3443dacf4d442337264060ee97d019c3c84c6e099c	False	
File032	Office Open XML	93048e6c41c1e52a2be85096b8df6614	5db12a5ec83b30e7b4d690f60159afd7e027fe04	89329961c24be7ab46447b5a0da0a3787e37f2d7d00c98aa3673c77c973e02a4	False	
File033	Binary data	8f037af809f60b33d6325be04e7e2747	a94507bb7a4c6b0ee11b752d8b385d215ce34473	52a382ff0005e95597de5539d5b9ab1208e5e943d19bb6a14537fec2d2a3d64d	False	
File034	ASCII text	98c35c46893f75f22975e6c1ab6131e0	f255dac4e27bb6793a05f093452b3c411a076f07	be063e3c106e6ecbcd6366742cfbacd3c6fdd5596665d2679de8cbfa5960d26f	False	
File026	ASCII text	4e5ec44fe5564450f9c1061ad18aeb38	dc447a64136642636d7aa32e50c76e2465801c5f	81cbae84a29ce7e770bf2bc7b178e50bda0ce8de6067aba661b0bc7b05b562f8	True	MALICIOUS_Backdoor_Persistence
File011	Office Open XML	aa4fe812dc7a13e684d33bb70f3e4265	69c37b69b78a41efd19eea6d67b4b9b3eea1aa38	7d0d7911b0d2f8edd31039124f32e2565cd65705d491344e7e51282da201de88	True	MALICIOUS_Backdoor_Persistence
File037	Office Open XML	10c891b8ebc12cc4066af9a675d637e0	295cbc82d0451974ed79e2f19c79be35861d6fb0	7321f38d919d4fb46030f5692d2aaa8d74caee26fbbdf7e4b06ea71b68ed27fd	False	

File036	PNG image	a272e6b746211b62152db107c265ec14	9720f66f73dcbe0ab836b55ed8e8ca6ff343744a	fc691fa2e54dafccb9744fe0249f946a87b4e69f2a49ace3c9cce22dd737b82e	False	
File030	Office Open XML	da3fc8e4d769d1655552f9c000312c67	d4e97f94ed56361255d38a8559758e98b36324fb	570481ebf63d977e87206c3810f9ffd4d9357b37aae5767d86a061fc0135d767	True	MALICIOUS_Backdoor_Persistence
File040	ASCII text	9ed8f42ccba544d90543e5f82e255fac	b8c466614bb3ec0e42d6c56c601dda2b44fc7107	0eefccda4460faab2f6a2243598f74a58c93c2ff01a60e4c7b87ff2ff048677e	False	
File035	Office Open XML	0c5393a75584590513c78674fa3f0d6f	09671885a19250f7c5512043ea3816b174e150e5	155f0ed931821473758ba6f804d113c50afe262eb1c4c182e3a0f1a7a09970ec	True	MALICIOUS_Backdoor_Persistence
File039	Binary data	8f179f9b48d6c13fa96e801c31c5120b	97a36bda7a3707c371a61ca1be5babd819f22dfc	abbc52db824b9a134dfd0ef0c646a8e2382a05e7af9a9f5865b77cc704844495	True	MALICIOUS_Backdoor_Persistence
File043	Office Open XML	43e1b1fb5bcd16c937b0da2c4a6e540c	661d2ddf9bd727de31c653aba3edd83c19e0959a	13e84bab911c370a9c80081d6a711f6f334296996cac27348aaad72aa59dc5b7	False	
File044	ASCII text	58e6cf4d000a0330e09c022cf94f9be7	8318fd3c54c48db49737abadab240afb2c7fa7e3	d6a99d5fa30152aa09b95e8198641cc9ee95c27051ceb5031b22bacb674a7073	False	

File045	JPEG image	6d8f6f9f0df51f804f0d9e706373d881	1a4ac39badd635bb1a9e764a70e72fa901b3c98d	e3a4563c0f11178ce2eb4623899a43c9221b68e6e0b8ecaeebca23c4277d9496	False	
File042	PE executable	74a4b714c851de93c773b1772a40807d	48506056f569a40ee28155c507e3578af5ea4579	8b9d7b249fd8b32d14465b6d038060da3545114bcf0041ada1f7205b81d6887d	True	MALICIOUS_Backdoor_Persistence
File041	PNG image	6f821e51b6c6f33e1768187ddd1fba46	9d978cfbbe2db83aa8c852ec6955ddbf1969e504	d2c03ebbcb573163994d6eb247209a6cd9ffd5141b14511e4766a31b303c0b65	True	MALICIOUS_Backdoor_Persistence
File046	PE executable	f63d16a402673307cbd7e0a69205baa4	afb6c414197688a1bc6c113bb5add0f02f308a50	2300646f4daccc0773f9cdc078f603814c2025828cb978137e3b38fb1a2a64a3	True	MALICIOUS_Backdoor_Persistence
File047	PE executable	130368dcac9cf8171b792bb164b1cd1c	a4609bedc4d885a3c0dd0f3fc9757af6a377dc8c	6a34ddbbc2a43d2c911fbf7b59824c4b7860448f7c1435e520b562af33614d50	True	MALICIOUS_Backdoor_Persistence
File050	ASCII text	c9ee034bbc5f70cc0246bde27c1be76f	600cc227ee8d10820ed06a214b58ea151c213371	0bfdd7cb1d0cb8697316b798e83792d11634273aa4471e31409fe86c15e70104	False	
File048	Binary data	e117cf3498fa380b6c06de68232ee65a	f6745cee730e1665db5932e0ab51d53a885a5ea8	5ac465d7d37bad28b424c139e142cea865d163495182eecdebf02ea91705d0a9	True	MALICIOUS_Backdoor_Persistence

File052	JPEG image	a978b63e14d67b4d8a9625655adb3838	2aebfa2cce4c9a5498dc9541bee9cb27cce887ab	2853b3fa183165c4ec4fe29a88c722c5573c60c40ac92581fa2ee53f6af22a8c	False	
File053	JPEG image	b7bbbb113e2913b4d54d17b47ec11308	e7840553b64fad9bab39e0c5a138e21a18d313ee	cbc1858fa38513e9d63f90483cccc15745a5bfa58642b9fe8f60374f7d684308	False	
File049	Binary data	428232826229b4484bd2a9de0091b60e	1819ba9f99f764843716e04ed1d150fbe9efa16d	9f85d5be7ccd66b89e730bb720e157bac05d165d38cb41c343eb21421c10e359	True	MALICIOUS_Backdoor_Persistence
File051	PE executable	db97870e110d2e495f191c432797ad04	3a73e650e6386cfd23f3d726c347a9359a37478b	bb09f3b4e07672d3ac19638db0979ec4c5442492af49b14006c60c55a97f7d04	True	MALICIOUS_Backdoor_Persistence, SUSPICIOUS_PE_HighEntropySections
File056	Office Open XML	1c03344f83275ac70b5391df2e8e0876	bcc8198a1d3d16525e4bf56a0d42a078a8893424	eebca9f734909c752bd248639b0b387908cb6df0182ea97346857fc88affb08a	False	
File054	Binary data	2c6444207e051dbb8f9f8ceb8f05bd06	7df60f39ece494a400b4afb33484d4652cd16003	b2234d575d7257d1280199c630c2a9bc15d637dd04e040cff0d42dd8001f9531	True	MALICIOUS_Backdoor_Persistence
File058	ASCII text	18039dade136470d4112c839cc043efe	6cd50628e742766f69ce4dd62c8a351b4eefc8c5	0b37ab6ce65c46852c4201e7756c243a8a019ceba7b62558b2666a7eb24afb17	False	

File038	Binary data	ee65f859755a2a7aae846761cccad234	9cac09cf3f08cdfa6042d1bc8ef45cdbb1ad7f8e	131694dfcd8c271b37e54ca9e5c1abbb9ac88526e15cbb9356084667282d36eb	True	MALICIOUS_Backdoor_Persistence
File057	Binary data	7e40596c2d0399b64cc871ff4158ae57	f440d75b7b3c54939699edb42c48fed43ed131b1	9c81453c7de7f7b959263051a8107ddafaad8a53fad4125c6a72541eb06a06bb	True	MALICIOUS_Backdoor_Persistence
File061	ASCII text	0a9bbaa0e74a6e389c73c1a18d9897ac	c4b960d9a12c103dcf41b68eedc34c1fb59e1c37	d3dd96a21af17539c75ef957bd5a90b745c590af2667c282b160bdb2f0732067	False	
File055	Binary data	386812055de7c60fd297046a661ffd76	7e5659b43870dffc2bc7e1cebaf5aab028085d4f	044dabbdcc1d8d6dcac29643daf6a30de44ba64d551361019d0ac29f3d09c551	True	MALICIOUS_Backdoor_Persistence
File060	PE executable	8d404ac68cc517b456d0c46305caf6b4	da8c7f8945337943f5e1fe1d1513395144711333	35c1692f61a508740541e3ce33ebd664db88b7d78bbd99994b9082c42b99efdd	True	MALICIOUS_Backdoor_Persistence
File064	ASCII text	ca807e6ce43369833704d96678d19667	f53f88263eb2968f54c43bf34a4905d1c30ee469	06b41becd146579b61b1ff06b083406ec810d047b89adef3eaa0e50a3d6ade33	False	
File065	ASCII text	3ba7c71043ab870a87d59a781def60a5	baeb7379a48da8345479b809c9084eb419d38764	ae1723cc3fb17ca87d6be46de09162dcb52f4b5291c58a095e12cbe3bb5e2414	False	

File059	PNG image	ebc5f89869f7306fe420ca02a9677f9f	d8096c532ab21c705c08de6304672f2a0e6def72	5220da04b5ef146ca0230533d56c2085c170d9557c7333faad22f3bc766fb2b0	False	
File067	PE executable	ea15bc1156bf638002b99c63bd4abb1a	1256d16942369961683c7f21c7f77128ea1f73c9	78d9ccc3aea19b6656afb4cb7a7afddf0eb3fa2ee3c348a628b3beeacd0e98a2	False	
File062	PE executable	f0c3b5b14d0b5c1420d1e22f3585f4c3	cde069a5cbc8bdefcff8a67dcf26538cf35eb99b	145cca35fc844f511e8af7756374d27125a440b088410ade80c15527fb39e224	True	MALICIOUS_Backdoor_Persistence
File069	PNG image	314de58ba4408e243fc5e7f691014c26	96deb65cd007cbcf829b7ff81e5629b03cd87907	4597944501acf1ef4475162997e25e581a6f5d7d760f85826e6960ef5a49d1fe	False	
File063	Binary data	2b2cf30e60c4c5c6a7bfaf7f05f559b3	82cf0d9d4f3ea20c6e2f8f55c883865d5f77b5ae	f1330cf05c37e77a9b29e7078d564282ca9c897ceebf50fa1e2078d21e48cffd	True	MALICIOUS_Backdoor_Persistence
File068	PNG image	4f81b658e6cc40fbf7c4664e14b24a29	6dc6d6c0568f695a24a42fcedfcb2409aa550654	e9099b712728530d628b1198561e6c2cac999f54e07a0a3923b9ad6f2d36004b	True	MALICIOUS_Backdoor_Persistence
File066	PDF document	597122e0437a3753b00487694f30de80	a99f8441c9dc40ce170c0b7e129caff0ee4206ea	dacc4061e08088885f6338bdee771df124755e4b59c8abdd6bf96b9b81ddf538	True	PDF_Embedded_JavaScript, MALICIOUS_PDF_JavaScript_Objects, MALICIOUS_PDF_Exploit_Strings,

						MALICIOUS_PowerShell_Suspicious_Commands, MALICIOUS_Backdoor_Persistence, MALICIOUS_Generic_Suspicious_Strings, MALICIOUS_PDF_EmbeddedFiles
File073	JPEG image	614cf260d4d13d5ffb6f947cc806d671	8082d965f637e78065d417dcf7c42915e7cbb988	bd678df29699f8c9af45ec4bf10f8b7a197bb0d07850fa5c64fd013910bc5e5c	False	
File072	PNG image	5956a153b3144e3256047ab8eead83e5	463c9c107e470c360e75f5e7e569bd6c3a4ae9e1	cfe80c6d2e2233791b84e68c40dbba5be93805c5f8ad829cf525b9dfd67c25a8	False	
File070	PNG image	4effb217ca55d93bb55192693ef0ee9c	72acaa872934217c3fc370c361a21e22853c6565	c4cc527b4b4c5cab414a2ca920a3596df19ed8f5808509672b6b25b7feeca047	False	
File076	Office Open XML	585ce4469a6adfebb17e5c7b1163f349	f90e097bb2880d250c2d1fe6d6d69f49a4c89ed3	f7bd687eddd64d561b50372ddaef38b90ae58a358099a06dbf33a894856b9760	False	
File071	PE executable	08422e540f71afbf8895440dbce80aa4	22f690f251032776e47565d46f3285c4011695bd	f650b7d8b20d0dc12ecdd0f683c3a1252d4fa0d9941378541738f38614af3c67	True	MALICIOUS_Backdoor_Persistence

File078	JPEG image	2431f8e75577a0e57896d66782cb9890	11df096f561dcf067384125730ea9c354217abef	18bda15f6aed8448f4d714569c4ca343ad8e7dc0ebdd7016fe4ecf8ba51b2c2d	False	
File077	Binary data	f5fa09dace753bebba5b6075f127dc26	6b628dfe25adc892b188fbf79dd627b55f0474cd	611d489613cbef351d56d9e5b5e51e44d95376d60b70cee8c1bf4fd9a3d128e0	True	MALICIOUS_Backdoor_Persistence
File075	PE executable	cbf27b79169a22e2ed9f2f6ded176cdb	8929c844ae8a6f80694d0e8de149adbae60fd78a	44703ef6d22f6859c137eb844f3939d0e7aa21fb7f6bed16861758bb9e5229c5	True	MALICIOUS_Backdoor_Persistence
File081	PE executable	52376b5fbaa7a368612ca909730cf9b3	b4e26ea63c2994e7d994c052c52d3f4e8da27c9d	9a253a8e0737cd0f157afcdcccc509e8da3e15aa072c5d1aeb3921aa9364216b	False	
File082	ASCII text	559dd7a83d0d7688e334978e75d45beb	9fa22c8cb4ee7c89f6787dc7c131758bf524e13f	a8e59a9df5508fc37a2bbbd42cb25427b0916d474b012ba2bf826190145594cb	False	
File083	ASCII text	d324ff068ad2ee0ce710cfe95169711 6	5355854158f0f85378e0e61a6176f12e805af563	07c2b8430a74a8ac1622b5223d7b11b1cdfbc24db0e0b34ef72697c35799a91f	False	
File084	PDF document	0003f23eb36cf0145bb82e108295b143	c514b8f17c465ef3d259d44ba6af4295b568f802	8d274a84770ef915b9a2470fa11e0127ca78cff6717552b57a8a155aa3381533	False	

File079	Binary data	b6167a060ce70bcc90de22e27a3fbb07	875417a1f75bd6b215b49df4ac0daf19697f412a	96cb4aaffd42b83c1f3325208d88d1d37bdc985d1e09bf3a01c42c3f4ce0a943	True	MALICIOUS_Backdoor_Persistence
File086	ASCII text	6130cc3a7797bd20785db9a1e38246bc	c60b117808fc4e70da2fbc77f3a66da0ff9d655c	5c2ca1f15d4e9c3964785018c1f0a0309b2c64279c1b93cf3116fb71b60b9a0d	False	
File080	PE executable	12961a8d12e77fc9b66345b80456f7fe	ec320d7a73dcf97b4b4fcb6e764a238a318e38c4	4879d504f9bfb07bea63f737a7d075d02bd3ec8fdbec4acbcf13c22e2bece3b8	True	MALICIOUS_Backdoor_Persistence
File088	ASCII text	822123f73ba273c1a622f5d5cf32633b	3e089d46050c8b97e53a7892ff27fce5577062e9	5cdf52caf1a723074afda063ada5d669fa3035e4122618a1379530863b6ff471	False	
File089	JPEG image	905363c4abd5f39bc6f98ff3e25b0281	3eea1966f23ac6499d0cb9a013a1d14fa1919fca	34452eded8f8e27c105408e04149ed8de01b80fb4f05e8683ba24e831554734c	False	
File090	JPEG image	3e736f0149faed08b112e43245750aa8	20c0b1492e950b392bf8088f673a5bb9072983cc	bd72109e3820a742b32a683a315f5bc0512250da6dcb4c46744af3e85f22da12	False	
File091	ASCII text	00cf085bfe51c5de71ff776ac8257e34	5b487a6cd44e96db251408822b2f08c833f77152	22711f384d3443a30000d7829ff71f09745995c935bfd2e5f730884a4aa557a9	False	

File087	Office Open XML	b4ea3fa8c44c0e05e86e4b0d8859cf0f	ad9ada5e4004b1892ff0800575ffb3ba1ca2af0e	0b56a46148fe47c91eb72148db0d6948ba562aea7431e4b71eb5cd855ac0f1a3	True	MALICIOUS_Backdoor_Persistence
File085	Office Open XML	5a237806a6afec0a40fb4335303870 01	3163223d1fc12c29ad6a01051f00054c5995f8c4	ef72773db6a85f9958aa9902903f0e901526100d826007770c095467ed72e9e3	True	MALICIOUS_Backdoor_Persistence
File094	Office Open XML	d46229b61ac5fa694f1e5547d7cf424e	6e3978345a445e7e659069466f1707e46f8c15b7	c5f3ac4a574e2bc4485f118c2861d93f6bdc8668cd28bb307fbd685fc98a26ed	False	
File092	PE executable	cb22ec32b6b7d83582b7fda7d682ff89	47a126f450d91264f4a399c52d25efb622d2a169	4cd742b67eab294606c6f7813618af6d5824c2fefdc7469941a7f4394d8c3d12	True	MALICIOUS_Backdoor_Persistence
File095	PE executable	bb24a4f283eeeb0a5b6a63256e197fad	e8039cd053f2974edaa188541159dfe12bea125f	bc00e73aa9064d4acca54d5b4d69bf2e7182f31169fd8ad0a214ed6b306a7b0c	True	MALICIOUS_Backdoor_Persistence
File097	Binary data	d9b96769f858761 92c5efcfa556ab7f4	fe10fede7cc3507280bc6913686a0488224abf47	98758513eab9dec8874b5bbe1c651544721007d344f9d8945c95c76a70a20bf9	False	
File096	Office Open XML	6b8f2e87dfddec9238fb7252e63ad558	f426e87d01f2d3d46e2217e053ffaf2b8f518511	188899ce3ec824a842b71508e6a2b8fe96bb96d4bd7be338042a88df447f11b2	True	MALICIOUS_Backdoor_Persistence

File099	ASCII text	492b46063a1fa97a738da1a411aad4d1	a85e740b01159f63bf255c5feaa89eeb7a917766	1458551d93f12aeada0bacd52b1e17f75167cc90fc09cf7504cce2b296352f65	True	MALICIOUS_Backdoor_Persistence
File100	Office Open XML	95b6f67c991027de4266a1e0e7e8d527	9be1f1347b642ed0becdc2fea3e4f272860a91d5	8d5335dba28ff705ead7a9f4c83872cdb0908e34f21421580a52c2ec8e72e446	False	
File101	PE executable	b022e52fae3ea7da7b43c8e80ca646a9	cc1fab36ee81c119aae9856ebea2e1735a309b30	655ce7fca0959a0eeb39b41dc7bdf3e094558875fb433067b6abce1be34c7c08	True	MALICIOUS_Backdoor_Persistence
File093	Office Open XML	8a0b5139a1e35f636b28893cb6436fb9	2cc913d580f766965d2a1c95cc60303f84f5dece	a3c1db4b5a48165f66b1c52610c8c3c4232417756cf6f638a273020cd7eb8707	True	MALICIOUS_Backdoor_Persistence
File074	Office Open XML	f9b2c6c68a3ddfc2678fcd9f0cf0464e	7178220d10182f9ab2c3efd1054a8872e24800e9	1a8bd54fb55f42a50ab36aae0ec2e366a7e16f2169086aa448d062be79b1ab1a	False	
File098	PNG image	e23a82888ff953e592c9935f447348e7	78146598cdd205e741b122cf067352b1d7ec9cb7	a920e0ea11d2ba741547e289f88de354c5719366a8b3d32cf37db91cd68bd219	True	MALICIOUS_Backdoor_Persistence
File102	Binary data	63acfbba8e6e6dfba235018deaad7ab6	6bb6cc99155a64d8a0210ffeb2d455c4f67b32fa	6f07fef56af1ab119fa864ceb2d4e47a1877573ad12d4adae77d44e854c19552	True	MALICIOUS_Backdoor_Persistence

File106	ASCII text	fe975f7601a2126ba0431d2e84cb71d8	9b6443c7c129555a34070863f8112406befea355	f14a00c695926cd69546d07703f863828470524760afbce999abb84a6c13d900	False	
File107	JPEG image	62b5f962ab419a0e7d39693a47f6f8cf	85f3663b76b7a5cf7a6a7509c92bfbffc574a15f	9ae9a368cd561264a95abb5e5198f0b449c17afb9eb4eb93352f399978eac44a	False	
File104	Office Open XML	315c12e381824fa8187634f37e4d12da	ceb6330fa3b9c7221d97447fff5201796eedc3d0	b8c95670d177067441851cf45d2da12f39ec89bf3c9e85dea17a531ffadddfee	False	
File105	PNG image	2e8f9dd9eacb56032b79a06250e949a6	1f7d9c86df7456e49ea0bc11e8a6a86c4c07923c	67cc5157e5d6ac139820be90071de05cf2d90400bb01f87610ec0b9f1aea9dc2	False	
File110	ASCII text	8b17d3c1a890583d649554e66df1c9e6	8e542857954afc39520828bcdd48bf58cd3f08eb	e64f9accbe14dc70a30ded34c08ff2a374e1b69a9ba777693eee04f83986d5af	False	
File111	Office Open XML	bbfd94f750618d4eb4fc8c3d6a9ff11d	d21850fb8f7ff21a8d48aaa9bb928043416924eb	be4813b07294520099ee561ef594396d61726eb042568cdca2bd27af6582e2d5	False	
File112	ASCII text	c75dbb43150fd2ce4732c77ef9997def	82c0591ef8737c0e3a74a0b98e4ba4f9996f1189	c46cfe45b8b38d43397873bcb7fa2e4d17a4d20727e77ab8d86a66bc8acd9a32	False	

File113	JPEG image	8019c9631f622de19c81a3e19df8b4ed	eb7d1a333bc894fb8c1a9afa2f70768246f9fa0c	c556339e370a7f8bc3fd9c1966adad947090eb47a158f906f0a21e292cdc54ee	False	
File109	Binary data	a8db5b9ff83ebe7e87a39715761589 77	3ece35df69d47b45cf2f3d6cf825def15af9f531	03ef8efddfe82b94dd5ab780706f928bfb3c614e21fae4686e9812f06fc000b4	False	
File108	PNG image	2083b7845475c50f4d3d0ce43abddf47	90f76c4eefb5140ed52e3cb09aa9dafe54e90501	c4652a51f423257ee8f535c8ebab08230171ab681f25bfaed0caeaa7f4d91ecf	False	
File114	PDF document	9b702a3a329410a22d2877816ece84c5	83e2b181ad279115050e23bce261ddd082f1b650	7662e8701c4b0dd703a57a78f4be877a743315267c28fb78ced2ba2f286bb8dc	True	MALICIOUS_PDF_EmbeddedFiles
File117	Office Open XML	0d7783aa3a521e6d64aa1a829c5ea90a	6a6a963ea8245d4034c2ebcc01e98e1c6feb3440	9d21d85165b09487ce74a8b955757454616cda16525b02d2e2043f21de7c0bff	False	
File116	Office Open XML	b153dc51097eaf6bc297727b41d16fa9	73c3036755de9c36bd22f36f0238d56dfd094240	358f93a1d4b7abbfbb30e5e1cf307101626b20898ae1979c4cc53238f24ea25e	False	
File115	PE executable	9e25484b1907186fa37395a4c3b79940	65e3ec37850f9d2d5ef82f264171611be5414d5c	ad1d9a68a9007982adbf3af743d79b5e21bd8048c8ebf513cbed290406c26a4b	True	MALICIOUS_Backdoor_Persistence

File119	Binary data	fdea5ae8a8ede952bc58bdb26b47d57a	8960ef02e567df940ebd6bd1f46c06a3565fbcf9	1b94ffee23b76220566a8507d7e1b0b1940a971e1a700d4496658fcd3b5f5f8c	True	MALICIOUS_Backdoor_Persistence
File121	Office Open XML	78db5fb8d3b5f11d349384c5c808d94c	65b6579a5c5dbe6d42f49031bb4c416a0d36f254	efbbc257b678eba3fd5a0d5ed8d5490a4a55fad8e4cd1e491a6b8cfcb3ac6945	False	
File120	Binary data	0f3ee7c9eb9340ae44a93bfcd27523ba	6d9e489b5b0f096484935299db389aa2fc42332c	13a6ac51b47813e8e2bc1fea590c05cc6bc8e3ad964e3518de976d3ebb66f8f9	True	MALICIOUS_Backdoor_Persistence
File118	PE executable	e729b5cc0ca22d973378a595f0c02cf8	c278fc9f054943a8bc9fb22e322e588d30494a4b	624ee8195382d2c555046293546c65c215e25751b75c0aedaa8a842cc22aca11	True	MALICIOUS_Backdoor_Persistence
File124	JPEG image	d2183bc6bc61647833e480df21079df5	eeb2d7a4366130e5523204e5e85832c7a0b2d5be	115d22a6395054520a5e9e5e3e3d802ed1e2152455daea71b1bbb3fb3818cfcd	False	
File122	Office Open XML	423e0563fe719395891fa51a000a41ad	83e586e9a4ccd04f8c77cf11bca3d1339e891340	af914f86a773bb8a035937c1603ae4cc1576b83dc6e4149bbf3d5f1219500015	True	MALICIOUS_Backdoor_Persistence
File123	PDF document	bd0ea60d14eea556636110 9dc08af4ae	e84d7eedce3c34193739ab5f41af545a56467971	2c174db14e47a3582b5949c040ee19766c0a1e79810b924609c737ef14a77de6	True	PDF_Embedded_JavaScript

File127	JPEG image	07167ed9288a6fb574bbb0ad229bf1b0	ce30a350e536ad0ae9af5f625a0213d6fbdd8e6b	e188f2817b33216c311c017d1b92e112abcb5bee1a2d4437efc58ab7490cc273	True	MALICIOUS_Backdoor_Persistence
File128	ASCII text	c79e9121e51d9373a410b53ae8a3d964	52bf71c606ab98d918cda1fae17ecbdaf58fe34d	aecfaf78992755f73ee068196d43a6775e2ece96c2770171529e2517633396b6	False	
File125	PNG image	b4eb8e22a873e2bff9a2c056c20da979	e182a3a956d31cf17eae094508ecda51bdf0f9b3	314241eeef941f55c2be2c9da7d161d5c55e91c020f2fb5e8af91e418a695f4e	True	MALICIOUS_Backdoor_Persistence
File126	PNG image	eff3a759ab08c2c95fdba8c3cde7c1f8	2515298c9ac3310c0edada1f005041b1dacc8750	ec201bdcb6f90ca2416ded4104e63c1a1f08ea14f4c22a2cbda99ee6c8259b6f	True	MALICIOUS_Backdoor_Persistence
File131	Office Open XML	ec026d8ed4a1aeaeca8eeb4e7bd17715	25d67d32197d0c32dab7c19404d7d33eb6c3e89b	ca081c2d441688ece3475cdbd3136d3249fe993605e1053eeafbbb441f9486d8	False	
File129	PDF document	321ae2c8ccc6a16073c16910ec598891	c0ec1fdf59c40d422a2fc1d63bc7fb23c845db7a	34768eb405fa77fd6375a3d87e40ad1df08c268171dfaf20230774e6c7612a8f	True	MALICIOUS_Backdoor_Persistence, MALICIOUS_PDF_EmbeddedFiles
File133	JPEG image	89e432c568ecd36d5136ce58c923eb9a	0d7e846937431043925526e882e17ca758adb83d	0dcaf05f6c9f659cb8e342464316504b7ee03cb0aaae1731307206bb8130d74c	False	

File134	Office Open XML	d6eadec77ea7a7436fb9e5f39bdc9b36	711fdd86b47733ffdeeec89f21a52c540ee7c23a	338d8b3851a2bed5444ff6364c5b70d6e5e1901195611e4a0a7592423375cab9	False	
File130	Office Open XML	1e40fe728f54ab560be84048b37e2136	5be30928b8af0b3e684761d3e192bc7d52692a0e	1e6534a07ccd6b616c9a80228698c73903dce2aad3053db598e9ae8ab4788c1e	True	MALICIOUS_Backdoor_Persistence
File132	PDF document	d0787c3ca7aab644472e1a015c23597c	7c0efa4720977994df80da127d1f3fd7f278d197	d795e506a98b382038c9b29117996c95f2edf9388105fc61347539c8312fbc83	True	PDF_Embedded_JavaScript, MALICIOUS_Backdoor_Persistence
File137	ASCII text	81543b22c36f10d20ac9712f8d80ef8d	892b34f7865d90a6f949f50d95e49625a10bc7f0	36266a8fd073568394cb81cdb2b124f7fdae2c64c1a7ed09db34b4d22efa2951	False	
File103	PDF document	1f6d1bba0572b67079ce8b19f66377f4	310d4f9892781915a9b1a06be70a145e13089b8a	fc04c312670430be2afbd2119e448d49c95d53351777cf308b0f95c94ce254ff	True	MALICIOUS_PDF_Exploit_Strings, MALICIOUS_PowerShell_Suspicious_Commands, MALICIOUS_Backdoor_Persistence, MALICIOUS_PDF_EmbeddedFiles
File139	ASCII text	0618ec59c2401e95bc66f5f48bb46bdc	de10d7b09c2b49ae3b8beda9ee949f1cf084a519	2c214b7b450719ebdfde3e695e521d5b68592086983ee6bec6b9cc8a9983fab2	False	

File136	Binary data	31bbd83292460bdfb30a7bd9ff9aaf4b	369cc313e0ff83e79e030eb69673e2287386f3d8	e9e9b42ee499b4d0e717bbfece1011af2127c9bc70dd9d1e8feb0a59e654c2f7	False	
File140	JPEG image	f83e3fd9b34363578c2128dc3eac6f01	64d091ad38f3289dcdaac720635577f1fabbcea6	2616a07362b644bcaf0bdb9f045cf066d1dc270dc3ccce937d4922d73954007b	False	
File138	PDF document	302b502d4e62ce9364d06014e544c095	418baca3e862279dd832d4489d6a82d8665168ba	41074035c291360d30b057eab0418219e3c531bb3186be50bb15bffbb8d7fb64	True	PDF_Embedded_JavaScript, MALICIOUS_Backdoor_Persistence, MALICIOUS_PDF_EmbeddedFiles
File141	PE executable	ac1b94145d5bf11f34eba1137d8b39e2	c33584ab2f523cbb8556599ccbefc67129602668	e9b15661934525f6e87edabbc0ada0a8604398b2f1f47435803a9cec645a3881	True	MALICIOUS_Backdoor_Persistence
File143	JPEG image	cef114c5875ba465fbdcb7dc7b297a42	b0e30d82001184e7373b2576fc229609046f104f	81759f96075607fb7a984d8e9a6471e85a637146fec753e48eefe4d05a37e626	True	MALICIOUS_Backdoor_Persistence
File145	Office Open XML	d688f330a67743da9edf360bd87a4757	50314186f6005a2f6383fbfb9f112588096a9323	dff503b08e4f8818a56b64390d279d741919f743891329a4280bbb0061f6b2c0	False	
File146	PE executable	290a44f94f15d6ae3591532	dbba2d945d3a10d313896d32d90af9278559bba6	31096b3f8270edc5ddcb3dbd462c41afb39b2caabb928	False	

		0294947f6		aa6938010a7e24578f2		
File147	ASCII text	a4da80722c9b88f6fd9261637af70098	15d07a19e6cdfeca5bec0472779a3a4258e759e2	2ddbd5f4ce4eccba52f3cc7ecf4929dfb571361d97a33ab3246617f70c9d482f	False	
File142	Office Open XML	ecf92b48eaa6ff7d658170aa49a6e660	ab486d834224d33b59515b7d1c738911b949c126	c6d39b4da918770a0a8ba7af099d3fe93723b0fa335814e073d2aa75d1c18bd9	True	MALICIOUS_Backdoor_Persistence
File149	Office Open XML	0a35b950306059 7bb052f21b4b4290d6	98629156401fa0e895aed31c838fd1519d2674ec	ca651505e86f7be138e5fc8a11b450a175064952e4bf882c332aba1831497212	False	
File148	Binary data	ef063d467563865aa7a97b4dae9924ce	c87af5eee0cfaf88c5176d1ba7591bee24a32673	9e345ad76741371fbba34da23c39ccc3897ae35846ee44ab79fbfe18d3a9fd4e	True	MALICIOUS_Backdoor_Persistence
File150	Office Open XML	4cc3cee04f7e695a8de8591e15f19fc1	06b1ffb9db80cdce1e1623453882ed96b9602335	9db6a42860fa24a4f1b731f6ec736bd1235f95806c66b6d27064dde5917a974e	False	
File144	Office Open XML	d19b96636f75b86f0f2740e6434e7b6f	e601679953b0423457f636adb46fd7b3192cf6e1	fdf18d8256d5950e63a8ae81982706e49c206760e03896555156f2d935718b9c	True	MALICIOUS_Backdoor_Persistence
File153	Office Open XML	07b039a576b247f2719d7c1	8ade27f7dcc94cc0537bfa99754daaf97b8adbff	2d722a116f991e38f3f64fd486109be6280d91c1804682	False	

		372e7d1c7		b4e40f67a7682d899f		
File154	ASCII text	a1c1ccba318f1630f5ffbdca6cdd12b2	926b7d6b492c6f9c315cfbfbec82a7bbcad7d30b	42f2787a27c9889b60b1ef6da602c57c9a4fe0b6a945c7cc0fda60a8980d7e35	False	
File152	JPEG image	d647f67138863d8bf85b2467743927 09	c5eec9ec4683f0eafa40897a37d7d88b52fc21a6	6329dda11c90cc6461e58fbebfb0ed1ae744d5177b647748826eb22472b128c4	False	
File135	PDF document	14ca718fd2c6718e4a31472da950627a	363e3fb566ee15d4f4a5c35ceefbd5d1a5343f03	dbcc6774709f846bfd632ba7f29bc152b24a5afcd431e1c875a7e67e97c9f6b3	True	MALICIOUS_PDF_Exploit_Strings, MALICIOUS_Backdoor_Persistence, MALICIOUS_PDF_EmbeddedFiles
File157	ASCII text	362889c21e1a9f7404c13205ea1b9fde	5c86fc5e9473205eba78ad2f46718d01c588b502	320bee996580871e3863b5d8bcb333a83519bff327e2764bb82f84065f44d25d	False	
File155	PE executable	774faa35314ccf72d1b9d841b4532b03	ef57d337b2480bf4981091d9ec3072494f3c76f1	3e30293590a4f124e5ad3a783f312f0a4607331799d31e103011a90af5aa8e92	True	MALICIOUS_Backdoor_Persistence
File156	PDF document	8e984f349c1855d2ac331a822f548ddd	a8a0da335bc884d7c7314719036f326c92233995	2dc04d6ab4d83b63c572f6a2aa0817c70394b8cb7342ea603a04e8cee59d764e	True	MALICIOUS_Backdoor_Persistence, MALICIOUS_PDF_EmbeddedFiles

File159	PE executable	2fb6c16c0f1c39d9ec32dbb311990b9c	dd56bde584ef1c333304c30f2cfa7311f576139a	c50e84b679470efe6494352ebef66cda2b5bda1dd30981036ff4faaca28d9544	False	
File161	Office Open XML	c875c6cc8e2086011748fd6f9de83ff9	b75a7ca6961ffdcafd25f6b3b5a523a71b0fefc1	b424fe8fe48d94e754bf63fc1d31eca150177edbd63065873931b833b1d92c8d	False	
File162	Office Open XML	174c53b13bb9ddbdb1984108cf13e210	7be93d31bbb834fd3e1a0733d8d507ee168852a3	19cfcbd8a84eafdc7d99627b9a8e6a3b38d2414e124a412cd928ff0c3107467e	False	
File158	Binary data	5eba08d45abbd9fdbf82cc748a2b9774	1e768d7f6c7229c0db5e17f636f203ee2e084f72	164de5ced33b91f414e4837d123d2d77b44a1abc6bd23ccb8183f0e1355d0d9b	True	MALICIOUS_Backdoor_Persistence
File163	JPEG image	0691b1534a7b7dfa1f311471d386d105	68623a4e28d08d0de4e03b9089981c77563d58ba	a318143f9603d13e7c8f2afbcb27d8dd4045676773ea4170e60543e39f9d5aef	False	
File160	PE executable	5b88c51ea10f9db362369f3d71bea569	bee2ed1bea54e7d00b1bd0d44d817406d74a2af3	d0e714e047c50e9635d0eb408a19a5487f64950d40aa7e3f5fb03fd80550d9ed	True	MALICIOUS_Backdoor_Persistence
File166	PE executable	1af712a59ab34e0c131b40018d31c0f2	2c8951f8b0528ea9db00688cd76e14383ed0ed21	565d4e064b9757bf3e30ff5f229f401c113aa6f073eb823f6d8994be5b189f0b	False	

File164	PE executable	f3dec47bdc290fb01d5d908775321ea7	f0eefa4f62179cf8ed63de2d287512089e95a9be	2d6f72967598597 38048cf02b07f381cab62045037950e590f419df824adfc36	True	MALICIOUS_Backdoor_Persistence
File165	Binary data	1d786070021811ce3d962a420d868155	6e09b863e9d2f5d7bdff1e2b2bd4e4afef7876dc	1adecb3b5a3882cedce9fd64cddafebdcb045fe7035aac8f02ffe64dc827593c	True	MALICIOUS_Backdoor_Persistence
File167	ASCII text	dfb1b490080d1aae0067e35550910c4d	d88c7c5224c1139118602158ad1d0217210780b0	147a3761456127bcb8ada2c34728a301d01acd316946aad7d605c3a9ce37e6f2	True	MALICIOUS_Backdoor_Persistence
File168	ASCII text	cc46e3e9ef97cbdc56318ee7ff23d73e	288f748f941f87169209eb1a058034c9a83e9b48	f831e7eed577481687a9bc0b48024e5e40b6f655fcde073ede964b50be5d55d9	True	MALICIOUS_Backdoor_Persistence
File169	Binary data	d44a541590b9d5973136d0e595da9ae2	35c5bf4c4b20ff967dc35d6dd1ed79037f3189b8	5ca9c2b5b336be53bfe6310c3ea4a51f85d003898e267d9871803d05b1c1214f	True	MALICIOUS_Backdoor_Persistence
File172	ASCII text	e322949af834af9a143a29625e33a4e4	018baebf31f8635703aa99d8e66ee097760a46f8	f98118e352c0c17b1bd04afee6f258d5c29116bf5e6b74fa0a28148ab372519a	True	MALICIOUS_Backdoor_Persistence, MALICIOUS_Generic_Suspicious_Strings
File171	Office Open XML	19af6dc29171e27844e34c3c807231d9	976a891eb695b834305d28d287311793cf803222	021ea4dc427c81adebedb8152f67e67de7dc74417b65c2e276c58ca7d27112fc	True	MALICIOUS_Backdoor_Persistence

File174	ASCII text	a8dfc4d3598873ff3fd86e28be66108f	474981497504ac58781039c01b9d3b32b18ed191	40e4e9e028ff2c341f0f10c7e39bf16bb5328d3faa71e87b4e9bdb117101f0da	False	
File173	PE executable	3c555bd977fd395f0e3e0b3758071d52	06849f3bfb38b3b4aa5e2a0c0ac88734478bb9f7	d042f2a62c3169aae27e079f596effacf5c7d98e3777f230908e1a27f97d4aeb	True	MALICIOUS_Backdoor_Persistence
File175	Binary data	ec7522bc9873d8fb52d2dba4f19161ea	989ae95e6645c4f35a6326c1e69973954b2d75c8	e1be23b940aa48907364cf126cd3ba656d13f6e9db0317f5072e22952e3c36f2	True	MALICIOUS_Backdoor_Persistence
File151	Office Open XML	c0e566ec23e885ddeb11e5a4070fb841	4247d02a033cb8 99485941a63f2f36e76c76c65f	861b99687b257f226a124b0bf7d028d46bef5cfb31cccf4e68bd448af06fc2d4	True	MALICIOUS_Backdoor_Persistence
File176	PE executable	6d8a087c87e8c522ae0b7948357ee226	3001282ecd140d7120b94da8defcb02dbb26a8b0	827be39a735f8c49e3fc483b5a57401c463c0aa306ab14f277f20453f45833a1	True	MALICIOUS_Backdoor_Persistence
File177	PE executable	894e538fbd29d9af2dac82abbb798aa8	3c28b3063ce80b3fd61e0afc6934e3180f5bef12	b12679d33126d2dcb0cd3625fccf5c3afc40d95c1be36dc55f7471de94929d23	True	MALICIOUS_Backdoor_Persistence
File178	Binary data	955ddefc4f016045fb167016e499dd15	ecf760f19d07c912466db0f04b5bebb91cb07a22	8f10f3044eedf59e35d2a9b06eafeb6c73acfb25cb2cbac88b55e57e49034e9d	True	MALICIOUS_Backdoor_Persistence

File181	Office Open XML	cdc2e1b7b617de1db19b5820fc9380a0	c19d042042cfe77f58ce28d55eca3f9741ba942c	8a44bdbeaa77f132076aba2f6e3f59738ea5f75cc17717536c65f862aaa3131c	False	
File179	ASCII text	130668abb2367024e10447831c36f700	6b0e6307e68f873cc4c01e3c9d02b61269784b85	2d0f446840b79edc6392af5a19603d1092bf816a0853aa7e4017029ba05c7616	True	MALICIOUS_Backdoor_Persistence
File183	Office Open XML	5c4c9c4f71b373d50967fea0d9293b80	ed4ac6cf06a6828888f24b58416542dacf6e9960	42efce0f18b4f04638c0f8804ac4b80a69992259e8e4cd842fe6b105eea3bfb2	False	
File182	PDF document	adc7a2cddbfa9080df9a9bce33153410	7774fd6a224c351c149d5db71d5c40fcfa762bf1	2e4e83d7d42bbd305f4bb20a15f0f4b3346b7b335cb50d8c3a546f681ca1b832	True	MALICIOUS_PDF_Exploit_Strings, MALICIOUS_Backdoor_Persistence, MALICIOUS_PDF_EmbeddedFiles
File170	Office Open XML	fb9de5ec633240495f1ee5773583942 0	18771e2a544fdaffb6403aedf84b539f9d1fdb22	bf2180708e187ea7b3118b65d83767054c8fbdfc258c3fc7d0980f4d936de028	True	MALICIOUS_Backdoor_Persistence
File186	ASCII text	cd58b4860366a300cfb76a652d6e1dbf	1376127733d1bfb923dafc1aa0d47dcd078cd97e	c665a33a2462010c88cc488ac006544014700c2d19b931e528cf40041cc5e9b0	False	
File187	Office Open XML	77d26a68e9264e9facac42	028eee19afce5d914baec769287298cb9a3f8b8e	d0a731438645729fef6185853644eb033f08944318d608	False	

		42de2e740f		5f46c9561a1a7dc074		
File188	ASCII text	ce1742175ff199f4d207ac4edc013419	c7324e8cca9b6ac3a39e2baaa564d52e13966a9f	915c0560471eece77640068019d519f6f5cb345877fc4b7b2006a2cc58277ddf	False	
File189	Office Open XML	38c425bff814be81cacd15d2b4bcfeae	431f43d1284050f1a979a7260e72910213395704	61907f67de8563ebbc76740b23bd97e332cb78a052b4491e9d7f740946cc692f	False	
File180	PE executable	6a9a905f16f64919e6892a2e8d37a5b7	faef2d937021c3ca367f275f0128b5ebaf50dbd8	93ad8b3ae0098063a59ecb9d4784d6222778e52c7c5f1d18238992842ba02ed6	True	MALICIOUS_Backdoor_Persistence, MALICIOUS_Generic_Suspicious_Strings
File185	Office Open XML	5b1392a9f190a431ac6bac3b2f5be47c	b8fc3a4d1bb60155967dbf863b71ddd42148b1e0	5a61ca0690a051e9dace2cacd65a9aa956278cbef643e4a259d37d8b1da2055a	False	
File184	PNG image	edf9fd64840aca234a2cf40acd08026d	6faea83b19c6cbd392abe6df9c6fc0d10570d67b	ef084f96946a33800a10c8cc5baea73a48025bcb100e1ab2c7f9c68fdd5755c3	True	MALICIOUS_Backdoor_Persistence
File193	Office Open XML	a7f70b3ec9911480be7dacbd1e37820d	420cc47965de6c73fe3065931d8d6fef01792ea2	d3f23962bd711525f1e2797f0ec17f15d2a0b73665065bd20cb45d23fa6e3c99	False	
File191	Office Open XML	7c3c2be5d0f5bd562546cc8d	ae984a2eebfad940599f7bc75bd60838d6d4eb2d	19df1c62878cf4ef4634f87f35ad6b66a1df5433e934177	True	MALICIOUS_Backdoor_Persistence

		03829aa4		2efa0ab2449962e92		
File195	Binary data	8581f7c7288375f14b8f164c83ea0ac9	3e38c02ce6df71a9aba792e89ea20beeb8032819	c0f86d1003cd3c7117dc032b943928910944d00a61de2b3aef60f51c902c896f	False	
File192	Binary data	eb0293aa9d3cd061798d7fe931dc5fa6	78642c9626ac9ca7e1812f397923b81a0adfabf2	105e7abe3ee6468f590368d72d02b63ff6eafe147a0cfdd994f952e2b7c9acbd	True	MALICIOUS_Backdoor_Persistence
File190	PNG image	2ebe5a3b54f71e08d247aaacd9536fc7	aef536ba6dd1c9f499815a869340642f8a1a7d2a	4ed04cdb00ed9809033efbf8f9ad3debee924ff4564f200c96d88a9dc508d7dd	True	MALICIOUS_Backdoor_Persistence
File194	Office Open XML	44661e9d8e0b19cc4cdc71c58dac4595	355ed21d8da04a0ea27205b807979240fbefb356	9c78b4fcd0a2b8c46af7574db506f6e5c1ae85250034bef6e52c26aa88424b59	True	MALICIOUS_Backdoor_Persistence
File196	Office Open XML	3acb832c0b5bcd7df8cc140a771c612e	8e63f741347a04d664a023c6ec09c6c8ac99d327	9e22d6eedee0a3399dc254fdfa1695b4a33e37782c2fe7dddca72f137e831c3c	True	MALICIOUS_Backdoor_Persistence
File199	PDF document	fd3900a6c106515c2db20b09720c7b3d	cf4cfbf3c7f8db99bedb4b58242b5a93f2e24408	be349d9ae09d8cf0f9065dd7a790a41416d3ebf93eb75015bd852266d3b223f9	True	PDF_Embedded_JavaScript, MALICIOUS_Backdoor_Persistence
File197	Office Open XML	abfdce4332a47d0056f54ec	8ff3d262e9c83355e5c7ca3255c0688b53c6c983	f5afab8876440ad6cb9b95a3988d0d97bc9eaffef86a3d6	True	MALICIOUS_Backdoor_Persistence

		d5ff64e41		68e74237d7013bd0b		
File201	PE executable	4e9a6f255bff5ab93b6a7a51ebf62e99	f4baeceff5c949c66a85edcbf3ac40a774e3b887	cca30ca51762ebc3627fb2bf0ad625f04e52ef1af1b5070132279f5f40015d7f	True	MALICIOUS_Backdoor_Persistence
File203	Binary data	80d158a5fce462fb2d54c71adc8e0da6	f0957d0ec699d079d4ad5adc73525925ce7ff829	24795cf6cd080bb4897f9e532b34c1e2ebd3d841aa5bfcd30d8c75b833d69518	True	MALICIOUS_Backdoor_Persistence
File204	Binary data	d601a536054abe2a84612a08b5b749f5	902f98265eb67fcd8d7da5eb0b8198071d887c85	a3a56b01c83255a27ffc21329316ee7bbc2fe6a0e0dc4449d1e8d154312b7ce4	True	MALICIOUS_Backdoor_Persistence
File200	PE executable	cb98f5f2982feefd5e6d9fca98bb7e39	c5c554b41d7005dafdbfc89c4f4b776331cabc48	e8f2c7602555751cf8fe40de5e51d1d51c3abcaf617af949cae601c7f21cb4fb	True	MALICIOUS_Backdoor_Persistence
File205	JPEG image	eae001e9a70c97bb5a2e6f17849c1d12	72cc668b05bac25159ee0198ce375b4358ab375c	d0aa4a315d66e1197e18c51b0924b09e5c716b1204fafb8ac43d101a9da06a89	False	
File206	Office Open XML	16aa2764840ee306b4d60ed118ed642b	3b02a535fa7a2ccbe80d975f2b468de92dd3b8d5	f435ebe4e5aef3479aedf5b2128f6111635cce44f4a7f19cb3ea8d92d21b216d	False	
File202	Binary data	2497589c198a5c7d7c48f56bd5c2a083	f69b900367fa62e24947333fad2e4d8094002990	4fb8184bee8c99d59555e7e387a0ebec6df0e3e1801aa	True	MALICIOUS_Backdoor_Persistence

				640133374ead7c56e78		
File207	PE executable	8c5c9499e5bf2515b46ad201e8340748	df382e06b0b50420b6d63240efe0681998fa8dfb	101bcf563ebe0e8ef4f0a63513614791b3c306c10f8382416276992a85dd63b1	True	MALICIOUS_Backdoor_Persistence
File210	Office Open XML	d23ddb02acd5a876e306962bd71f1631	43db097d4df5012de9903c2de395a68147071827	ff59a2acd6b318ffc75e2ed3c5936c69d56eda24b6a57049a5d75f14308c00e7	False	
File209	PDF document	76909cea13b5097b22a7faa800652758	fc407e7e35ab2470cc9228bacc41394a169a6fd5	19de82512ea67a8ca22985d597dc2190f33d23a410315fde887e60bd525a268b	True	MALICIOUS_Backdoor_Persistence, MALICIOUS_Generic_Suspicious_Strings, MALICIOUS_PDF_EmbeddedFiles
File212	ASCII text	4a08c4bd4d73b050f07c51d9d7192e57	df37ae666a00d846792533782a95b28b66d1320a	5949cf679127b5a6fdb19a4200f72a8bb2b6095bd61e64692cb6b39a06fa96f9	False	
File211	Office Open XML	c70ec8c66f59ecd212b7d08a4357b5d5	50b27cdf5ab5e9b9cc52a102fafae2c112403a1e	dcfff3750dfaabd52f4c62a35a98f1d0df5227aec8ebfcc2e8167bc71c4b05cc	False	
File214	JPEG image	3938baaa0bb6e35e2420178db164cb3d	03ad96e61cf716ff65919902d92c5cff197eee2f	568ea3e5a0735f8eb6cde01fe02dd588f2474ecb959188c4fe55a4252cf1d429	False	

File213	PE executable	2a951f81a38691f52fb9985513e46946	c79bf016e59f5a63697184e470304af54d1689da	47914ae30332a9b287cd49cf1848a923790dec9693742ea370ee5ae9d5f4cd4f	True	MALICIOUS_Backdoor_Persistence
File215	Binary data	ebc84c0bae4a06d2db24c264d52dbb1f	d9c43a3c4df555360bf5e0eace9d2c04a5df74fd	39f623db0d8f79359e3f585fec270bf9a189c19392e4a69d111ecc71415bad32	True	MALICIOUS_Backdoor_Persistence
File217	Binary data	f1e7f2ec19d0fd37d892e992a71fcbd6	5aa2bda8638878126e463b0f935a50d693bdbcee	0d359811829860ca2834f57db70b89d41ec044ab22f85f1a87f5c22228d5d899	False	
File216	Binary data	825341f997632ff776e477883fe6493c	e9bfe5ea0889af5ed63d66a63b7b5b0861212ebb	44e76ab7e070b8e79e26f13b31c2c574ac6194364864ed485961ecbce0a19dc7	True	MALICIOUS_Backdoor_Persistence
File219	JPEG image	e87decf83153221f84d73f70bb2f53b	043296334c60c14c8be4fce8b1a4bb962847581a	a119016709f406bf1550822b2bdc3807c1a2fdb953e387e9257e08d1a81549f9	False	
File220	ASCII text	db631805541e0958fd9e6e3616a42c98	28b9d9f7b21ca28bf2b7e01f1da1f5a2080fe674	a02767a52d8c1fcde77ed6d297d3ac17029275c6125af1c6ac5a618904f71c6e	False	
File208	Office Open XML	cb5e8db8123b89323a4822992b5ddd34	b7d12ca8a56e690c38b3d2c82a3b52e138176b4f	24a50953cdcb5923deb5c91fc03c89144d6b48ce288457a85f380f511cf6f0ff	True	MALICIOUS_Backdoor_Persistence

File218	PNG image	bb84661af617931ba5b294a8737c421d	76a256f4ddd4584f562af45a06ba9952434d1671	0037cf511e2b34db246d16b964f9373c21b2dc7b268b9cfcbe89620b16fc4c74	False	
File198	Office Open XML	dd461e6bc31f224519587dffd93048f8	93c63eb338cddb1cf2e23374e2bdebdf79302f4c	ada5a897f6c0a25fd1f71df86455346435639000cd10f1dae3d32675ecc75660	True	MALICIOUS_Backdoor_Persistence

Table L-1. Comprehensive Malware Scan Results

Additional Sections

1. Observations and Analysis

1.1 Static Analysis Limitations and Implications

All malware samples in this investigation were examined using static analysis techniques exclusively. No files were executed, emulated, or dynamically analyzed during the forensic process. This methodological choice, while forensically sound and essential for evidence preservation, introduces inherent limitations that must be acknowledged when interpreting the results.

Key Implications:

- Malware identification relied entirely on file header signatures (magic numbers), cryptographic hash values, and YARA-based pattern matching
- Behavioral characteristics such as network communication, registry modifications, file system changes, and process injection could not be observed
- Polymorphic or metamorphic malware that changes its signature while maintaining behavior may evade detection
- Packed or encrypted malware may appear benign until runtime execution
- False negatives may occur if malware employs anti-static analysis techniques such as string obfuscation or code encryption

The 113 files detected as malicious represent only those with identifiable static characteristics matching the YARA rule sets. Files that may exhibit malicious behavior only during execution would remain undetected in this static analysis framework.

1.2 YARA Rule Quality and AI Platform Performance Analysis

The effectiveness of malware detection was profoundly dependent on the quality of the YARA rules employed. Analysis of the four AI-generated rule sets revealed dramatic variations in detection capability, rule structure, adherence to best practices, and overall reliability.

1.2.1 Comprehensive AI Platform Performance Metrics

AI Platform	Malicious Files Detected	Detection Rate	Compilation Status	Rules Count	Average Rules per File
ChatGPT	5	2.3%	✓ PASS	12+	0.023
Deepseek	111	50.5%	✓ PASS (after corrections)	12+	0.505

Gemini	1	0.5%	✓ PASS (after corrections)	12+	0.005
Copilot	11	5.0%	✓ PASS (after corrections)	12+	0.050
Combined Rules	113	51.4%	✓ PASS	48+	0.514

Table 1.2.1 AI Platform Detection Summary

1.2.2 Platform-Specific Analysis

ChatGPT (5 detections, 2.3%):

- Demonstrated solid understanding of YARA syntax with zero compilation errors
- Rules focused primarily on PDF JavaScript detection, identifying 5 PDF documents with embedded JavaScript
- Limited coverage of PE executables, Office documents, and PowerShell scripts
- String specificity was adequate but lacked the comprehensive patterns observed in Deepseek
- Comment quality provided good context for detection rationale
- Primary detection: PDF_Embedded_JavaScript rule identified files File066, File123, File132, File138, and File199

Deepseek (111 detections, 50.5%):

- Most comprehensive rule set with extensive coverage across all file types
- Required manual correction of unreferenced strings at lines 326, 382, and 385
- Excellent comment quality explaining detection rationale for each rule
- Superior string specificity with carefully chosen patterns that minimized false positives
- Best adherence to forensic best practices after corrections
- Primary detection: MALICIOUS_Backdoor_Persistence rule triggered on 111 files across multiple file types including PE executables, Office Open XML, PDF documents, PNG images, JPEG images, and binary data
- Additional rules detected: MALICIOUS_Generic_Suspicious_Strings (multiple files), MALICIOUS_PDF_Exploit_Strings, MALICIOUS_PowerShell_Suspicious_Commands, and SUSPICIOUS_PE_HighEntropySections

Gemini (1 detection, 0.5%):

- Overly restrictive condition logic resulting in minimal detections
- Required correction of unreferenced string $s3 at line 270 (Shell.Application)
- Comment quality adequate but significantly less detailed than Deepseek
- Rules demonstrated understanding of malware concepts but implementation too narrow
- Single detection: File051 (PE executable) triggered SUSPICIOUS_PE_HighEntropySections rule
- The restrictive nature suggests overemphasis on precision at the expense of recall

Copilot (11 detections, 5.0%):

- Moderate detection capability with acceptable performance
- Required correction of invalid 'in' operator syntax at line 45
- Good rule organization and structure with clear separation between file types
- Comments provided useful context for detection logic
- Primary detection: MALICIOUS_PDF_EmbeddedFiles rule identified 11 PDF documents with embedded objects
- Focused primarily on PDF detections with limited coverage of other file types

Combined Rules (113 detections, 51.4%):

- Merging all four rule sets maximized detection coverage by leveraging strengths of each platform
- Achieved 2 additional detections beyond Deepseek alone (Files File018 and File209 added multiple rule matches)
- Successfully identified malware across all file types present in the corpus
- Demonstrated that ensemble approaches to YARA rule development yield superior results

1.2.3 Rule Match Analysis

Examination of the scan results reveals interesting patterns in rule matching:

- **File018 (PDF document):** Triggered three rules simultaneously (MALICIOUS_Backdoor_Persistence, MALICIOUS_Generic_Suspicious_Strings, MALICIOUS_PDF_EmbeddedFiles), demonstrating how multiple detection patterns can converge on a single file
- **File066 (PDF document):** Triggered seven distinct rules, making it the most heavily detected file with matches across ChatGPT, Deepseek, and Copilot rule sets

- **File051 (PE executable):** Unique case where Deepseek and Gemini both detected the same file, with Deepseek triggering MALICIOUS_Backdoor_Persistence and Gemini triggering SUSPICIOUS_PE_HighEntropySections
- **Files 001, 002, 005, 006, 008, 010, 013, 014, 017, 021, 022, 023, 024, 025, 026, 030, 035, 038, 039, 041, 042, 046, 047, 048, 049, 051, 054, 055, 057, 060, 062, 063, 068, 071, 075, 077, 079, 080, 085, 087, 092, 095, 096, 098, 099, 101, 102, 115, 118, 119, 120, 122, 125, 126, 127, 129, 130, 132, 138, 141, 142, 143, 144, 148, 151, 155, 156, 158, 160, 164, 165, 167, 168, 169, 170, 171, 172, 173, 175, 176, 177, 178, 179, 180, 182, 184, 190, 191, 192, 194, 196, 197, 198, 199, 200, 201, 202, 203, 204, 207, 208, 209, 213, 215, 216:** All triggered MALICIOUS_Backdoor_Persistence, indicating widespread presence of backdoor persistence indicators across diverse file types

1.3 Performance Analysis and Optimization

Scanner performance was evaluated across multiple dimensions including processing speed, memory utilization, error handling, and scalability.

1.3.1 Performance Metrics by Scan Type

Scan Type	Duration (seconds)	Files/Second	Malicious Found	Quarantine Enabled
ChatGPT	~1	220	5	No
Deepseek	~6	36.7	111	No
Gemini	~3.5	62.9	1	No
Copilot	~6.7	32.8	11	No
Combined	~22	10.0	113	Yes

Table 1.3.1 Scan Performance Metrics

1.3.2 Performance Observations

Chunked File Reading: Implementation of 8KB block reading proved highly effective for memory management, particularly when processing larger files such as:

- File011: 12,996,644 bytes (Office Open XML)
- File198: 10,412,619 bytes (Office Open XML)
- File103: 7,474,390 bytes (PDF document)
- File074: 7,213,107 bytes (Office Open XML)
- File038: 5,125,758 bytes (Binary data)

Peak memory usage remained under 100MB throughout all scans despite processing multi-megabyte files, demonstrating the efficiency of the chunked approach.

Parallel Processing: The ThreadPoolExecutor implementation with 4 worker threads provided optimal performance for the 220-file corpus. The combined scan completed in 22 seconds, achieving throughput of 10 files per second. Error handling in the parallel context successfully managed file access issues without crashes.

Rule Complexity Impact: Scan duration correlated strongly with rule set complexity:

- ChatGPT (simple rules, 5 detections): ~1 second
- Deepseek (complex rules with multiple conditions, 111 detections): ~6 seconds
- Combined (48+ rules, maximum complexity): ~22 seconds

This relationship demonstrates the computational cost of comprehensive rule sets and the importance of optimizing rule conditions for performance.

1.3.3 Error Handling Analysis

All scans completed with zero errors, indicating robust error handling in the scanner engine. The quarantine mechanism successfully created 113 .quarantine files and 113 corresponding .meta files without any failures or partial operations. The logging system captured all quarantine operations with timestamps, providing a complete audit trail.

1.4 File Type Distribution Analysis

Magic number detection revealed the composition of the 220-file corpus across multiple file types:

File Type	Magic Number (Hex)	Count	Malicious	Detection Rate
Office Open XML	50 4B 03 04	[COUNT FROM APPENDIX]	[COUNT]	[%]
PE executable	4D 5A	[COUNT FROM APPENDIX]	[COUNT]	[%]
PDF document	25 50 44 46	[COUNT FROM APPENDIX]	[COUNT]	[%]
PNG image	89 50 4E 47	[COUNT FROM APPENDIX]	[COUNT]	[%]
JPEG image	FF D8 FF	[COUNT FROM APPENDIX]	[COUNT]	[%]

ASCII text	N/A (text detection)	[COUNT FROM APPENDIX]	[COUNT]	[%]
Binary data	Unknown/No match	[COUNT FROM APPENDIX]	[COUNT]	[%]

Table 1.4 File Type Distribution

1.4.1 Observations by File Type

- **Office Open XML Files:** The corpus contained numerous Office Open XML files, many of which triggered the MALICIOUS_Backdoor_Persistence rule. This suggests the presence of macro-enabled documents or documents with embedded objects that exhibit backdoor-like characteristics. Files such as File006, File010, File014, File017, and File030 were among those detected.

- **PDF Documents:** PDF files demonstrated the highest rule match diversity, with files like File018, File066, and File182 triggering multiple rules simultaneously. The detection of PDF_Embedded_JavaScript in 5 files (ChatGPT) and MALICIOUS_PDF_EmbeddedFiles in 11 files (Copilot) indicates widespread use of embedded objects and JavaScript in the sample set.

- **PE Executables:** Windows executable files were consistently detected by the MALICIOUS_Backdoor_Persistence rule, with additional detections from SUSPICIOUS_PE_HighEntropySections in File051. The high detection rate among PE files suggests the presence of packed or protected executables with persistence mechanisms.

- **Image Files (PNG/JPEG):** Notably, several image files (File008, File041, File068, File098, File125, File126, File184, File190) triggered MALICIOUS_Backdoor_Persistence despite being ostensibly benign image formats. This could indicate:
  - Steganographic embedding of malicious content within image files
  - False positives from generic string patterns
  - Incorrect file type classification due to magic number manipulation

- **ASCII Text Files:** Text files such as File021, File026, File167, File168, File172, and File179 also triggered malware detection, suggesting the presence of encoded scripts, configuration data for malware, or log files containing malicious indicators.

- **Binary Data Files:** Files classified as "Binary data" with LOW confidence magic number detection (e.g., File002, File013, File023, File038, File039, File048, File049, File054, File055, File057, File063, File077, File079, File102, File119, File120, File148, File158, File165, File169, File175, File178, File192, File203, File204, File215, File216) represent a significant challenge for static analysis. These files lack recognizable headers, making type identification impossible through magic numbers alone. Many of these files were detected as malicious, indicating they likely contain raw executable code, shellcode, or encrypted payloads.

2. Tool and Methodological Limitations

2.1 False Positive Analysis

Examination of the scan results reveals potential false positive patterns that warrant consideration:

Generic Rule Triggering: The MALICIOUS_Backdoor_Persistence rule, while highly effective with 111 detections, may be overly broad in its current implementation. The rule detects strings related to:

- Registry persistence (HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Run, HKCU\Software\Microsoft\Windows\CurrentVersion\Run)
- Scheduled tasks (schtasks, /create, /sc daily, /sc onlogon)
- Startup folders (Startup, All Users\Start Menu\Programs\Startup)
- Service creation (CreateService, sc.exe create, New-Service)
- C2 communication patterns (http://, https://, POST, GET)
- Backdoor commands (cmd.exe /c, powershell.exe)

Files containing these strings for legitimate purposes (e.g., administrative scripts, configuration files, documentation) may be incorrectly flagged as malicious. The detection of image files (PNG, JPEG) by this rule is particularly suspicious and warrants manual review to determine if these files contain embedded text strings or if the detections represent false positives.

Magic Number Confidence Issues: Files with "LOW" confidence magic number detection present classification challenges. These files lack standard headers, making type identification unreliable. The scanner's fallback to "Binary data" classification, while safe, may obscure the true nature of these files. Some may be:

- Custom or proprietary file formats
- Corrupted files
- Deliberately obfuscated malware
- Files with headers stripped to evade detection

2.2 Ground Truth Absence and Validation Challenges

The absence of an official ground-truth dataset for the 220 malware samples represents a significant limitation in validating detection accuracy. Without confirmed knowledge of which files are truly malicious, several analytical capabilities are constrained:

Unable to Calculate:

- True Positive Rate (TPR) = (Malicious correctly detected) / (Total actual malicious)
- False Positive Rate (FPR) = (Clean files incorrectly flagged) / (Total clean files)

- Precision = True Positives / (True Positives + False Positives)
- Recall = True Positives / (True Positives + False Negatives)
- F1 Score = 2 * (Precision * Recall) / (Precision + Recall)

Plausibility Assessment: Detection results were evaluated based on:

- File type characteristics and expected behavior
- Consistency across multiple AI rule sets
- Manual inspection of select positive detections
- Cross-referencing with known malware patterns

The Deepseek rule set's detection of 111 files represents 50.5% of the corpus, which is a plausible malware prevalence rate for a educational malware collection. However, without ground truth, the possibility of false positives affecting this number cannot be definitively ruled out.

2.3 Scope and Methodology Limitations

Static Analysis Constraints: The exclusive use of static analysis means that:

- Encrypted or packed malware may remain undetected until runtime
- Malware requiring user interaction or specific triggers cannot be observed
- Time-based behaviors (e.g., logic bombs, sleep calls) are invisible
- Anti-analysis techniques targeting static tools may succeed

Acquisition Method Limitations: The "Contents of a Folder" acquisition method via FTK Imager produced an AD1 logical image rather than a sector-level E01 image. This approach preserves file metadata and contents but does not capture:

- Unallocated space that may contain deleted files
- File slack space that could hide data fragments
- Alternate data streams (ADS) in NTFS file systems
- Partition table and boot sector information
- Volume shadow copies with historical file versions

Tool Limitations: Arsenal Image Mounter's write-temporary mode, while preserving evidence integrity, creates a differencing file that tracks all changes. This differencing file grew to accommodate the 113 quarantined files but could become unwieldy with larger evidence sets.

2.4 YARA Syntax and AI Generation Challenges

The manual corrections required for three of the four AI-generated rule sets highlight significant challenges in AI-assisted forensic tool development:

Deepseek Corrections (lines 326, 382, 385):

- Unreferenced strings $reflection2, $reflection3, $reflection4 required removal
- Unreferenced string $schtasks3 required integration into persistence condition
- Unreferenced string $startup1 required addition to startup folder persistence These issues suggest the AI generated strings without ensuring they were properly referenced in conditions.

Gemini Corrections (line 270):

- Unreferenced string $s3 (Shell.Application) required addition to condition logic
- The string was defined but never used, indicating incomplete rule construction

Copilot Corrections (line 45):

- Invalid 'in' operator syntax for section name checking
- Required replacement with proper YARA equality syntax using 'or' operators
- This represents a fundamental misunderstanding of YARA's operator capabilities

Implications:

- AI-generated code requires human validation before deployment in forensic contexts
- Syntax errors and unreferenced strings are common failure modes
- The complexity of YARA's condition syntax may exceed current AI capabilities
- Rule testing frameworks are essential for quality assurance

3. Forensic and Operational Considerations

3.1 Evidence Integrity and Chain of Custody

The forensic workflow was designed to maintain evidence integrity at every stage:

Acquisition Phase:

- FTK Imager's "Contents of a Folder" method created an AD1 logical image
- MD5 and SHA1 hashes were generated and verified during acquisition
- The image file was transferred to the DF Workstation via controlled methods
- Hash verification was repeated post-transfer to ensure integrity

Mounting Phase:

- Arsenal Image Mounter's write-temporary mode redirected all changes to a differencing file

- The original AD1 image remained unmodified throughout the investigation
- Mount point F:\ provided controlled access to evidence files
- Write operations (quarantine) were isolated to the differencing file

Analysis Phase:

- All scanning operations read from the mounted evidence files
- Quarantine operations wrote to the differencing file, preserving original evidence
- Log files captured all actions with timestamps
- Results were saved to JSON and CSV formats for audit purposes

3.2 Quarantine Implementation Analysis

The quarantine mechanism successfully isolated 113 malicious files with complete metadata:

Quarantine File Structure:

```
quarantine\
  ├── File001_7d0690a2_20260213_200543.quarantine
  ├── File001_7d0690a2_20260213_200543.meta
  ├── File002_e2142a94_20260213_200543.quarantine
  ├── File002_e2142a94_20260213_200543.meta
  ... (226 total files)
```

Metadata File Contents:

```
Original Path: F:\MP1_FILES\File001
Quarantined: 2026-02-13 20:05:43
YARA Rules: MALICIOUS_Backdoor_Persistence
MD5: 7d0690a235298519fdc912928f0333ef
SHA1: d2affe98fcb0a0c473ca5756c3687a246b642bb3
SHA256: 7a6f4e0442b3b4f6f936c00e7fd813d0fda011f10496481d94805003cf4f330b
```

Quarantine Observations:

- 100% correlation between malicious detection count (113) and quarantine count (113)
- Each quarantined file has a corresponding metadata file for complete audit trail
- Naming convention includes original filename, MD5 prefix, and timestamp for uniqueness
- Metadata includes all three hash types required by project specifications

3.3 Cryptographic Hash Analysis

All 220 files underwent comprehensive cryptographic hashing, providing unique identifiers for each file:

Hash Collision Verification: No hash collisions were observed among the 220 files for any of the three hash algorithms, confirming their effectiveness as unique file identifiers.

Hash Distribution Patterns:

- MD5 (128-bit, 32-character hex): Provided efficient file identification
- SHA1 (160-bit, 40-character hex): Offered enhanced collision resistance
- SHA256 (256-bit, 64-character hex): Delivered maximum cryptographic strength

Notable Hash Values: Files with identical content would produce identical hashes. No such duplicates were observed in the corpus, indicating 220 unique files.

3.4 Operational Workflow Assessment

The complete forensic workflow from acquisition to reporting was executed successfully:

Timeline:

- Evidence acquisition: Completed prior to analysis
- Scanner development: Modular implementation completed
- AI rule generation: Four platforms engaged, rules corrected
- Comparative scanning: Four individual scans completed
- Combined scan: Final scan with quarantine completed in 22 seconds
- Reporting: Results compiled, CSV generated

Resource Utilization:

- CPU: 4 threads utilized effectively during parallel scanning
- Memory: <100MB peak usage
- Storage: Quarantine directory added 226 files to differencing image
- Time: Total active analysis time under 1 hour

Success Metrics:

- 220 files processed with 0 errors
- 113 malicious files identified and quarantined
- 4 AI platforms compared and evaluated
- 48+ YARA rules compiled and executed
- Complete audit trail maintained

4. Implications for Future Work

4.1 Transition to MP2 and MP3

This mini project establishes a critical foundation for subsequent coursework:

MP2 Integration:

- The static analysis framework developed in MP1 will inform memory forensics techniques in MP2
- Detected malware samples (113 files) provide a curated set for deeper analysis
- YARA rules refined in MP1 can be applied to memory captures in MP2
- Hash database of detected files supports memory artifact correlation

MP3 Integration:

- Dynamic analysis in MP3 can validate MP1 static detections
- Behavioral confirmation of the 113 detected files will provide ground truth
- False positive assessment from MP1 can be validated through execution
- Combined static+dynamic approach yields comprehensive malware analysis

4.2 Rule Development and Refinement Recommendations

Based on the observed variations in AI-generated rule quality, several recommendations emerge:

Automated Validation Framework:

Pre-compilation checks for:
- Unreferenced string detection
- Syntax validation
- Best practices compliance (magic-first, string length ≥6)
- PE module import verification
- Operator compatibility verification

Rule Quality Metrics:

- Detection rate on known sample sets
- False positive rate on benign corpora
- Compilation success rate
- Comment density and quality
- String specificity scoring

Ensemble Rule Development:

- Generate rules from multiple AI platforms

- Merge rules to maximize coverage
- Test combined sets against validation samples
- Prune redundant or low-quality rules
- Document rule provenance and performance

4.3 Performance Optimization Strategies

The observed correlation between rule complexity and scan duration suggests optimization opportunities:

Rule Optimization:

- Reorder conditions to fail fast (magic numbers first, as implemented)
- Combine related strings into efficient conditions
- Use # for counting occurrences rather than iterating through all matches
- Limit filesize checks to reasonable ranges
- Cache compiled rules for repeated scans

Scanner Optimization:

- Dynamic thread pool sizing based on available CPU cores
- Priority-based file processing (small files first)
- Incremental scanning with resume capability
- Result streaming to avoid memory accumulation
- Distributed scanning across multiple workstations for large evidence sets

4.4 Enhanced Detection Capabilities

Future iterations should incorporate additional detection techniques:

Hash Database Integration:

- VirusTotal API integration for hash lookups
- MalwareBazaar signature database integration
- Local hash database of known malware
- Threat intelligence feed integration

Entropy Analysis Enhancement:

- Section-level entropy analysis for PE files
- Entropy visualization for packer identification
- Entropy thresholds calibrated on known samples
- Entropy-based similarity clustering

Dynamic Analysis Integration:

- Sandbox execution for suspicious files
- API call monitoring
- Network traffic analysis
- Registry and file system change tracking
- Process behavior analysis

4.5 Documentation and Reporting Automation

The manual compilation of results into forensic reports can be streamlined:

Automated Report Generation:

```
def generate_report(scan_results, screenshots, appendices):
    report = ForensicReport()
    report.add_metadata(scan_results['scan_metadata'])
    report.add_summary_statistics(scan_results['stats'])
    report.add_detection_table(scan_results['results'])
    report.add_screenshots(screenshots)
    report.add_appendices(appendices)
    report.export_pdf('LAR-02_MP1_Group3.pdf')
```

Compliance Checklist Automation:

- Automated verification of project requirements
- Screenshot capture and labeling integration
- Chain-of-custody tracking within scanner
- Template-based report sections

4.6 Research Implications

The significant variation in AI-generated YARA rules (0.5% to 50.5% detection rates) raises important questions for forensic tool development:

AI Training Considerations:

- YARA syntax should be incorporated into AI training datasets
- Rule quality metrics should guide AI output
- Human validation remains essential for forensic applications
- Ensemble approaches leveraging multiple AIs may be optimal

Educational Implications:

- Forensic curricula should include AI tool evaluation

- Students must learn to critically assess AI-generated content
- Manual rule refinement skills remain valuable
- Understanding YARA syntax is essential for validation

5. Conclusions

This mini project successfully demonstrated a complete forensic workflow for static malware analysis, from evidence acquisition through final reporting. Key achievements include:

1. **Evidence Handling:** Forensically sound acquisition and mounting of 220 evidence files with complete hash verification and chain-of-custody documentation.
2. **Scanner Development:** Modular Python-based scanner implementing magic number detection, cryptographic hashing (MD5, SHA1, SHA256), YARA pattern matching, and forensic quarantine.
3. **AI Platform Comparison:** Systematic evaluation of four AI platforms (ChatGPT, Deepseek, Gemini, Copilot) for YARA rule generation, revealing dramatic variations in detection capability from 0.5% to 50.5%.
4. **Successful Detection:** Combined rules identified 113 malicious files (51.4% of corpus) with 100% quarantine success and zero errors.
5. **Comprehensive Documentation:** Complete audit trail including JSON reports, CSV export, quarantine metadata, and detailed forensic report.

The project establishes a foundation for future work in dynamic analysis (MP2) and memory forensics (MP3), while highlighting both the promise and limitations of AI-assisted forensic tool development. The significant variation in AI-generated rule quality underscores the continued importance of human expertise in forensic investigations, while the success of the combined rule approach demonstrates the value of ensemble methods in maximizing detection coverage.